Aalto University
School of Science

Master's Programme in Computer, Communication and Information Sciences

Scope Attribution for Static UI Transition Graph Extraction in Single-Page Applications

Duy Khanh Tran

Master's Thesis
2026

Aalto University
School of Science

Author Duy Khanh Tran

Title Scope Attribution for Static UI Transition Graph Extraction in Single-Page Applications

Degree programme Computer, Communication and Information Sciences

Major Computer Science

Thesis supervisor and advisor Prof. Petri Vuorimaa

Date November 7, 2026 **Number of pages** 100+11 **Language** English

Abstract

A user interface (UI) transition graph represents an application's screens and the navigation between them. For single-page applications, such graphs usually come from execution. Static extraction avoids execution, but its losses have not been measured construct by construct.

This thesis measures what static extraction recovers from Angular applications and why it loses the rest. It proposes scope attribution, a dataflow analysis that computes the screens from which each navigation site can fire.

An extractor was applied to six open-source Angular applications and checked with five evaluation methods. Every unrecovered navigation construct was classified.

Every screen readable from route tables has a node. Only 25 constructs yield deterministic transitions under stated assumptions, largely because the study defines them strictly. Under the study's classification order, the largest loss category is navigation outside components that routes render directly. Scope attribution recovers a per-application median of 40.2 % of these constructs (66.7 % pooled, most of them in the largest application). Almost all become possible transitions, mostly through mechanisms from earlier work. The added step, for services, guards and effects, places 36 further cited source locations, 2.0 % of all placed source locations.

On a reference that applies the analysis's own relations to one application's source, the added step trades precision for recall. A comparison with Rima et al.'s tool shows a similar trade-off and no overall ranking, with this extractor far less precise on the largest application. Both comparisons rest on small samples read by an artificial intelligence model, and one subgroup was formed after reading. Mutation testing only sanity-checks the mapping between source and witnesses.

Route tables give static extraction a starting point, but attribution and target resolution remain necessary. The main corpus results are in-sample because the extractor was developed on these applications. The graph's benefit to users is not measured.

Keywords static analysis, UI transition graph, single-page application, Angular, dataflow analysis, program analysis, software testing

Use of AI tools

I wrote the text of this thesis and made the revisions myself. Artificial intelligence (AI) tools were used to support implementation, the search for relevant projects and papers, and review. I assessed the suggestions and decided which changes to make. The research questions, methodological choices and interpretation of the results are my own.

An AI coding assistant supported the implementation of the algorithms presented in this thesis, including the extractor and evaluation code. I reviewed the implementation and remain responsible for its correctness.

AI tools also helped identify candidate open-source projects and research papers. I assessed their relevance and selected the projects and sources used in the study.

AI was used to review the implementation, evaluation and manuscript, and to identify possible errors, inconsistencies and gaps in the arguments. I evaluated that feedback and revised the text myself. The corrections made in response to reviews are recorded in Section 4.3.2.

AI-assisted readings of application source code also form part of the evaluation evidence, including the Alfresco structural reference, the must-tier readings and the transition-pair judgements described in Chapters 4 to 6. Their role and limitations are stated there; they are not treated as independent human validation.

I take responsibility for the final text, implementation, results and conclusions.

Contents

Abstract **2**

Use of AI tools **3**

Contents **4**

Abbreviations and terms **7**

1 Introduction **10**
1.1 Motivation 10
1.2 Research questions and scope 12
1.3 Thesis structure 12

2 Background **14**
2.1 User-interface transition graphs 14
2.2 Static extraction for Android 16
2.3 Navigation models for web applications 16
2.4 Dynamic state models for single-page applications 16
2.5 Static analysis of JavaScript and TypeScript source 18
2.5.1 Analyses over a source-level syntax tree 18
2.5.2 Abstract interpretation of JavaScript 18
2.5.3 What TypeScript's types contribute 19
2.5.4 Frameworks as the practical obstacle 19
2.6 Formal foundations 20
2.6.1 Modal transition systems 20
2.6.2 Soundiness 20
2.6.3 Dataflow analysis and abstract interpretation 20
2.7 Angular routing, dependency injection and reactive state 21
2.7.1 Routes, components and navigation sites 22
2.7.2 Dependency injection 22
2.7.3 Reactive state 24
2.8 Reactive programming and the limits of static dataflow 24
2.8.1 Propagation order and glitch freedom 24
2.8.2 Reactive code and the tools developers have 25
2.9 Mutation testing 26
2.9.1 The coupling effect 26

2.9.2 Mutation for JavaScript and web applications 26
2.9.3 Reporting a mutation experiment 27
2.10 Summary and the gap 27

3 Static navigation analysis and scope attribution 29
3.1 Graph model and soundness 29
3.2 Scope attribution 34
3.2.1 The defect being addressed 34
3.2.2 Scope 35
3.2.3 Input relations 37
3.2.4 Guards as program points 38
3.2.5 Boundary 39
3.2.6 Anchored edges 39
3.2.7 Extensions not evaluated 40
3.3 Implementation 40
3.4 Summary 42

4 Evaluation setup and research material 43
4.1 Corpus selection and validity 43
4.1.1 Selection 43
4.1.2 Validity of an open-source corpus 43
4.2 Metrics 44
4.3 Evaluation methods 50
4.3.1 Preconditions 54
4.3.2 Changes to the protocol 54
4.3.3 Reproducibility 55
4.4 Summary 55

5 Results 56
5.1 The two levels of loss (RQ1) 56
5.2 Where static extraction fails, and why (RQ2) 58
5.3 Scope attribution (RQ3) 59
5.3.1 Checking the scopes against the running interface 63
5.3.2 What the attribution discriminates 64
5.3.3 What each mechanism contributes 65
5.3.4 Agreement with a structural reference 66
5.3.5 Agreement with the applications' own suites 67

5.3.6 What scope attribution adds 69
5.3.7 On the applications of Rima et al. 70
5.3.8 The tool of Rima et al. on the same applications 72
5.4 Change sensitivity (RQ4) 74
5.5 What the losses were (RQ5) 75
5.5.1 The cause of each defect class 75
5.5.2 Construct accounting 77
5.5.3 The final tier and the node set 78
5.6 Dead links, found from the source 79
5.7 Summary 79

6 Discussion **81**
6.1 Key findings 81
6.2 Limitations and threats to validity 83
6.3 Soundiness and declared approximations 87
6.4 Transfer to other frameworks 89
6.5 Future work 90
6.6 Summary 90

7 Conclusion **92**

References **93**

A Corrections made to the protocol **101**

B Declared approximations **107**

Abbreviations and terms

Abbreviations

AGPL	GNU Affero General Public License
AI	artificial intelligence
Ajax	asynchronous JavaScript and XML
API	application programming interface
DOM	Document Object Model
GPL	GNU General Public License
HTML	Hypertext Markup Language
HTTP	Hypertext Transfer Protocol
ICU	International Components for Unicode (message format)
JSON	JavaScript Object Notation
JSP	JavaServer Pages
JSX	JavaScript XML, the markup syntax of React
k8s	Kubernetes
LGPL	GNU Lesser General Public License
MIT	Massachusetts Institute of Technology (the MIT licence)
MPL	Mozilla Public License
NgRx	Angular state-management library built on RxJS
RQ	research question
RxJS	Reactive Extensions for JavaScript
SPA	single-page application
SPIN	Simple Promela Interpreter, a model checker
UI	user interface
URL	Uniform Resource Locator
XML	Extensible Markup Language
ZAP	Zed Attack Proxy, a web security scanner

Terms

Anchored edge An edge stored once at its scope, $\top$ or $\mathit{subtree}(r)$, instead of once per screen (Section 3.2.6).

Attribution level One of the nested configurations at which the ablation extracts the corpus. The levels *owned*, *nesting*, *rooted*, *outlet* and *full* are nested, each adding mechanisms to the one before. *Class* replaces the call graph of *full* with a per-class join (Section 4.2 and Figure 3.1).

Attribution loss A lost navigation construct in one of the two attribution categories. It has no owning component, is owned by a class that is not a component, such as a service or guard, or lies in a component that no route renders directly

(Table 4.3).

Clean citation A witness citation that the witness check reads and passes. Citations at over-approximated sites, and those whose target the check cannot read, are neither clean nor failures (Section 5.5.3).

Demotion Lowering a site from must to may because a condition of Table 3.1 holds (Section 3.1).

Dynamic target, dynamic sink A nonscreen node, a screen's `u_` sink, for an unresolved destination. Edges into it have modality unknown (Section 3.1).

Modality The confidence label of an edge: *must* if it is claimed to fire for every input on which the enclosing code executes, *may* if it is possible but conditional, and *unknown* if it could not be determined (Section 3.1).

Navigation construct A `routerLink` binding in a template, or a call in class code to `Router.navigate` or `Router.navigateByUrl`, outside tests. It is *in scope* unless it changes only the query string, targets a named outlet or names no destination (Section 4.2).

Outlet descent Giving the component of route r, and the components it renders, the scope $\mathit{subtree}(r)$, the screens of r and its descendants (Section 3.2.3).

Placed screen A screen node, rather than an unplaced origin, as the source of an edge. Recovery counts must and may edges from placed screens (Section 5.5.2).

Red-team panel An adversarial review, run with the declared AI assistant, in which several reviewers read the graph and the source and each try to find errors (Appendix A).

Relative recall The share of the real transition pairs in the union of two tools' pairs that one tool finds (Section 5.3.8).

Route-order assumption Assumption A1: a catch-all in a separately registered route table is matched after every route of the others (Section 3.1).

Runtime route oracle The evaluation method that loads each built application in a headless browser and compares the addresses at which the Angular router mounts a component with the screens of the graph (Table 4.4 and Section 5.5.3).

Scope The set of screens from which a site can fire. The extractor emits edges from the *computed scope*, whereas the *true scope* is a property of the application that this thesis does not establish (Section 3.2.2).

Screen A graph node for a route that renders a component, identified by the concatenated paths of the route and its ancestors (Section 3.1).

Screen without a node A leaf address at which the runtime route oracle finds a mounted component but the graph has no node (Section 5.5.3).

Shell The components from the root to the primary router outlet. They and everything they render get the scope $\top$ of all screens (Section 3.2.3).

Site, navigation site A source location from which a navigation can fire. Scope attribution gives each site a scope (Section 3.2.2). The construct accounting counts one site per navigation construct (Section 5.5.2). The ablation counts every location that a witness cites, so one construct can contribute more than one site there (Section 5.3.3).

Static reader A tool that enumerates screens from the route configuration in the source: the extractor or the independent route parser (Section 5.1).

Structural reference The (site, screen) pairs read from an application's source under the analysis's own relations, independently of its implementation, so agreement with it does not establish the true scope (Section 3.2.2).

Unplaced origin A nonscreen node, prefixed `o_`, for a component whose sites could not be assigned to a screen. Its edges are labelled may (Section 3.1).

Witness, witness citation The file and line of the construct that generates an edge, or an observation identifier. Other constructs producing that edge become merged witnesses. Witness citations count every cited location, repeats included (Sections 3.1 and 5.5.3).

Witness check The evaluation method that re-parses the witness of every must edge at the cited line, compares the construct's kind and target with the edge, and tests the destination against the independent route set (Table 4.4).

1 Introduction

A user interface (UI) transition graph represents the screens of an application and the transitions that user actions cause between them. Each edge records the triggering event, a guard and the destination. The graph answers reachability queries and helps identify the screens that a change to a component affects.

Transition graphs support test generation and exploration for Android [79, 39, 14] and crawling-based web testing [50, 51]. Linked to source code and tests, they also support change impact analysis [4, 12].

1.1 Motivation

Dynamic extraction executes an application and records the states and transitions it observes. Its coverage therefore depends on test data, authentication and the exploration budget, and each new version needs another run. Static extraction instead derives the graph from source code without executing the application. It can run on every commit, but it recovers only the constructs its analysis supports and has to approximate the targets it cannot resolve.

The two approaches complement each other. Much of the navigation in the applications studied here is behind authentication guards. A static graph can give a crawler candidate screens, transitions and guards, and execution can confirm the transitions the crawler observes. This thesis evaluates static extraction alone and leaves the combination to future work (Section 6.5).

Navigation graphs also have uses beyond model-based testing. GPTDroid runs a question-and-answer loop with a large language model over the current mobile screen and reports higher activity coverage and more detected bugs than its best baseline on 93 applications [46]. AutoDroid gives such a model an application-specific transition graph built by offline dynamic exploration [73], and LLMDroid uses a model to guide exploration towards uncovered behaviour [70]. Related work explores web interfaces with vision-language models [72], and a survey covers large language models in software testing [71]. Coding agents could use navigation structure too, because SWE-bench tasks require coordinated edits across the files of a repository [31] and the interface an agent uses to work on a repository affects its performance [76]. This thesis does not test whether navigation graphs benefit any of these consumers.

Static extraction is established for Android. Window transition graph analyses derive windows and transitions from bytecode and link callback control flow to the widgets that register each handler [79, 78, 77]. For the web, early navigation models combined templates and configuration with crawling or server execution [62, 9], and a patent describes a fully static method [69].

Modern single-page applications, by contrast, have mostly been modelled dynamically. Crawljax builds a state flow graph by firing events against the running Document Object Model (DOM) [50]. FeedEx and FRAGGEN improve the exploration but still build the model from execution [51, 74]. Of the tools listed in a survey of model-based

web testing covering 2014–2025 [36], every one whose model construction was examined for this thesis executes the application. Framework-specific static analyses remain few. One builds a dependency graph of React components that leaves out routing [38], and three others extract navigation:

- Oshima and Kishi extract Angular page transitions for model checking. Their stated restrictions exclude nested routes and route parameters [56].
- Rima et al. extract Angular workflows as a coverage denominator for generated end-to-end tests [64].
- Zhang et al. analyse compiled single-page application bundles to check access-control policies against reachable screens [81].

Section 2.4 compares these approaches.

Early Ajax (asynchronous JavaScript and XML) applications had no declarative route tables, so crawlers observed navigation through DOM states. In Angular, React Router and Vue Router, route tables, template navigation and service calls now appear in source code, and a route can also name its guards [27]. Static extraction can take these declarations as its inputs. Existing studies evaluate extraction against hand-built references on their chosen applications, but they do not measure the losses construct by construct.

This thesis therefore builds an Angular extractor to measure how much navigation is recovered and to classify the losses. Navigation in code outside the components that routes render directly, such as services, guards and shared components, can be recovered only if each navigation site is attributed to the screens it belongs to. For example, a navigation call in a service method can fire on any screen whose components call that method, directly or through other services. To attribute such sites, the thesis proposes *scope attribution*, a formal model specified independently of the extractor. It is a dataflow analysis over screen sets that computes for each navigation site a scope, a candidate set of the screens from which the site can fire. Section 2.7 gives a running example. The model contains the attribution rules of the earlier Angular extractions of Oshima and Kishi and of Rima et al. as levels. It adds a member-level service call graph, a fallback join and stand-ins for guards and effects, which attribute the navigation in services, guards and effects to screens. The thesis measures what each level adds (Section 5.3.3). Most of the recovery comes from the earlier mechanisms, and the added levels contribute a smaller part.

Besides scope attribution, the extractor handles nested, parameterised and lazily loaded route trees (Section 5.3.6 compares it with the approach of Oshima and Kishi). It was developed and evaluated on the same corpus, so its measurements describe these applications and are not estimates of performance on Angular applications in general. Section 4.3.2 records the corrections made during development, and Section 6.2 discusses what they imply. Chapter 6 separates gaps in the implementation from limits of the approach.

1.2 Research questions and scope

The aim is to measure how far static extraction of a UI transition graph gets on real Angular applications, why it fails where it does, and what scope attribution adds. Five research questions (RQs) follow from this aim. Section 4.2 defines their measures, and Chapter 5 answers them.

RQ1 How much of the navigation in Angular applications can static analysis recover as deterministic transitions?

RQ2 Where does static extraction fail, and why?

RQ3 How does scope attribution affect the recovered share and precision of static navigation analysis?

RQ4 Does the extracted graph respond to source changes in the constructs it represents, as change impact analysis requires?

RQ5 Are the losses a property of the applications or of the tool?

The questions are answered with the final version of the extractor on six open-source Angular applications pinned to fixed commits (Chapter 4). The thesis does not claim completeness. It states a soundness property for the edges labelled must, which are claimed to fire for every input on which their enclosing code executes. The property is conditional on stated assumptions (Claim 3.1). The thesis also lists what the analysis approximates (Appendix B). It does not measure the graph's effect on any consumer, and it evaluates Angular alone. Because the extractor was developed on the corpus, its figures on that corpus are in-sample.

1.3 Thesis structure

Figure 1.1 summarises the study.

Chapter 2 reviews transition graphs, related extraction methods, Angular and the analysis foundations. Chapter 3 defines the graph model, its conditional soundness claim and scope attribution, and describes the implementation. Chapter 4 describes the corpus, metrics, evaluation methods and protocol corrections. Chapter 5 answers the research questions and reports statically detected dead-link candidates. Chapter 6 interprets the results and discusses limitations, framework transfer and future work. Chapter 7 summarises the contributions.

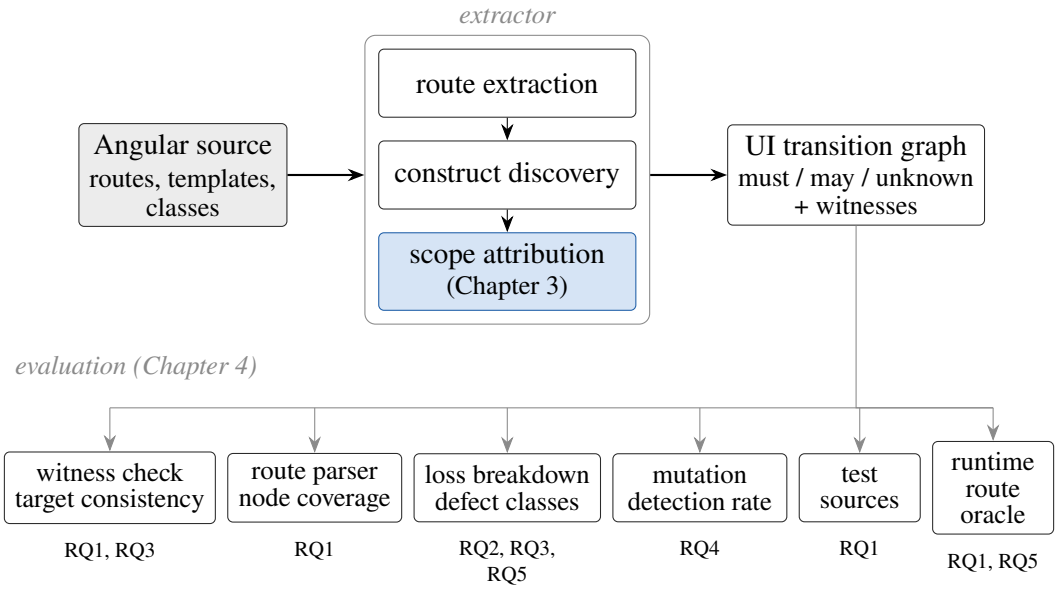


Figure 1.1: Overview of the study: extractor, scope attribution, the main measurements and the research questions they answer.

Note. The extractor turns Angular source into a UI transition graph; scope attribution is the analysis this thesis adds. The loss breakdown classifies what the other methods find missing rather than being one of the five methods itself.

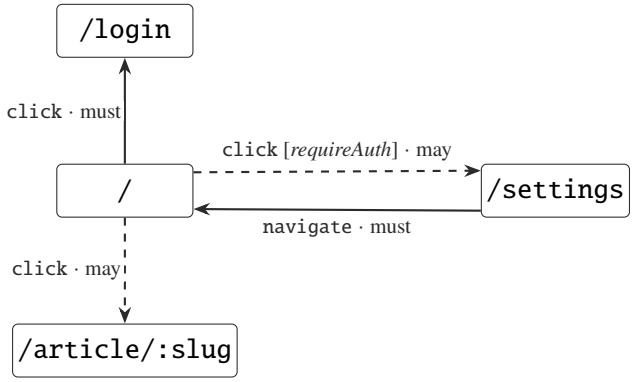


Figure 2.1: A small UI transition graph in the model of this thesis, with must and may edges.

Note. Each edge carries an event, an optional guard in brackets, and a modality: *must* always fires when its code runs, *may* is conditional (here on a route guard or a partly known target). In the measured graphs the unknown modality marks only edges into a sink for destinations that only the run time knows.

2 Background

This chapter defines transition-graph terminology in Section 2.1. Sections 2.2 to 2.4 review static extraction for Android, navigation models for web applications and dynamic models for single-page applications (SPAs). The chapter then covers source-level static analysis (Section 2.5), formal foundations (Section 2.6), Angular mechanisms (Section 2.7) and reactive programming's limits on static analysis (Section 2.8). Section 2.9 reviews mutation testing, the method that Chapter 4 specifies and Chapter 5 applies to answer RQ4. Section 2.10 summarises the chapter and identifies the research gap.

2.1 User-interface transition graphs

A UI transition graph is a directed graph whose nodes represent user-visible application states and whose edges represent transitions caused by user actions. Figure 2.1 illustrates the model used here.

Research communities use different names for these graphs. In Android static analysis it is a *window transition graph* [79], *activity transition graph* [14] or *screen transition graph* [39]. For Ajax (asynchronous JavaScript and XML) crawlers it is a *state flow graph* over DOM states [50]. In web engineering it is a *navigation model* or *navigation state machine* [62, 18]. This thesis uses *UI transition graph* for all of them, while recognising differences in state definitions and graph construction.

Two distinctions matter more than names. Extraction is either *static*, from source code or binaries without execution, or *dynamic*, from observed runs. Edges may or may not record *guards*, the conditions under which transitions are available. Table 2.1 classifies the reviewed work by both distinctions.

Table 2.1: Prior work closest to this thesis, by target, extraction mode, guard representation and evaluation basis.

Work	Target	Static	Guards	Evaluation basis
Yang et al. [79, 77]	Android windows	✓	✗	20 apps; manual on 6
Yang et al. [78]	Android callbacks	✓	context	manual vs. dynamic
StoryDroid [14]	Android activities	✓	✗	hand-built, 10 apps
GoalExplorer [39]	Android screens	hybrid	✗	manual, 36 apps
ProMal [45]	Android windows	hybrid	✗	40 apps, F1 score
UI-CTX [44]	Android widgets	✓	branches	2000 labelled widgets
Ricca and Tonella [62]	server-rendered pages	semi	✓	manual
WebUml [9]	server-rendered pages	hybrid	✓	case study
Tkachuk and Rajan [69]	Struts, JavaServer Pages (JSP)	✓	outcome	not stated
Crawljax [50]	Ajax DOM states	✗	✗	manual
FeedEx, FRAGGEN [51, 74]	DOM states	✗	✗	coverage
Kurant [38]	React components	✓	n/a	F1 score, no routing
Oshima and Kishi [56]	Angular pages	✓	✗	6 apps, faults
Rima et al. [64]	Angular workflows	✓	conditions	7 apps, tests run
Zhang et al. [81]	SPA routes, from bundles	✓	✗	4 apps (2 SPA), vs. Zed Attack Proxy (ZAP)

Note. Evaluation basis is a reference, a metric or the tool compared with, as each work states it.

2.2 Static extraction for Android

Most static transition-graph research concerns Android. Yang et al. introduced the window transition graph, with activities, dialogs and menus as window nodes and UI-thread callback transitions as edges. The analysis models the active window stack, including back-button behaviour [79, 77]. A related analysis builds a callback control-flow graph over handler–widget pairs [78]. StoryDroid builds an activity transition graph from `startActivity` calls [14]. GoalExplorer combines a statically derived screen transition graph with dynamic exploration [39]. UI-CTX finds the code context of a widget's behaviour with a backward data-flow analysis from the handler to the call that retrieves the widget [44].

These approaches attribute navigation to a screen through object resolution. A handler is registered on a widget that belongs to exactly one window at run time, so resolving the widget's identifier, usually by constant propagation of `getId()`, identifies that window [78]. ProMal illustrates the limitation: imprecision in the reference analysis aliases several widgets, assigning handlers to the wrong window and producing infeasible transitions. Its authors combine static analysis with runtime traces and a learned model [45], using dynamic evidence rather than refining static attribution.

2.3 Navigation models for web applications

Web navigation models were first extracted from server-rendered sites. Ricca and Tonella modelled pages, frames and forms connected by links and form submissions. Edges carry boolean conditions on input values, derived semi-automatically using a crawler and analyst annotations [62]. WebUml combines static analysis of Hypertext Markup Language (HTML), script and server-side pages with server runs to recover a state diagram. It derives guards through control-flow analysis of server code [9]. A patent assigned to Fujitsu describes a fully static method for Struts and JavaServer Pages (JSP) sources [69]. SiteHopper builds abstracted navigation state machines on the fly for model checking [18].

These approaches target server-rendered frameworks, where pages correspond to files and links to anchor elements. They do not address client-side routers, component trees or dependency injection. Section 2.4 reviews work that does.

2.4 Dynamic state models for single-page applications

With Ajax and client-side frameworks, model construction moved to run time. Crawljax executes user events, compares the DOM before and after each event, and builds a state flow graph over equivalence classes of DOM states [50]. Later work changed the exploration strategy but not the model's source [51, 74]. The survey by Kertusha et al. reviews web testing research from 2014 to 2025 [36]. Every listed model-based tool whose construction method was examined for this thesis executes the application to

build its model. The reasons are historical: Ajax applications lacked declarative route tables, one uniform resource locator (URL) could represent many states, and source code did not represent routes explicitly, although JavaScript static analysis existed [29].

Client-side routing frameworks declare route tables as data, navigation as attributes or method calls naming routes, and guards as named functions attached to routes. The literature search found four static analyses of such frameworks: three analyse framework-specific source code and one a compiled bundle. Kurant extracts a React component rendering graph but does not address routing [38].

The other three address routing. Oshima and Kishi extract Angular page transitions and verify them with the model checker SPIN (Simple Promela Interpreter) [56]. They analyse `RouterModule` declarations and each component's navigable paths, resolving parent–child relationships among root, routed and custom child components. Paths are attributed to the containing routed component, and the root component's own paths to every page. Section 5.3.3 measures this combination as the *rooted* level, which Section 4.2 defines with the other levels. The approach excludes `children` and route parameters and does not treat data-binding changes as page transitions. The paper does not state how it handles destinations that are nonliteral expressions, so no treatment is assumed here. Screens are identified by routed-component names instead of addresses. Guards, lazy loading and named outlets are not modelled. Navigation reached through a method call or dispatch is outside the attribution mechanism. Without modalities, always-available and guarded transitions have identical representations. The evaluation uses four constructed applications and two published applications of 1824 and 6893 lines, with six and twenty-three pages. It reports whether the intended models were produced and hand-injected transition faults were detected in constructed variants of the first sample. Losses on applications the extractor cannot fully analyse are not measured, and measuring them is the subject of this thesis.

Two recent works statically analyse client-side router configuration for other purposes. Rima et al. parse Angular sources and templates into a graph of navigation, interaction sites and execution conditions, including guards. They compile workflows enumerated from the graph into browser tests and use it as a coverage denominator [64]. This account draws on the publisher's abstract, the replication package's operational specification `softscanner-sw/softscanner_fa/docs/paper/approach.md`, and the anonymised manuscript `docs/paper/main.pdf` in the same repository. The published chapter was unavailable and may differ from both documents. Both describe attribution beyond containment: active components include those activated by the route and its ancestors, closed under composition, plus bootstrap components. This covers the attribution of the *rooted* and *outlet* levels in Section 5.3.3. Handlers are traced into same-class helpers to depth one and into the first `subscribe` callback. The specification excludes router-outlet modelling, provider resolution beyond module and root providers, and service calls. The implementation nevertheless traces calls into services whose class names match a router or navigation pattern, to depth one. Navigation inside these services is attributed to their callers' screens.

Zhang et al. parse compiled JavaScript bundles, using declared routes as vertices

and navigation calls as edges. They check an access-control policy against screens reachable from an authorised screen. Two of their four subjects are multi-page applications, analysed by crawling instead. Their graph is an over-approximation and is not narrowed. The paper does not explain source-screen attribution, and guards remain unmodelled runtime conditions [81]. Rima et al. compare their output with a ground truth derived from their pipeline's output and annotated by hand. Neither work classifies losses by construct or measures them across a corpus. The problem is therefore neither new nor settled, and Section 2.10 identifies the remaining gap.

2.5 Static analysis of JavaScript and TypeScript source

The analysis in Chapter 3 reads Angular source with the TypeScript compiler application programming interface (API), so it works on a source-level abstract syntax tree rather than on a compiled artefact. This section reviews that style of tooling, the abstract interpretation of JavaScript, what the declared types of a TypeScript program contribute, and the evidence that frameworks limit such analyses more than the language does.

2.5.1 Analyses over a source-level syntax tree

Analysing the syntax tree of the source, instead of bytecode or an intermediate representation, is a long-established style. Baxter et al. detect exact and near-miss code clones by comparing abstract syntax trees with a variant of the common-subexpression detection used in compilers. They apply the method to a C system of about 400 000 lines [8]. Working on the tree keeps the analysis close to what the developer wrote, so each result can be reported at a source location.

SPOON turns this style into a reusable library. It exposes a metamodel of Java source and lets an analysis or a transformation be written in plain Java, without modifying a compiler or learning a separate formalism [58]. The TypeScript compiler API plays the same role for TypeScript. It provides a typed syntax tree, a symbol table and a type checker over an ordinary program. With them, an analysis can follow a constant reference in a route table to its definition and read the declared type of an injected field. The cost of the style is that the analysis sees the program as written, including the parts that a compiler later rewrites or erases.

2.5.2 Abstract interpretation of JavaScript

Lattice-based analysis of JavaScript has a formal treatment. Jensen et al. present an analysis based on abstract interpretation that infers detailed and sound type information for the full language, including its object model and built-in functions. They propose it as a foundation for developer tools [29]. Their work shows that the framework of Section 2.6 applies to JavaScript at the level of individual values. That problem is considerably harder than scope attribution, which uses the same machinery over a

much coarser domain of screens, defined in Section 3.2.2, instead of abstractions of heap objects.

2.5.3 What TypeScript's types contribute

Angular applications are written in TypeScript, so an analysis can read declared types instead of inferring them. The type system is deliberately unsound. Bierman et al. give a precise definition of it [10], and Richards et al. describe its annotations as unsound by design [63]. A declared type is therefore evidence about the program and gives no guarantee about its execution.

Empirical studies point the same way. Gao et al. took bugs that had been fixed in the histories of JavaScript projects, added type annotations to the buggy versions and checked whether a type checker would have reported them. Flow and TypeScript each detected about 15% of these bugs [25]. Bogner and Merkel mined 604 GitHub projects, 299 in JavaScript and 305 in TypeScript. The TypeScript projects showed better code quality and understandability, but neither lower bug proneness nor shorter bug resolution time. Heavier use of the `any` type correlated weakly with worse measurements [11]. The declarations that describe untyped libraries can themselves be wrong. TSCheck, which compares a declaration file with the library it describes, found 142 errors in the declarations of 10 libraries [22].

For an analysis of Angular code, a declared type narrows the set of candidate implementations of a call, and the call graph described in Sections 3.2.3 and 3.3 relies on that narrowing. The type does not establish that the call reaches exactly one candidate. An unresolved or ambiguous type must therefore widen the result instead of being trusted. In the graph model, this widening is one source of the may tier.

2.5.4 Frameworks as the practical obstacle

In practice, the framework around the code limits a static analysis of a client-side application more than the language does. Madsen et al. observe that JavaScript applications depend on large frameworks and libraries whose behaviour an analysis must know. Hand-written models of that behaviour are tedious to produce, incomplete and sometimes wrong. Their alternative infers library behaviour from the way the application uses it, combining pointer analysis with such a *use analysis* [48].

A later analysis by Madsen et al. addresses implicit control flow directly. Their *event-based call graph* for Node.js represents the listeners that run when an event fires. The analyses built on it find event-handling defects, such as events that no listener handles and listeners registered too late [49]. An Angular application poses the same kind of problem, with the router, template event bindings and observable subscriptions in place of the Node.js event emitter.

Besides the navigation analyses of Oshima and Kishi (Section 2.4) and Rima et al. [64], one static analysis of a framework in the Angular family is relevant here. Aurebesh, by Ocariza et al., defines a consistency model for web applications built on

a model–view–controller framework [55]. It detects identifier and type inconsistencies between models, views and controllers, with a reported recall of 96.1% and a precision of 100%, and it found 15 real bugs. Aurebesh targets AngularJS 1.x, whose template language, component model and injector differ from those of the Angular versions analysed here. The two frameworks share the problem of relating an identifier in a template to the code that defines it. Aurebesh solves that problem for a consistency check, does not address navigation and reports no transition graph.

2.6 Formal foundations

The analysis builds on three ideas from the literature: a transition system whose edges carry a modality, a stated position on what soundness can mean for a practical analysis, and dataflow analysis as the computational framework.

2.6.1 Modal transition systems

The *modal transition system* of Larsen and Thomsen labels each transition as either required, a *must* edge, or merely permitted, a *may* edge [42]. The graph model in Chapter 3 borrows the names *must* and *may* with a different meaning. There they are confidence labels on a conditional runtime claim, not obligations on an implementation, and no refinement property of modal transition systems is used. The model adds *unknown* for edges whose modality could not be determined. The thesis claims soundness for the must tier alone, and the evaluation distinguishes the tiers because may edges have weaker supporting evidence.

With one edge kind, an unresolved transition must either be included, reducing precision, or omitted, reducing recall. Modalities record the evidence available, letting consumers select a tier. Chapter 5 therefore reports the tiers separately.

2.6.2 Soundiness

Livshits et al. argue that practical static analyses are inevitably sound only for a language subset. They recommend declaring approximated features rather than claiming general soundness [47]. Section 3.1 states this thesis's position. Sections 2.5 to 2.7 explain the limits: TypeScript annotations are not guarantees, framework configuration determines call targets, and reactive firing conditions are not written in the program text.

2.6.3 Dataflow analysis and abstract interpretation

Kildall expressed many program analyses as fixpoint computations over lattices of abstract values. A join combines information, which propagates along graph edges until stable [37]. A finite-height lattice and monotone transfer functions ensure termination and an order-independent result. Cousot and Cousot developed the general

theory of abstract interpretation [16], and Nielson et al. provide a textbook account [54]. Scope attribution in Chapter 3 uses this framework, with abstract values and join defined in Section 3.2.2.

An instance needs a lattice of bounded height and a propagation graph. When unresolved calls leave the propagation graph incomplete, the result omits information in a way the framework itself cannot detect.

2.7 Angular routing, dependency injection and reactive state

The analysis in Chapter 3 must account for Angular-specific mechanisms, described in the framework documentation [27]. Listings 2.1 to 2.3 introduce a running example.

Listing 2.1: A route table with a guarded route, nested children, a lazy route table and a constant path.

```
export const routes: Routes = [
  { path: '', component: HomeComponent },
  { path: 'settings', component: SettingsComponent,
    canActivate: [requireAuth],
    children: [{ path: 'profile', component: ProfileComponent }] },
  { path: 'admin', loadChildren: () => import('./admin/admin.routes') },
  { path: paths.about, component: AboutComponent },   // constant, not a
    literal
];
```

Listing 2.2: The root template: the header sits next to the outlet.

```
<app-header />                <!-- shell: rendered on every screen -->
<router-outlet />             <!-- the active route renders here -->

<!-- header.component.html -->
<a routerLink="/settings">Settings</a>
@if (user()) { <a [routerLink]="['/profile', user().name]">Me</a> }
```

Note. Its links are present on every screen but belong to none of them.

Listing 2.3: Navigation from an injected service.

```
@Injectable({ providedIn: 'root' })
export class AuthService {
  private router = inject(Router);
  private session = inject(SessionStore);
  logout(): void {
    this.session.clear();
    this.router.navigate(['/login']);
  }
}
```

Note. The call site belongs to no component, so its screens can only be found through the components that call `logout`.

2.7.1 Routes, components and navigation sites

Angular declares screens through a tree of route objects. Each has a `path` pattern and a `component`, `redirectTo` target or `children` list. A route with children but no component provides a path prefix and no screen. Routes can load lazily through `loadChildren` or `loadComponent`. Guards include `canMatch`, `canActivate`, `canActivateChild` and `canDeactivate`, and they return a boolean, a redirect, or a Promise or Observable of either. When `canMatch` returns false, the router tries the next matching route rather than cancelling navigation. Resolvers fetch data that the router awaits before activation. Paths need not be string literals: resolving `path: routes.about.path`, for example, requires tracing the constant to its definition.

Each screen is rendered by a component: a class and template. Templates instantiate other components by selector and use `<router-outlet>` to mark where the router inserts the active child component. Components above the outlet form the application shell, present on every screen; those below a routed component are present only on that screen. Structural directives such as `@if` and `*ngIf` make nesting conditional. Components can also be created programmatically at run time.

A template expresses navigation with the directive `routerLink`, whereas class code calls `Router.navigate` or `Router.navigateByUrl`. The option `relativeTo` permits targets relative to the current route. Navigation sites can occur in components, injectable services, guards, interceptors and resolvers. Their source screens are not explicitly declared and must be derived.

2.7.2 Dependency injection

Angular creates services and supplies them to the components and services that declare them as dependencies. This mechanism is dependency injection, a form of inversion of control: a class does not construct or look up its collaborators but receives them from a container [32, 24]. A dependency is requested by type or by an injection token. The same token can be bound to different implementations in different injector scopes, through `useClass`, `useExisting` or a factory function.

The pattern is widely used outside Angular. Yang, Tempero and Melton defined operational criteria for dependency injection and detected its use in the source code of 34 open-source Java applications [75]. The configuration that the pattern introduces can itself be analysed statically. Laigner et al. catalogue 12 dependency injection anti-patterns and detect them with a static analyser at a relative recall of 92.19%, and at least 9 of the anti-patterns occur frequently in open-source and industrial projects [40]. Dependency injection configuration can therefore be read from the source, but it does not state which binding a given call site receives.

For a static analysis, injection turns a simple method call into a dispatch problem. A call such as `this.nav.goHome()` cannot be linked to the method it invokes until the type of `nav` and the binding of that type in the relevant injector are resolved. Linking it is an instance of the call graph construction problem known from object-oriented languages, where a virtual call must be resolved to its possible targets. Class hierarchy

Table 2.2: Static analyses that model framework behaviour explicitly, and the mechanism each of them resolves.

Work	Framework	Mechanism modelled
F4F [67]	Java web frameworks	reflective calls, from a specification generated from code and configuration
JackEE [3]	Java enterprise frameworks	components and entry points, declaratively
Jasmine [13]	Spring	dependency injection and aspect-oriented programming
Aurebesh [55]	AngularJS 1.x	identifiers shared between template and controller

analysis resolves such a call from the declared type and its subclasses [17]. Rapid type analysis narrows the result to the classes that the program instantiates [5], and points-to analysis narrows it further by tracking which objects each variable may hold [2]. Grove and Chambers place these algorithms in a common framework that trades precision against cost [28]. For JavaScript, Feldthaus et al. show that an approximate call graph, which tracks functions by the property names they are stored under, is precise enough for practical tools [23].

Dependency injection adds a further difficulty, because the binding of a type to an implementation is written in framework configuration instead of ordinary code. The Java literature has measured what this costs a general analysis and what recovers the loss. Table 2.2 lists the analyses that model framework behaviour explicitly. Sridharan et al. report that the heavy use of reflection in frameworks makes static taint analysis ineffective, and that adding support for one framework at a time does not scale. Their tool, F4F, instead generates a specification of framework behaviour from the application code and its configuration files. An unmodified taint engine consumes the specification, and the approach found 525 new issues across nine benchmarks [67].

Antoniadis et al. report that the major Java analysis frameworks offer almost no support for enterprise applications. Their analysis, JackEE, captures the concepts that enterprise frameworks share, such as components and entry points, in a declarative and largely framework-independent form. Against the most precise conventional analysis, it measures about four times higher completeness and about a sixfold speedup [3]. Landman et al. set a bound on what such modelling can achieve. In a literature review and corpus study of Java reflection, they find that reflection cannot be ignored in 78% of projects and that resolving it requires unsound assumptions [41].

The closest analogue to resolving Angular's injectors is Jasmine, by Chen et al. [13]. Spring's core mechanisms, dependency injection and aspect-oriented programming, rely on dynamic language features, so a conventional call graph of a Spring application misses edges. Jasmine extends the Soot framework to make the call graph of a Spring

program more complete. On micro-benchmarks and 18 real Spring programs it improves on tools built with Soot and with Doop. The structural problem is the one that Angular poses. An injected field has a declared type, the binding of that type lives in framework configuration, and an analysis that does not read the configuration cannot connect the call to a body.

Angular differs from Spring in where the configuration lives and how far its effect reaches. Provider arrays belong to modules, routes or components, so the same token can resolve to different implementations depending on where the injection happens. Resolution is relative to an injector, which makes the provider hierarchy an input to the explicit framework model of Section 3.2.3.

2.7.3 Reactive state

Recent Angular versions include signals and effects [26]. A signal is a value that changes over time, and an effect is a computation that runs again whenever a signal it read changes. Signals and effects are an instance of reactive programming, which goes back to functional reactive programming [21] and has been surveyed by Bainomugisha et al. [6]. In a reactive system, the runtime records the dependencies between values as they are read and propagates updates without explicit calls in the program text.

A navigation call placed inside an effect therefore fires under a data dependence in the reactive graph instead of a control-flow condition. Static analysis can identify the signals the effect reads but cannot in general determine when the effect runs, and this thesis does not attempt to bound it. Angular applications also use observables of Reactive Extensions for JavaScript (RxJS) for the same purpose, in subscriptions to router events, in responses from the server and in state containers. Section 2.8 reviews what the reactive programming literature establishes about this boundary.

2.8 Reactive programming and the limits of static dataflow

In reactive programming, the runtime decides when a computation runs. This section reviews the propagation discipline that reactive runtimes implement and the evidence on how well developers can reason about reactive code with tools.

2.8.1 Propagation order and glitch freedom

A reactive program is a graph of values in which an update to one value must propagate to the values that depend on it. Cooper and Krishnamurthi embed dynamic dataflow in a call-by-value language, so that an expression evaluates to a node in a dataflow graph [15]. They name the central hazard. A *glitch* occurs when a signal is recomputed before all the signals it depends on are up to date, which exposes an intermediate value that no consistent state of the program ever had. Their runtime prevents glitches by giving each signal a height and updating signals in order of increasing height. The order is topological, so the dependency graph must be acyclic.

Drechsler et al. carry glitch freedom into a distributed setting. Their change-propagation algorithm needs no central knowledge of the dependency topology and avoids propagating updates that cannot change a result [20]. An Angular signal graph records its dependencies as they are read (Section 2.7.3), so it is the kind of structure over which these propagation algorithms are defined.

For a static analysis, the order in which reactive computations run is a property of the graph that the runtime builds, and that graph is built from the reads that actually happen. Static analysis can over-approximate the set of signals an effect may read. The number of times the effect runs and the application state at each run depend on which writes occur and in what order, so they cannot in general be determined exactly from source. They could still be over-approximated, but this analysis does not attempt it.

2.8.2 Reactive code and the tools developers have

Empirical studies separate reading reactive code from reasoning about its behaviour. Salvaneschi et al. ran an experiment with 127 subjects that compared reactive code with object-oriented code using the Observer pattern. Comprehension was significantly better for the reactive version [66]. The same group identifies tool support as the weak side. A conventional debugger shows a call stack, whereas the developer's question concerns a dataflow graph, so reactive programs need debugging concepts of their own. Their Reactive Inspector implements such concepts and outperformed conventional debugging in a controlled experiment [65].

Studies of RxJS report the same gap. Banken et al. find that conventional debuggers offer no support for reactive libraries such as RxJS, and that developers fall back on inserting console logging. Their RxFiddle visualises the structure and the values of an RxJS dataflow, and in an experiment with 111 developers it let them finish debugging tasks faster [7]. Alabor and Stolze interviewed 10 professional engineers and observed 4 of them debugging RxJS code. The engineers relied on manually added trace logging instead of specialised tools. The study is a small workshop paper, so it suggests the practice without establishing it [1].

The surface that a tool would have to cover is large. Zimmerle et al. mined the use of the reactive libraries RxJava, RxJS and RxSwift on GitHub and Stack Overflow. They found that 95.2% of the libraries' operators are used in practice [82].

Angular navigation analysis inherits this limit. An effect or subscription exposes a read set in source, but that set does not exactly determine firing. Chapter 3 therefore specifies *unknown* edges with the read set as symbolic guard, following the soundiness position of Section 2.6 for unresolved constructs. The prototype emits no such edges for effects or subscriptions. Its unknown edges have destinations it cannot resolve to screens.

2.9 Mutation testing

RQ4 asks whether the graph responds to changes in the source constructs it represents. Mutation testing is applied here in an unusual direction. The mutants are inserted into the application source, and the artefact whose response is measured is the extracted graph instead of a test suite. This section reviews the technique and the evidence for its central assumption, its use on JavaScript and web applications, and the reporting practice that Chapter 4 follows.

2.9.1 The coupling effect

Mutation testing introduces small syntactic changes, called mutants, into a program and measures the share that a test suite detects. It rests on the *coupling effect* stated by DeMillo et al.: tests that detect simple errors are often effective against more complex errors as well [19]. Without this assumption, a mutant would say nothing about the faults that developers actually make.

Jia and Harman provide the standard survey of the field, covering approaches, tools and empirical results. Their trend analysis suggests that the techniques and tools are reaching maturity [30]. Papadakis et al. survey the following decade of work on the fundamental problems of the technique and list those that remain open [57]. The evidence that mutants are a usable proxy for real faults comes from Just et al. On 357 real faults in 5 open-source applications, mutant detection correlated significantly with real-fault detection, independently of code coverage [33].

2.9.2 Mutation for JavaScript and web applications

Mutation analysis has been adapted to JavaScript and to web applications, where the cost of generating and running mutants is a central concern. Mirshokraie et al. use static and dynamic analysis to direct mutant generation at code that is error-prone or likely to affect the output. The guidance reduces the cost and the number of equivalent mutants, and the authors propose mutation operators specific to JavaScript and the web [52]. The journal version adds a ranking of functions. In it, more than 93% of the generated mutants are non-equivalent, and more than 75% of the surviving non-equivalent mutants lie in the top 30% of the ranked functions [53].

Praphamontripong and Offutt target faults in the way web components interact with each other, covering client-to-server and server-to-server interaction. They define mutation operators for such faults and implement them in a tool called webMuJava. A small case study suggests that the resulting tests find web application faults, and the work appeared at a workshop [59].

Stryker is a mutation testing framework for JavaScript and TypeScript, with further implementations for C# and Scala. Its documentation lists more than 30 supported mutations and a design that works with any test runner [68]. No peer-reviewed publication describing Stryker was found, and the project cites none. Stryker counts a mutant as detected when a test fails or times out with the mutant active [68]. This

thesis instead scores a mutant by whether the extracted graph shows the change the mutation predicts, so Chapter 4 specifies its operators, sampling and scoring.

2.9.3 Reporting a mutation experiment

Papadakis et al. also give best-practice advice on the use of mutation in empirical studies [57]. Chapter 4 follows it and states the operators, mutant generation and sampling procedures, exclusions and per-operator detection rates instead of only an aggregate figure.

In each of the works above, the mutants evaluate a test suite for the application. The use of mutation in this thesis belongs instead to change impact analysis. Arnold and Bohner compare impact analysis approaches by how they are used, how they work internally and how effective they are [4], and the collection they edited is the standard reference for the area [12]. Chianti is a classic tool of this kind. It decomposes the difference between two versions of a program into atomic changes and reports, for each affected test, the changes that affect it [61]. Lehnert's taxonomy classifies impact analysis approaches by finer criteria than the earlier frameworks [43].

A transition graph whose edges carry source witnesses supports the same kind of query at the level of screens. The mutation experiment checks whether the graph changes where its source changes. It is a sanity check of the mapping from witnesses to source and does not measure change-impact accuracy (Section 4.2).

2.10 Summary and the gap

Static web navigation extraction has largely targeted server-rendered frameworks, where pages are files and links are anchors. Client-side routing frameworks have mostly been modelled dynamically. This literature search found three static extractions of client-side navigation graphs. Two analyse framework-specific source code and one a compiled bundle, each for a different consumer. Oshima and Kishi establish the problem addressed here, and Section 2.4 sets out what their approach leaves out. The other two works use their graphs without narrowing them: as a generated test suite's coverage denominator [64], or as an over-approximated reachability relation for access-control checks [81]. Rima et al. already analyse nested and lazy route trees, constrain edges with guards and trace router-service calls to a bounded depth. No earlier work found in this review combines cross-class and service attribution under provider bindings, which generalises that bounded tracing, with modalities that separate unconditional transitions from guarded or scope-uncertain ones, and with construct-level loss measurement on applications large enough to contain difficult constructs.

Android provides the closest static techniques. They resolve navigation's owning object, which usually identifies the window, and use runtime traces when resolution is imprecise. Fragments, background tasks and aliased widgets are exceptions handled with dynamic evidence. Web navigation sites often have no such object, which

prevents direct reuse of Android techniques.

Sections 2.5 to 2.9 identify established methods and remaining limits. JavaScript abstract interpretation is established [29], as is source-level analysis over typed syntax trees [8, 58]. Explicit framework models recover call edges missed by general analyses [67, 3, 13]. Aurebesh addresses AngularJS 1.x template–controller consistency rather than navigation [55]. TypeScript types narrow dispatch targets without resolving them, because the type system is deliberately unsound [10, 63]. Reactive execution depends on a runtime-built graph, limiting what source analysis determines [15, 7]. Mutation testing measures model responses to source changes [30, 57], and on web applications it has been used to evaluate test suites [52, 59]. For static client-side navigation graphs, it has been applied only to hand-injected faults in constructed applications [56]. This review found no operator-level mutation study of such graphs.

3 Static navigation analysis and scope attribution

This chapter defines the graph model and its soundness claim (Section 3.1), scope attribution for RQ3 (Section 3.2), and its partial implementation (Section 3.3).

3.1 Graph model and soundness

Definition 3.1 (UI transition graph)**.** *A UI transition graph is a labelled transition system* (S, Σ, G, M, E) *with*

$$E \subseteq S \times \Sigma \times G \times M \times S.$$

S *is the finite set of* screens*, the states of the system and the nodes of the graph.* Σ *is the finite set of* event kinds *that can be raised at a screen, such as the activation of a* `routerLink` *or a call to* `Router.navigate`*.* G *is the set of* guards*, each held as the symbolic source text of the condition under which an edge may be taken.* M *is the set of* modalities*, which has exactly three elements,*

$$M = \{must, may, unknown\},$$

and whose elements are confidence labels: a modality states how firmly the extractor asserts an edge, and never denotes a screen, a route or a navigation target. An edge $(s, \sigma, g, m, s') \in E$ *reads: at screen* s*, event* σ *under guard* g *leads to screen* s'*, asserted with confidence* m*. Both the source* s *and the target* s' *are elements of* S*, the first and last components of the tuple. Constructs in the source that agree on source, event kind, guard and target yield the same edge. That edge carries the strongest modality among them, with* M *ordered unknown* $<$ *may* $<$ *must. So* E *holds at most one modality for each* (s, σ, g, s')*. The paragraph on witnesses below says how constructs are traced.*

Nodes outside the model. The extractor emits two nonscreen node kinds outside the edge relation. An *unplaced origin* represents a component and navigation sites that could not be assigned to a screen. A *dynamic target* represents an unresolved destination. These preserve unplaced sites for the loss counts in Chapter 5 and unresolved destinations that may still occur at run time. Section 5.3 counts both kinds. Of 13830 edges, 550 reach a dynamic target. Another 71 leave an unplaced origin and are labelled *may*, so Chapter 5 includes them in that tier despite their nonscreen sources. An edge to a dynamic target records outgoing navigation without identifying the destination screen. Claim 3.1 and must conversion (RQ1) concern only edges between screens.

Modalities. A *must* edge is claimed to fire for every input on which the enclosing code executes. A *may* edge is possible but conditional. *Unknown* marks an edge whose modality could not be determined, including unresolved destinations. In the measured graphs it labels exactly the edges to a dynamic target, the `u_` sink (Section 5.5) that records events with unresolved targets. Such edges occur in 5 of 6 applications, and

the census rejects any graph in which the unknown edges and the edges to nonscreen targets differ. Unknown edges count towards totals but towards no rate. The model also assigns *unknown* to navigation inside signal effects (Section 3.2.5), but this version emits none.

The artefact also records numeric *confidence*: 1 for must, at most 0.6 for may, and 0 for dynamic targets. This numeric confidence is outside the model, and all figures use modalities.

The tool stores guards as source text and does not evaluate them, which avoids representing application state.

Screens and routes. Not every configured route contributes a screen to S. Write *Routes* for the routes the application declares and $component(r)$ for the component that route r renders. Then

$$S = \{\, screen(r) \mid r \in \mathit{Routes} \text{ and } component(r) \text{ is defined} \,\},$$

where $screen(r)$ is the node for route r, identified by its full path: the concatenated paths of r and its ancestors. Two route kinds contribute no element of S:

- A route that declares `children` but no `component` is a path prefix only (Section 2.7). Its path is carried into the full paths of its children.
- A route whose only effect is a `redirectTo` renders nothing.

Angular rejects `redirectTo` combined with `children` or `component`, as verified in the configuration validator of the vendored Angular 18.2.14. Other routers differ: Vue Router applies a redirect only to the last matched record, so a redirecting parent renders its component for a matching child (Section 6.4).

Angular permits component-bearing routes with the same full path, distinguished at run time by `pathMatch` or `canMatch`. The model assumes no such duplicates. The extractor keeps only the first route, silently for Angular and with a note for Vue and React (Section 6.4). Different paths that render the same component produce distinct screens, which requires the attribution of Section 3.2.

Witnesses. Each static or runtime edge has a witness: its generating construct's file and line, or an observation identifier. When constructs produce the same edge, one becomes the primary witness and the others are recorded as merged witnesses (Section 4.2). The base graph requires witnesses, and manual edges stay in a separate overlay. The first evaluation method checks this mechanically.

The soundness claim. The thesis takes the soundiness position of Section 2.6.2 and states its claim relative to four assumptions. For each assumption the list gives a class of counterexample, a kind of program or execution on which the assumption fails and the claim can fail with it.

A1. Routes and route order. The routes are those the application declares, and a catch-all route in a separately registered route table is matched after every route

of the other tables. A route added at run time is a counterexample, and so is a registration order that puts such a catch-all earlier. Table B1 lists the matching order among the declared approximations. Most of the must tier rests on A1. Of the 159 must edges, 126 hold only under A1, and these come from 3 constructs in 2 applications. Without A1, 33 must edges remain, on 22 of the 25 constructs.

A2. Selectors. The component of a must edge's site is not placed by a join over an ambiguous selector, the fallback of the nesting relation when no import resolves the selector (Sections 3.2.3 and 3.3). A join that places a component on a screen that never renders it is a counterexample. Such joins occur in k8s dashboard (the Kubernetes dashboard) and bitwarden, but no final must edge cites a construct placed through one.

A3. Calls. In handling the event of a must edge, no call that the reader cannot resolve to a body in the application starts a navigation, and no work that the handler schedules for later starts one. A library call that navigates is a counterexample, and so is a callback that the handler schedules and that navigates.

A4. Execution model. The event is a plain primary activation that navigates the current browsing context. No other listener that the event reaches starts a navigation, and all synchronous evaluation that precedes the navigation completes normally. The counterexamples are an activation with a modifier key, navigation into another browsing context (Section 6.3), and a second listener on the element or an ancestor. An exception before the navigation is another, whether it comes from a call or from the evaluation of an expression.

The edges that rest on A3 and A4 are not counted, and neither assumption is checked against a run.

Claim 3.1 (Soundness of the must tier)**.** *Under assumptions A1 to A4, every must edge corresponds to a navigation that occurs for every input on which the enclosing code executes.*

A navigation occurs when the router starts navigating to the route of s'; any guard or resolver the navigation runs demotes the edge (Table 3.1). For a merged edge, the enclosing code is that of a construct that earns must. The claim is conditional on that code running, so it does not show that a user can reach the trigger. It states what the demotion rules are designed to meet and is not proved. Where an assumption fails, for a program or for one execution, a must edge need not correspond to a navigation that occurs, and Claim 3.1 does not cover that case. If the claim holds, the must tier is a lower bound, relative to A1 to A4, on the navigations that occur whenever their triggering code runs.

What the evaluation tests. The evaluation tests only a necessary condition of Claim 3.1, which limits the reading of Chapter 5. The witness check (Section 4.3) inspects every cited source location for agreement with the edge's destination. If the construct supporting a must label names another destination, the claim fails there.

Agreement does not establish the claim. Other cited constructs support may edges only and are not judged against it.

Demotion conditions. Classification as must requires navigation *for every input on which the enclosing code executes*. Table 3.1 lists all conditions that demote a site to may.

Table 3.1: Conditions that demote a navigation, must → may. Every condition yields may.

Group	Condition	Example construct
Guards and resolvers	The navigation runs a guard or resolver, whatever it would decide	`canActivate` on a route it enters; a resolver, which demotes as an `await` does
	The guard list cannot be brought back to an array	a guard list built at run time
Target and source addresses	The target is the address the source already displays	navigating to the current URL, which the router ignores or follows only when a parameter or the query differs (held at may as a matter of policy, the safe side)
	The source is never displayed at its own address	a redirect answers the source's address
	Two declared routes, or a route and a redirect, can answer the address	two routes matching one path
	The base a relative target resolves against cannot be read	a relative target with an unreadable base
	The URL names an auxiliary outlet	a URL with a named-outlet segment, whose route the adapter does not read
Conditional execution	The site is under a template control-flow construct	a control-flow block, a structural directive, a container that renders lazily
	The site is in a template fragment that need not render	`<ng-template>`, an International Components for Unicode (ICU) case
	The site is under a branch, loop or fallback in code	a branch, a loop, a `catch` clause, an optional call, a default value, a callback
	The site follows an exit in its block	after an `await`, an early `return` or `throw`, a `break` out of its block
	No handler calls the enclosing function unconditionally	a module-level function, or a member of its class
	The component is mounted conditionally	a component under a template condition on that screen; a route component behind a conditional outlet
A later navigation may cancel it	Code that runs after the site, before the event is handled, may navigate	the rest of the expression holding the navigation (the body of a call it is an argument of, a right operand, a branch of a conditional whose condition it is)
		the statements after it in its block and in every block around it, and a `finally` block around it

Group	Condition	Example construct
		what runs once its function returns: what each caller in the class does after calling it, handler or not; for a constructor or lifecycle hook, the hooks Angular calls after it

A navigation runs the guards the router runs. These are the `canMatch` of every route its address is matched through. They are also the `canActivate`, `canLoad` and resolvers of every route it enters (`canLoad` is taken as run every time). Next come the `canActivateChild` of every route above those. Last come the `canDeactivate` of every route it leaves. A route the source and the target share is re-run only if its parameters can change or its `runGuardsAndResolvers` policy directs. A shell or anchor edge is judged per screen and drawn per screen where the screens disagree.

A handler is a member that the framework calls itself: a lifecycle hook, or a member that `@HostListener` registers. A handler is also a member that the template binds to an event as a bare call outside any template condition.

The router cancels a navigation if another starts before it finishes. Later code may navigate through a navigation call, including in a branch or callback, or through a call to a member of the class, a function in the file or an injected service's method whose body navigates. An awaited navigation finishes before its function continues, so subsequent code in that function does not count. Callers continue at its first await, so their subsequent code still counts. Calls without resolvable application bodies are taken not to navigate, which is assumption A3 (Section 6.3).

Exceptions before the site are not counted, because almost any evaluation can throw and Claim 3.1 assumes normal completion in A4. An `await` is counted because request rejection is an ordinary outcome. Lazy-route fetches are not counted, since a chunk-load failure is a deployment fault.

How far the rules are checked. Unit tests cover each rule in both directions. A mutation sweep deletes each rule in turn and requires a test failure (Table A1). The tests share the extractor's interpretation, and no second classifier derives it independently. The declared artificial intelligence (AI) assistant checked final must-tier constructs against source with the extractor's decisions visible. This tests whether the decisions are defensible, but not whether an independent reading reproduces them (Section 5.5). An earlier review of a development tier found condition-rule defects that the witness check missed (Section 4.3.2). That check establishes only source-site existence and target agreement. The stronger claim rests on the demotion rules and remains independently unverified.

Scope of the claim. This thesis claims soundness for the must tier alone. A may witness establishes that a call site exists, not that its transition ever occurs. Target agreement at the witness location is meaningful for must edges and literal-target may edges, but not for over-approximated edges, where applying it produces spurious failures (Section 4.3.2, Table A1). No soundness property is claimed for may or

unknown.

The may tier as a design intent. Following modal transition systems [42], must and may together are intended to over-approximate every transition between declared screens. This intent is not an established property, because several rules under-approximate: selecting the first matching route, treating interpolation as one path segment, ordering catch-all routes last, placing dispatch and dialogs, and assuming calls without application bodies do not navigate. The graph can therefore omit intended transitions. An infeasible may edge costs precision, whereas an omitted actual transition is a defect. No completeness theorem is claimed. The end-to-end check (Section 5.3.5) measures omissions against the transitions of test suites. The structural references (Section 5.3.4) also count omissions, within the limit stated where Section 3.2.2 defines them. Section 6.3 sorts what is still missing by whether it lies inside the static boundary.

3.2 Scope attribution

Scope attribution uses five *input relations*. The evaluated analysis implements three of them:

- component nesting (N): which component's template instantiates which;
- router outlets: which components lie on the path from the root to an outlet, and below a route's component;
- the call graph with dependency injection (CG).

The other two, dispatch (D) and guards (T), are defined in Section 3.2.7 and are not implemented. The prototype replaces them with stand-ins. The derived relation *Mem* links components to their members.

The analysis computes a fixpoint over these relations, and the prototype then adds fallbacks after the fixpoint, one of them nonmonotone. Termination and the least-solution property of Section 3.2.2 hold for the fixpoint alone. The evaluation in Chapters 4 and 5 therefore covers three relations, two stand-ins and a fallback post-pass.

Guards returning `UrlTree`, the injector hierarchy, and navigation inside signal effects and observable subscriptions (labelled *unknown*, Section 3.2.5) are model-only features that the extractor approximates or omits, as listed in Section 6.3. A site assigned no scope by implemented relations contributes no edge and counts as "Attribution, service" in Table 4.3.

3.2.1 The defect being addressed

Without scope attribution, only sites directly in a screen's routed component are attributed to that screen, which explains three baseline symptoms when the attribution

pass is disabled (Section 3.3). Shared components contribute no edges: the shell places the header beside the outlet, outside every screen's subtree. A component mounted at several routes contributes one edge instead of one per route. Injected-service navigation contributes none. Every baseline application nevertheless has edges.

3.2.2 Scope

The lattice. Every navigation site is anchored at a *scope*: the set of screens from which it can fire. A scope is an element of the powerset $\mathcal{P}(S)$. The abstract domain of the analysis is $\mathcal{P}(S)$ ordered by inclusion, with set union as join,

$$X \sqsubseteq Y \iff X \subseteq Y, \qquad X \sqcup Y = X \cup Y,$$

the powerset lattice over the screens. Its least element is the empty screen set $\bot = \varnothing$. Its greatest element is $\top = S$, every screen.

True, computed and reference scope. The thesis distinguishes two scopes.

- $scope^*(x)$ is the *true scope* of a site x, the set of screens from which x really can fire. It is a property of the application, not of any analysis.
- $scope(x)$ is the *computed scope*, the set of screens the extractor emits edges from for x. In the model it is the fixpoint of Section 3.2.3. The prototype widens it with fallbacks that are not constraints of the model (Section 3.3).

A screen in $scope^*(x) \setminus scope(x)$ indicates a scope too small; one in $scope(x) \setminus scope^*(x)$ indicates a scope too large. This thesis establishes no true scope. For two applications, source inspection defines a third set, the *structural reference* $scope^R(x)$. The author constructed it for one application and the AI assistant for the other. It applies the analysis's own relations, independently of the implementation (Section 5.3.4). Agreement with $scope^R$ therefore shows that the implementation applies the specified relations. It does not show that the computed scope equals $scope^*$, and it cannot reveal a navigation that the relations themselves miss. Every scope reported elsewhere is a computed scope.

The empty scope. An empty computed scope can indicate two distinct situations.

- A site is *unreachable* when $scope^*(x) = \varnothing$: the code exists and no screen can reach it, so contributing no edge is the correct answer.
- A site is *unresolved* when $scope(x) = \varnothing$ because no rule of the analysis reaches it (a navigation in a service nothing is known to inject, or behind a construct the reader could not follow). The true scope is unknown.

Both yield $\bot$, which cannot distinguish them: an empty computed scope means only that the analysis has assigned no source screen to the site, not that the site is unreachable. The extractor therefore anchors a site in an unplaced component at

Table 3.2: Scope of each construct that can contain a navigation site.

Construct	Scope
component above the router outlet (the shell)	$\top$
component of route r	$\mathit{subtree}(r)$
component mounted at routes $r_1, \ldots, r_k$	$\mathit{subtree}(r_1) \sqcup \cdots \sqcup \mathit{subtree}(r_k)$
component rendered in the template of another component	join of the scopes of the components that render it. The prototype applies this row only to components that no route mounts (Section 3.3)
member (method, property initialiser, lifecycle hook) of a component	the scope of the component
method of an injectable service	join of the scopes of its callers
`canDeactivate` on route r	$\mathit{subtree}(r)$
Hypertext Transfer Protocol (HTTP) interceptor	$\top$

an unplaced-origin node recording its file, with edges labelled *may*. A site in any other component with empty scope contributes no edge, but a note records its file and reason. These traces keep such sites in the construct accounting of Section 5.5.2, which does not tell unresolved sites from unreachable ones.

Named elements. Three elements of the domain are named, because attribution anchors edges at them. Write $r' \succeq r$ for the reflexive ancestor relation on routes: r' is r itself or an ancestor of r in the route tree. The three elements are

$$\{\mathit{screen}(r)\}, \qquad \mathit{subtree}(r) \;=\; \{\, \mathit{screen}(r'') \in S \;\mid\; r \succeq r'' \,\}, \qquad \top \;=\; S,$$

and they form the chain

$$\bot \;\sqsubseteq\; \{\mathit{screen}(r)\} \;\sqsubseteq\; \mathit{subtree}(r) \;\sqsubseteq\; \top.$$

The chain is stated for a route with a component; for a prefix route only $\mathit{subtree}(r)$ is defined. The names anchor edges and do not restrict the domain. A component mounted at unrelated leaf routes has scope $\{\mathit{screen}(r_1), \mathit{screen}(r_2)\}$, none of the three named elements. Anchoring at $\top$ or $\mathit{subtree}(r)$ stores an edge once, not per screen. Table 3.2 assigns scopes to Angular constructs.

Computing the scopes. Services require more than component-tree traversal: their scopes join their callers' scopes, including calls from other services. The model computes a fixpoint over the relations in Section 3.2.3. Outlet-derived initial scopes propagate along N, *Mem* and *CG*. This core is a monotone dataflow problem [37, 54], with two properties that do not extend to the fallbacks in Section 3.3.

- Finite height gives termination. The height of $\mathcal{P}(S)$ is $|S|$, so the scope of one node can rise at most $|S|$ times before it reaches $\top$.

- Monotonicity gives the result. The iteration reaches the least solution of the constraint system, independent of the order in which nodes leave the worklist. The transfer functions are monotone because the only operation they apply to a scope is a join, and union is monotone in both arguments.

The height also bounds the cost of the fixpoint. A node's scope rises at most $|S|$ times, and each rise applies one transfer function to each pair that leaves the node in N, Mem, CG, D or T. The fixpoint therefore applies

$$O\big(|S| \cdot (|N| + |Mem| + |CG| + |D| + |T|)\big)$$

transfer functions, and the implemented core is the case in which D and T are empty. With scopes held as bit sets, one application is a union of two subsets of S and costs $O(|S|)$, so the running time is bounded by $O\big(|S|^2 \cdot (|N| + |Mem| + |CG| + |D| + |T|)\big)$.

The worklist handles cycles, including mutually recursive templates. The restrictions, fallbacks and stand-ins of Section 3.3 lie outside it.

3.2.3 Input relations

Three relations, in order of increasing difficulty, provide the input to the fixpoint.

- *Component nesting.* $A \rightarrow B$ when A's template instantiates B's selector, resolved within the correct import scope. Matching selector text globally is unsound, since two components may share a selector string in different scopes.

- *Router outlets.* Components on the path from the root to the outlet form the shell and take $\top$. Components below the component of route r take its scope $subtree(r)$. A named outlet carries a second mount, which makes auxiliary routes representable without separate treatment.

- *Call graph with dependency injection.* A call links to a method only after resolving the receiver's type and its binding in the relevant injector. Resolution follows class hierarchy analysis [17]. Angular's provider declarations are modelled explicitly as the source of bindings, as the framework-modelling literature recommends [67, 3]. Where a token admits several implementations the analysis joins over all of them, a source of *may* modality.

Algorithm 1 adds screens only along N, Mem, CG and the model-only relations D and T. No update removes a screen, so scopes only grow. It therefore computes the least solution of the constraints in Table 3.2 together with those that D and T add (Section 3.2.7). A component mounted by a route and rendered in another component's template receives the join of both scopes. The prototype instead blocks propagation into routed components and adds fallbacks outside these constraints (Section 3.3). Its result is not the least solution of this system or of any restriction of it.

Algorithm 1 Scope attribution as a worklist fixpoint over $\mathcal{P}(S)$.

Require: routes R with route tree; for each route r its entry guards $entry(r)$, named in `canActivate`, `canActivateChild` or `canMatch`, and its `canDeactivate` guards $leave(r)$; components C; nesting $N \subseteq C \times C$; HTTP interceptors I; call graph CG; dispatch relation D; guard relation T, linking a callable x to $g \in entry(r)$ when x holds a navigation site whose target lies in $subtree(r)$; guards and interceptors are given as the callables that run, such as a guard's `canActivate` or an interceptor's `intercept`

Ensure: $scope(x) \in \mathcal{P}(S)$ for every component and callable x; the callables include the members of components and of services

for all x **do** $scope(x) \leftarrow \varnothing$
end for
for all $r \in R$ and c the component of r or a layout wrapping it **do**
    $scope(c) \leftarrow scope(c) \cup subtree(r)$ ▷ own screen and outlet descendants
end for
for all c on the nesting path from the root component to the holder of the root-level primary outlet **do**
    $scope(c) \leftarrow S$ ▷ application shell, $\top$
end for
for all $r \in R$ and $g \in leave(r)$ **do**
    $scope(g) \leftarrow scope(g) \cup subtree(r)$ ▷ *model only*: runs as the user leaves r
end for
for all HTTP interceptor $i \in I$ **do** ▷ *model only*: registered for the application
    $scope(i) \leftarrow S$
end for
$W \leftarrow$ every x with $scope(x) \neq \varnothing$
while $W \neq \emptyset$ **do**
    take x from W
    for all y with $(x, y) \in N$ **do** ▷ x's template renders y
        PROPAGATE(x, y)
    end for
    for all members m of x, if x is a component **do** ▷ a member runs where its component is shown
        PROPAGATE(x, m)
    end for
    for all y with $(x, y) \in CG \cup D \cup T$ **do** ▷ CG: member level, bindings per injector; D, T: *model only*
        PROPAGATE(x, y)
    end for
end while
procedure PROPAGATE(x, y)
    if $scope(x) \not\subseteq scope(y)$ **then**
        $scope(y) \leftarrow scope(y) \cup scope(x)$; add y to W
    end if
end procedure

Note. The relations marked *model only* belong to the full analysis and are not implemented in the prototype evaluated in Chapter 5; the prototype's other departures, among them the stand-ins for guards and effects and the rule for routed components, are listed in Section 3.3.

3.2.4 Guards as program points

A guard returning `UrlTree` initiates navigation, typically to login. A `canActivate` or `canMatch` guard on r redirects from the navigation's source screen, not the guarded

route. Only `canDeactivate` redirects from a screen in $subtree(r)$ itself. The prototype does not implement this edge class.

The model's edge guard conjoins structural conditions from screen to call site with the router guards run during navigation (Section 3.1). The measured graph omits this conjunction: structural conditions and enclosing `if` statements demote edges to may but are not included in the guard. Screen-to-screen edges record only router guards, so they identify conditional navigation without its full condition. An optional mode retains enclosing conditions, but no Chapter 5 table uses it.

3.2.5 Boundary

Navigation inside a signal effect depends on a reactive graph whose read set alone is statically visible (Section 2.7). The model labels these edges *unknown* and records the signals read as a symbolic guard. This version emits none and does not attempt to determine when they fire.

3.2.6 Anchored edges

Explicit attribution would create one edge per screen for a shell link. For the largest application, the bound is its screen count (Table 5.2) times its 1343 constructs. Anchoring removes this source-scope factor: edges are stored once at their scopes, and outgoing edges are computed on demand:

$$edgesFrom(r) \;=\; E_{\top} \;\cup \bigcup_{r' \geq r} E_{subtree(r')} \;\cup\; E_{\{screen(r)\}},$$

where E_X contains edges anchored at scope X. The $r' = r$ term includes $E_{subtree(r)}$. Since the query is for a screen, $component(r)$ is defined; r' ranges over all ancestors, including prefixes. Other screen sets are stored per screen and included through $E_{\{screen(r)\}}$ in $edgesFrom(r)$.

Claim 3.2 (Expansion)**.** *The expansion of the anchored edge set equals the materialised edge set.*

Section 5.3 tests, rather than proves, this claim by comparing the sets on the corpus and fixtures.

Relative targets. Attribution to several screens represents possible firing from each, but relative targets do not resolve against each source screen. Angular resolves a `routerLink` without a leading slash, or a `Router.navigate` call relative to the injected route, against the route rendering the component containing the site. Its template also renders on descendant screens, where links still resolve against that component's route. Write $holder(s, c)$ for the nearest route at or above screen s whose component is c. A relative target at a site in component c that fires on s resolves against the path of $holder(s, c)$. If no route on that chain renders c, as with template-nested components, the full target's base is unresolved and the edge is may (Table 3.1). A target prefix falls back to s's address; all prefix edges are may.

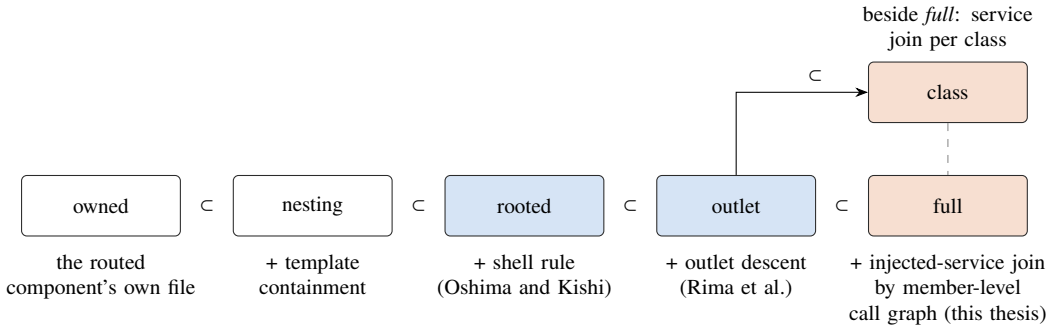


Figure 3.1: Attribution levels: ownership and containment baselines $\subset \cdots \subset$ full analysis.

Note. Blue: the levels that reimplement the two earlier Angular extractions compared in Section 5.3.3. White: the ownership and containment baselines, the first being the direct ownership of the Android literature. Orange: what this thesis adds; *class* is the per-class variant of the service join. Both orange levels also carry the fallback join and the stand-ins for guards and effects. The levels nest by construction: each withholds every mechanism above it (Section 5.3.3).

3.2.7 Extensions not evaluated

Two model-only input relations were not evaluated. *Dispatch* (D) treats dispatch as call-graph edges from each dispatcher to each handler, covering NgRx effects (NgRx is Angular's state-management library) and router event subscriptions. An effect for action a receives the join of all dispatchers' scopes for a.

Guards (T) links each callable containing navigation to the entry guards it runs. A `canActivate`, `canActivateChild` or `canMatch` guard on r receives the join of scopes of sites with known targets in $subtree(r)$, approximating the source screens of navigation into r. With these relations, propagation along N, Mem, CG, D and T remains monotone. The prototype uses stand-ins instead (Section 3.3).

3.3 Implementation

The accompanying extractor was built for this thesis and evaluated at commit `a9d29df` (Chapter 4). A framework-independent core defines the graph model and operations; framework adapters map source constructs to that model. The Angular adapter uses the TypeScript compiler API to analyse routes, templates and class code. It emits screens and edges for `routerLink` attributes and `Router.navigate` calls, with the generating rule identifier and source location as witnesses.

The Angular adapter's separate attribution pass is enabled with `--attribution`. Disabling it reproduces the baseline edge set on every corpus application. The pass implements component nesting, router outlets and the call graph, with stand-ins for D and T below.

The evaluation enables these mechanisms cumulatively (Section 5.3.3). Figure 3.1 distinguishes levels representing earlier work from this thesis's additions.

Component nesting. The pass analyses each component's selectors and template,

excluding test hosts and stories. It adds $A \rightarrow B$ when A's template contains an element matching B's selector. Ambiguous matches are restricted to components imported by A's file. If none is imported, the pass joins all candidates and records a note. Such edges are not capped at *may*, so sites may be attributed to screens rendering another candidate. Assumption A2 of Section 3.1 excludes these edges from Claim 3.1, and Table B1 lists the join among the declared approximations. Attribute selectors are matched similarly, assigning scopes to directives as well as components.

Router outlets. Route extraction records the route tree. A route component's template renders on its own and descendant screens, giving its sites scope $\mathit{subtree}(r)$. An unrouted component that contains the primary `<router-outlet>` and is unplaced by nesting is treated as the shell; everything it renders receives $\top$. An unrouted, unreferenced component whose subtree contains the outlet is also treated as shell. It is likely mounted through an unresolved route declaration, such as a route-table helper. Its edges are capped at *may*, protecting Claim 3.1 from an excessive scope.

Departures from Algorithm 1. The pass departs from it in two ways.

First, propagation into routed components is blocked. They retain $\mathit{subtree}(r)$ without receiving the scopes of components whose templates render them. Traversal still passes through them, so their nested components receive scopes from above. This restriction alone can only shrink a routed component's scope, but two other rules can enlarge scopes beyond the model's:

- components that nothing references and no route renders, but whose subtree holds the primary outlet, are treated as shell and given $\top$, with their edges capped at may;
- an ambiguous selector that no candidate's import resolves joins over all candidates, which Claim 3.1 excludes.

A smaller scope loses edges without inventing them, so this restriction affects recall and leaves Claim 3.1 intact. Chapter 5 counts the constructs it leaves unplaced.

Second, the call graph differs from Section 3.2.3. The pass operates on class members, assigning component members their component's screens. A call `this.f.g()` is resolved through `f`'s declared type (a constructor parameter, typed field, `inject(T)` field or `@Inject(TOKEN)` parameter). The call resolves to:

- the class that type names;
- the classes a provider binds it to with `useClass` or `useExisting`; or
- for an abstract class no provider binds, its subclasses in the application.

This is class hierarchy analysis [17] restricted to the declared type. Bindings are collected application-wide, not per injector. Different bindings for one token are joined, which can only widen scopes. Factory and value providers are not analysed; calls on types declared outside the application contribute no edges.

The seeded call-graph iteration reaches its constraints' least solution. Service scopes then receive two fallbacks:

- a member no caller reaches falls back to the join over every component that injects its class, the per-class approximation that the *class* level of the ablation applies to every service (Section 5.3.3);
- a member reached by the iteration receives additional screens from callers not represented in the graph: that caller's class join is added, or this class's own join if the caller is unplaced.

Neither fallback belongs to Table 3.2. The first is nonmonotone in the call graph: adding a call edge can narrow scope. Emitted service scopes are therefore the least call-graph solution widened by fallbacks, not the least solution of Table 3.2. Service sites remain *may* because conditions along their call paths are not analysed.

Anchored edges. Edges with scope $\top$ are stored once at a `scope` node. Edges scoped to exactly $\mathit{subtree}(r)$, where r is a screen with a descendant, are stored once at a node listing its screens. Prefix routes that are not screens are not anchored; other sets are stored per screen. The core provides $\mathit{edgesFrom}(r)$ and expands anchored edges.

Stand-ins for D and T. Two approximations replace unimplemented relations and are withheld below Figure 3.1's *full* level in Section 5.3.3, so that lower levels exclude their sites.

- *Guards (for T).* A guard receives the screens of routes naming it as a bare identifier in `canActivate`, `canActivateChild`, `canMatch` or `canDeactivate`. It receives each route's own screen, not $\mathit{subtree}(r)$, and none for componentless prefixes. This is narrower than the model for `canDeactivate`. For entry guards it differs from the model, since redirects leave the navigation's source screen. Scope may therefore be too small when navigation starts elsewhere, and too large when it does not start at the guarded route.
- *Effects (for D).* An effect lives as long as the application and is given $\top$.

Both produce *may* edges only. Classes not reached by calls or injection remain without scope and count as "Attribution, service" in Table 4.3. Moving inline-template witnesses from the template's first line to the construct's own line leaves the baseline graphs on the corpus unchanged. Unit tests cover each case above.

3.4 Summary

The graph model uses three modality labels. Its must-tier soundness claim rests on four assumptions about routes and route order, selectors, unresolved calls and the execution model. The evaluation tests a necessary condition of the claim. Scope attribution computes screen sets as a least fixpoint over five relations. The prototype implements three, substitutes stand-ins for dispatch and guards, and adds fallbacks outside the model. Chapter 4 describes the measurement.

4 Evaluation setup and research material

This chapter defines the corpus (Section 4.1), metrics (Section 4.2) and evaluation methods (Section 4.3) used to evaluate the analysis of Chapter 3. Chapter 5 reports the results.

4.1 Corpus selection and validity

4.1.1 Selection

Ten candidate applications were screened against six criteria: a real application rather than a library or template; a nested, lazily loaded route tree; guards present; end-to-end test configuration for a browser driver; retrievable history in which tests and source change together; and an active repository. Six candidates were retained and pinned to commits for repeatable measurement (Table 4.1). The criteria were fixed before any results were obtained.

The fourth criterion required test configuration, not a usable suite. At the pinned commits only alfresco has specifications the test-source cross-check can parse, and ghostfolio and bitwarden have no browser-test configuration. The criterion was applied inconsistently: `gravitee` was rejected for a sparse UI suite while two applications with none were retained. Route-tree size and end-to-end test density were inversely related. Methods that need test suites therefore cover only applications that have them; the rest cover all six.

Four candidates were rejected. The counts below were read from the repositories and are not generated. `angular/components` is a component library with no product router. `vendure` has 169 end-to-end specifications, all of which exercise the GraphQL server and none the user interface. `gravitee` declares 618 routes but has 14 UI specifications. `SAP/spartacus` has a large Cypress suite, but the extractor does not support its configuration-driven routing. Tool support was not one of the six criteria but became a rejection criterion after screening began.

4.1.2 Validity of an open-source corpus

The selection follows Kalliamvakou et al.'s recommendation to use and report explicit criteria [34, 35], with the departures stated above. External validity remains limited because open-source applications may differ from proprietary ones. Selection bias is also possible because I selected the corpus and wrote the extractor. The selection was also not blind to tool support, which became a rejection criterion after screening began. Mifos dominates pooled percentages, so per-application figures are the primary summary where available. The corpus spans nearly two orders of magnitude in route counts, and the pinned commits and released harness permit repeated measurement.

The extractor was also developed on this corpus, so its measurements are fitted to it (Section 6.2). The sole out-of-sample measurement compares the frozen extractor

Table 4.1: The corpus: commit, route count and role in the study of each application.

Application	Commit	Licence	Routes	Role in the study
realworld	`dd99ed2c`	MIT	14	size floor; all literal paths
alfresco	`b88d47f4`	LGPL-3.0	≥57	87 Playwright specs including a navigation suite
ghostfolio	`9884fa0b`	AGPL-3.0	≥135	constant-valued route paths; self-hostable
k8s dashboard	`0ba796d9`	Apache-2.0	140	mid-size control
bitwarden	`e40ce6f0`	GPL-3.0‡	≥168	many guards, guard factories among them
mifos	`7f781796`	MPL-2.0	≥909	size ceiling

Note. The repositories are gothinkster/angular-realworld-example-app (realworld), alfresco/alfresco-content-app (alfresco), ghostfolio/ghostfolio (ghostfolio), kubernetes/dashboard (k8s dashboard), the web client of bitwarden/clients (bitwarden) and openMF/web-app (mifos). Licences are read from the local checkouts at the pinned commits; ‡the `bitwarden_license/` directory is under the Bitwarden License. Routes are the route entries reached by the independent route parser of Section 4.3, as it stands in the final version (Section 5.5). A count marked ≥ is a lower bound: the parser records a blind spot for that application, so the entries behind it are not in the figure; Table 5.3 gives how many blind spots each application has. The marks are on alfresco, ghostfolio, bitwarden, mifos, on these grounds: children not resolvable to a route array; matcher route: path not statically knowable; spread route source: not followed.

with Rima et al.'s ground truth on their seven subjects, preregistered before extraction (Section 5.3.7). A proprietary Angular case study reporting aggregate figures is planned to address external validity (Chapter 6). The literature reviewed in Chapter 2 provides no standard benchmark corpus of single-page applications, so the corpus and manifest are an output of this thesis.

4.2 Metrics

Table 4.2 defines the measures for each research question. RQ1 has two levels: the share of declared screens with a node and the share of navigation constructs yielding at least one must transition. Both are needed because a transition requires its endpoint screens. For RQ2, a defect class requires two lines of evidence, at least one from an evaluation method (Section 4.3), except the guard class, which was accepted on source review alone (Table 5.4). Every unrecovered construct is assigned to one loss category. RQ3 compares the same extractor with the analysis disabled and enabled, using the references below. RQ4 uses mutation testing as a witness-mapping sanity check, not an independent test. For RQ5, source analysis assigns each defect class of Table 5.4 to one of three causes: resolved by the final version (a tool defect), still unresolved but statically resolvable, or beyond a static route-table reader. The result is a cause list. Every site without a transition must carry a label (Section 5.5).

Navigation construct. A navigation construct is a `routerLink` binding in a template, or a call to `Router.navigate` or `Router.navigateByUrl` in class code,

Table 4.2: The ratios and measures of the thesis, with numerator, denominator and research question.

Name	Numerator	Denominator	RQ	Reported in
Node coverage	declared screens that have a node	screens the independent route parser declares	RQ1	5.1
Must conversion	in-scope constructs witnessed by a must edge	in-scope constructs	RQ1	5.1
Recovered share	in-scope constructs witnessed by a must or may edge from a placed screen	in-scope constructs	RQ1, RQ3	5.3, 5.5
Attribution recovery	constructs assigned to attribution loss in the baseline that the analysis recovers	constructs assigned to attribution loss in the baseline	RQ3	5.3
Accounted share	recovered constructs plus in-scope constructs recorded as dynamic or unplaced	in-scope constructs	RQ2	5.5.2
Must-edge target consistency	must edges whose witness passes (lower bound) or does not fail (upper bound)	all must edges	RQ3	5.5.3
Node-level recall	distinct URLs named in an application's end-to-end test sources that have a node	distinct URLs so named	RQ1	5.2
Structural-reference recall and precision	(site, screen) pairs the reference and a level share	pairs in the reference (recall) or pairs the level places at the sites the reference determines (precision)	RQ3	5.3.4
End-to-end containment	observed screen pairs joined by an edge of the graph	observed screen pairs	RQ3	5.3.5
Scope width	mean screens a site is attributed to	the same when every site has scope $\top$	RQ3	5.3.2
Detection rate	injected changes that produce the predicted change in the graph	injected changes the graph can score (excluded mutants reported)	RQ4	5.4
Cause trace	defect classes traced to a cause in the source, by kind (read by the final version, still not read, beyond a static reader); no ratio	defect classes	RQ5	5.5

Note. Each measure has one name, used throughout. A construct is *in scope* when it is neither a query-only navigation nor excluded by the first rows of Table 4.3. Node coverage is the only measure called coverage in this thesis.

in the application source excluding tests. A separately written parser finds constructs and shares no code with the extractor. Its independence is qualified by Table A1: both tools use the same resolution rules for route-path forms added during development, and three parser corrections followed disagreements with the extractor.

Must conversion (RQ1). Must conversion is the fraction of in-scope constructs that are witnessed by at least one must edge. Three rules make it precise.

- A construct is matched to a witness on its own line or on a line either side, to absorb offsets in the extractor's locations. Two constructs on adjacent lines can therefore both be credited by one witness, so the ratio is an upper bound at line granularity.
- Query-only navigations and the two exclusions of Table 4.3 are left out of the denominator as well as the numerator.
- When several constructs produce the same edge, the extractor keeps the most certain as the witness and records the others as merged witnesses, each with the modality it earns on its own. A construct counts as converted only where it earns must itself.

The other ratios. Prior work calls several measures coverage, but here they differ by more than an order of magnitude: the share of sites accounted for (Section 5.5.2) is far above the share that convert to must (Section 5.1). Node coverage rises when the parser recognises less of an application, so it is reported with a blind-spot count. Accounted share measures whether losses are declared and says nothing about how much is recovered. The loss tables measure the unrecovered constructs, or recovery gap.

Agreement between a scope and the rendered interface (RQ3). The measures above do not test whether a site's computed scope matches the screens from which it can fire. The *rendered-interface check* tests one direction using the runtime route oracle's harness (Section 4.3). It navigates to each screen in the graph, extracts in-app links from the DOM and compares them with the graph's per-screen edges.

A rendered link without an edge shows that a site was under-attributed or not extracted. The converse is weaker, because links may render conditionally and the harness does not sign in. An unresolved target also yields reduced-confidence edges to every address matching its static prefix, and these do not assert that a link exists. The converse is therefore assessed only for exact destination predictions, whose count is reported. Three restrictions apply. Only template sites are judged, because event-handler calls to `Router.navigate` leave no DOM link. Guard-redirected screens display another screen's links and cannot be judged, so they are counted separately. They make up most screens here (Section 5.3.1). Ghostfolio's per-locale build is incompatible with this harness, so only this check excludes it.

The ablation and the structural reference (RQ3). These measurements test whether attribution improves agreement with a structural reference or merely produces larger screen sets.

The ablation extracts the whole corpus at 5 cumulative levels of the mechanisms of Section 3.2, each adding mechanisms to the one before:

- *Owned*: a site belongs to the screen whose routed component's own file holds it, the direct ownership of the Android literature.
- *Nesting*: adds component containment through template element selectors.
- *Rooted*: adds the shell rule, so a component the root renders is attributed to every screen. Oshima and Kishi assemble their page information from the root component as well (Section 2.4), so this level measures how far the prior work reaches here.
- *Outlet*: adds router-outlet descent.
- *Full*: adds the member-level call graph, with its fallback join and the stand-ins for guards and effects.

A difference between two levels is the effect of the added mechanisms given those before them, and *full* adds several at once. A sixth level, *class*, replaces the call graph with the per-class rule, joining a navigating service to every component that injects its class. The harness verifies nesting on the application graphs and refuses to produce a table if any (site, screen) pair disappears at the next level.

Scoring against the structural reference counts (site, screen) pairs in both directions, at the sites the reference lists. For one application, source inspection identified each site's screens with a derivation in `route-oracle/ground-truth/realworld.json`. This is the structural reference of Section 3.2.2, termed ground truth below. A missing reference pair indicates under-attribution and an extra pair over-attribution, both relative to the reference. Four limits apply.

- It covers one application, realworld, which is small enough to read exhaustively. A second, alfresco, was built under the same rules by the AI assistant from the sources alone, without the extractor's output (`route-oracle/ground-truth/ALFRESCO-PROTOCOL.txt`), and it leaves 17 sites undetermined (Section 5.3.4).
- The realworld reference is the author's reading, so it is not independent of the tool's author. It is published with its reasoning so a reader can check it.
- Its rules are the analysis's own relations, and under them a template condition or guard that admits a screen only in one sign-in state does not narrow a scope. The ground truth therefore tests whether the implementation applies the relations correctly and does not test whether they match what the running application renders.
- The completeness of its site inventory is checked in one direction only: the scorer reports every site the graph places that the ground truth omits. This

inventory check is separate from precision and recall, and it cannot reveal a site that both the graph and the reference omit.

The end-to-end navigation check (RQ3). This check takes its reference from execution, so it can show a navigation that the analysis's relations miss, which the structural reference cannot. Two applications ship an end-to-end suite that can be run here. The first, realworld, runs against the public demonstration backend, which is not pinned. The second, mifos, runs against a local Fineract server with the image pinned by digest. A small module added to each application logs every navigation the router completes while the suite runs, with the route patterns of the screen left and the screen reached.

An observed pair consists of source and target route patterns for an application-triggered navigation. Back-button navigations and the first screen after a full page load are excluded. A pair is *contained* if any must or may edge connects its screens after scope expansion, ignoring parameter names. Unknown targets never count. A hit need not come from the site the test used, so it does not validate an individual site's scope. Redirects are compared on the requested address, as preregistered. The scorer initially departed from this rule (Table A1), and that figure is not used.

A missed pair is classified automatically from the graph by the first matching rule:

- *redirect*: the navigation was redirected and the graph holds the pair on the address it landed on (the preregistered redirect class);
- *base*: the target is reached from a screen above the source but not from the source, as a relative link resolved against the wrong route appears;
- *dynamic*: the source screen's routed component, or one above it, holds a site whose target the extractor records as unresolved;
- *other*: none of these.

The base and dynamic classes were defined after inspecting the misses. Dynamic counts only misses not assigned to base. Every mifos miss met the dynamic test regardless of order, so that count alone was uninformative. Section 5.3.5 identifies the sites repaired after the check.

Except for that departure, the counting rules, application-specific predictions and three validity checks were recorded before inspecting logs. The checks require matching test outcomes with and without instrumentation, at least ten distinct pairs, and the same pair set in a second instrumented run. The experiment measures containment only for exercised transitions, and how to read it depends on graph density. Section 5.3.5 gives, for each application, the pairs the graph contains out of all possible screen pairs, and the hits that uniformly sampled pairs would yield at that density. This chance figure is a reference and not the expected success of a suite that chooses its paths. Untested edges are not errors, so the check cannot measure precision, and one run cannot refute a must edge.

Loss breakdown (RQ2). Each construct is matched to the edges whose witness cites it. A construct that is in scope, cited by no must or may edge and not accounted for is *lost*. It is assigned to exactly one loss category by the first rule in Table 4.3 that applies, so the categories partition the lost constructs. Named outlets are excluded before assignment, and guards are assessed separately. Constant target, Node missing and Other were not assessed under the defect-class admission rule (Chapter 5).

Guard defects mislabel edges without removing them, so they are absent from this table. No method directly validates guard labels. Guard handling is assessed only by guard-removal mutation and source review (Section 5.5), and red-team review of the graph and source found the defect.

The nested branch classifies the *owning* route component. A node-category construct can still convert if attribution places it through a shell or subtree (Section 5.3). *Other* counts constructs needing manual inspection and does not stand for a single cause.

Change in recovery and must-edge target consistency (RQ3). Must conversion and the loss breakdown are measured at one extractor commit on identical inputs, with attribution disabled and enabled. Disabling attribution reproduces every application's baseline edge set exactly (Section 3.3).

Must-edge target consistency requires two conditions: the witness line contains a construct of the claimed kind with the claimed target, and the independent route set declares the destination node's address. Without the second condition, a node at an address the application never mounts could pass.

Unread destinations are neither passes nor failures, so consistency lies between the share of must edges that pass and the share that do not fail. Recovery and consistency alone cannot assess RQ3, because attributing every site to every screen would raise recovery with no loss of must conversion or target consistency. Three measures are therefore considered together: recovered share and must conversion over the same denominator; scope width against that trivial assignment (Section 5.3.2); and agreement with the two structural references, the application suites and Rima et al.'s tool (Section 5.3.6). These references do not derive solely from attribution output. Consistency bounds are reported for both runs but cover different must-edge sets, so they are not compared as rates.

Detection rate (RQ4). This sanity check tests whether an injected source change produces the predicted graph change, reported per operator with its n. It does not establish change-impact accuracy. Detection is nearly guaranteed because mutants are sampled from the graph, and the harness was tuned on its misses (Section 4.3.2). Changes to unrepresented constructs are excluded and not counted as misses, because they repeat recovery gaps measured by the loss tables. The rate is conditional on recovery and reported with the exclusion count.

Table 4.3: Rules that classify a construct, in the order the harness tests them.

Category	The construct . . .
Query only	changes only the query string of the current screen (`navigate([], {queryParams})`), so it is not a transition and is not counted as a loss
Named outlet	targets an auxiliary outlet (`{outlets: ...}`)
No commands	names no destination at all
Converted, must or may	is cited as witness by an edge of that tier from a placed screen
Accounted, dynamic	is cited by an edge into a sink: the extractor records its destination as unresolved
Accounted, unplaced	is recorded by the extractor as a site it read but could not place
Attribution, service	has no owning component, or its owner is a class that is not a component: a service, guard, interceptor, resolver or effect. A site in a template whose owner cannot be resolved falls to *other* instead
Attribution, shared	is in a component that no route renders directly
Owner is a route component, but that component has no node:	
Component unlinked	a node for the owner's route exists, but the extractor did not link it to the component
Constant path	every declaration of the owner's route gives its path as a constant reference rather than a literal
Other	some declaration of the owner's route does have a node
Lazy boundary	some declaration of the owner's route is in a lazily loaded routes file
Node missing	a declaration exists but none of the above applies, or neither parser found a route for the owner
Constant target	owner has a node, but the construct's own target is not a literal
Other	matches none of the above; inspected by hand

Note. The first three rows take a construct out of the denominator, the next counts it as recovered in the tier that cites it, and the two after that as accounted for. Only a construct that reaches the rows below is lost. The indented rules are the nested decision taken when the construct's owning route component has no node.

4.3 Evaluation methods

Figure 4.1 locates the five methods in the extraction pipeline. Table 4.4 states their measures and blind spots, and none establishes correctness alone. RQ3 adds four checks not listed there: the rendered-interface check, ablation, structural reference and end-to-end check. Figure 4.2 identifies the graph targets and ground truths of the four source-based methods.

Four of the five read the application's sources, and two readers of the same text can be wrong in the same way. The fifth, the runtime route oracle, was added for that

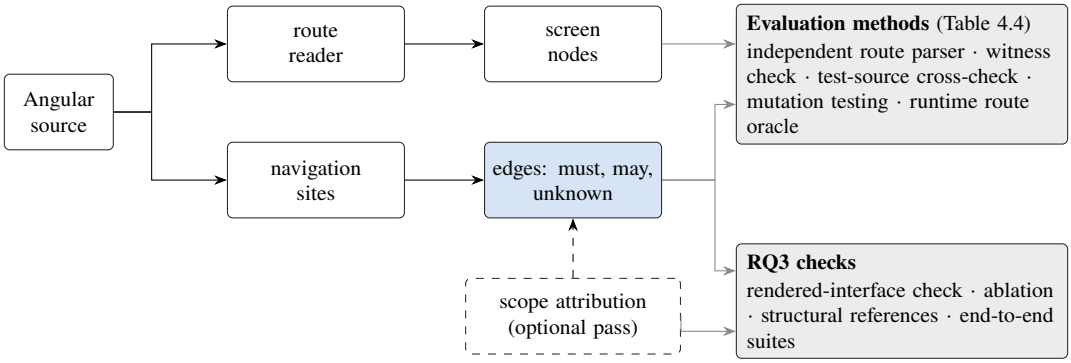


Figure 4.1: What is evaluated: the extractor, scope attribution and the evaluation methods that check the resulting graph.

Note. The extractor reads route tables into screen nodes and navigation sites into edges, tiered must, may or unknown by the demotion rules; scope attribution, an optional pass, places sites that belong to no single screen. The five methods of Table 4.4 check nodes and edges against independent readings; the RQ3 checks examine the attribution.

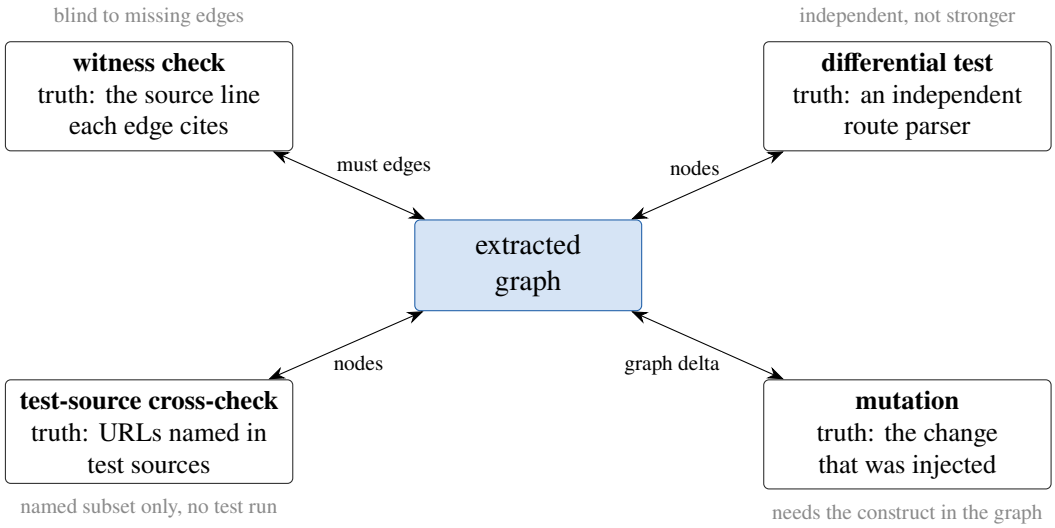


Figure 4.2: The four evaluation methods that read the applications' sources, the graph part each checks, and its ground truth.

Note. The fifth method, the runtime route oracle, is not drawn because its ground truth is the running application rather than any part of the graph. No method sees everything; the grey notes give the blind spot of each, as listed in Table 4.4.

reason.

Table 4.4: Evaluation methods, the quantity each measures, and the blind spot each has.

Method	Quantity	Blind spot
Witness check	Must-edge target consistency. For every must edge, the witness file is re-parsed at the claimed line, the construct kind and target are compared, and the destination node's address is tested against the independent route set.	Cannot see missing edges. Target agreement is invalid for over-approximated may edges, whose witness proves only that the call site exists. Where both tools read a table at the wrong prefix, they agree on a wrong address (Section 5.5.3). Target agreement is literal, so an edge to the screen a redirect lands on is reported as a failure.
Differential test	Disagreements with a separately written route-configuration parser.	Diagnostic, not a metric. The parser is written separately but is not necessarily more capable than the extractor. Both use the same resolution rules for the path forms added during development (Table A1), so agreement on those forms is not independent validation.
Test-source cross-check	Node-level recall against a fourth reader of the source. The URL literals in the application's end-to-end specifications are collected; a URL its own test suite names is an address its authors expect to exist, and the extractor is asked whether it created a node at that address.	Reads test source text and runs nothing, so a collected string is an address a test *names*, not one a test was observed to reach. Tested subset only. Textual extraction under-samples click-driven navigation. A direct `goto` is not a UI transition, so visit pairs are not counted as edges.

continued on the next page

Table 4.4 continued from the previous page

Method	Quantity	Blind spot
Mutation test	Detection rate over four operators: rename a path, remove a guard, delete a `routerLink`, retarget a `navigate`. Ground truth is the authored delta. Detection is judged on the node and edge sets and on the witness set, since several constructs can produce one edge.	A mutant whose construct is absent from the baseline graph re-detects a recovery gap and is excluded, as is a guard mutant on a screen that no edge enters. A component several routes render maps to every screen those routes declare, which errs towards scoring a mutant rather than dropping it. Detection is conditional on recovery, and the attributed and unattributed graphs do not exclude the same mutants, so they are also compared over the mutants both can score, paired by operator, file, line and subject. The sample is drawn from the graph, so it is not independent of the artefact under test.

continued on the next page

Table 4.4 continued from the previous page

Method	Quantity	Blind spot
Runtime route oracle	Node-level soundness against the framework itself. Each application is built in its development configuration, loaded in a headless browser, and asked for the route configuration the Angular router holds, with every lazily loaded subtree expanded through the application's own `loadChildren`. The addresses at which the extractor creates nodes are compared with the addresses the framework mounts. Nothing re-reads the route files. Which addresses count as mounted is itself a reading, so the harness navigates the running application to a list of addresses and records what the recogniser made of each, before any `canActivate` guard runs and after the `canLoad` and `canMatch` guards that take part in matching.	Requires the application to build and boot without its server. It reads the configuration and not the rendered screen, so a route that a guard always refuses still counts as mounted. An address claimed through a `UrlMatcher` cannot be decided from the configuration. A workspace application that composes the measured library mounts the shell's own routes as well, so part of any difference is scope and not reading. Its result files record no extractor commit or runtime version (Section 4.3.3).

4.3.1 Preconditions

Two properties of the extractor were checked before any measurement. *Determinism*: two extractions from the same unchanged source give identical nodes and edges. *No accumulated state*: extracting into an existing database gives the same graph as extracting after deleting it. Without them, differences seen in mutation testing could not be told from noise.

4.3.2 Changes to the protocol

The thesis distinguishes three extractor versions: *early development* (commit f17a302), before evaluation-driven corrections; *intermediate* (extractor eb9dab80, with 20 must edges, tables printed at 7d109c7); and *final*, whose results are presented throughout. Only the intermediate must-tier figures are cited as printed, because the underlying runs were lost and an archived rerun produced a different loss ledger. Superseded

instrument figures cannot be regenerated with the released harness and are not cited as results. Unexpected figures are checked against their inputs before reporting, because several came from measurement defects, not extractor behaviour.

The mutation harness was corrected after inspecting its misses, so the final mutation run is not independent of the earlier one. Every correction is listed, one row each, in Table A1 of Appendix A, together with the change in the mutation sample.

Availability. The extractor, the harness and the scripts that generate every table are in the accompanying repository, `https://github.com/kanetran29/uigraph`, under the MIT licence; file paths named in this thesis are relative to its `thesis-handoff` directory. Rima et al.'s tool was run from its public repository at commit `f33003f`; it carried no licence when checked on 5 November 2026, so it is not redistributed.

4.3.3 Reproducibility

Scripts generate numerical macros; `REPRODUCE.md` lists each table's script and commit. The final results use extractor `a9d29df`, after repairs prompted by Rima et al.'s subjects (Table A1), with harness 1.10.0 and mutation harness mutate 1.9.1. The demotion-rule sweep and may-reason census use the same commit. The pre-repair end-to-end figure (Table A1) uses the archived graph from `c42f220`. The first Rima comparison uses `2aac284`, frozen before its evaluation-driven repairs as preregistered (Section 5.3.7). Two measurements remain historical: the rendered-interface check (Table 5.11) and the runtime route configurations recorded on 29–30 October without an extractor commit. The latter are compared with the final graph (Section 5.5.3).

4.4 Summary

The corpus is six open-source Angular applications pinned to commits, selected by explicit criteria with the departures stated in Section 4.1, including tool support as a rejection criterion added during screening. Table 4.2 maps each research question to its measures. Five evaluation methods check the extractor: four against other readings of the same sources, and the runtime route oracle against the framework. Four further checks test the attribution: the rendered-interface check, the ablation, the structural reference and the end-to-end check. Table 4.4 states the blind spot of each method, and together the methods and checks do not amount to a complete correctness test.

5 Results

This chapter reports measurements from the harness of Section 4.3 on the corpus of Section 4.1. Sections 5.1–5.4 answer RQ1–RQ4: losses, defect classes, scope attribution and mutation sensitivity, respectively. Section 5.5 traces loss causes (RQ5), and Section 5.6 reports dead-link candidates. Section 5.7 gives a short answer to each research question.

The numerical tables are generated from the harness result files, and hand-compiled summaries are marked. The development versions and the corrections between them are listed in Section 4.3.2.

All extractions use source trees without installed dependencies. The corpus was selected with tool support in view, so results are likely optimistic for Angular applications generally. Section 6.2 discusses both conditions.

5.1 The two levels of loss (RQ1)

RQ1 measures declared screens represented as nodes and navigation constructs yielding deterministic transitions (Section 4.2). Nodes are a precondition for transitions, but the first ratio does not numerically bound the second, because the first counts screens and the second counts constructs.

Table 5.2 gives the second measure on the final snapshot. The constructs that earn must number 21 in the whole corpus without attribution and 25 with it, 1.4 % of the in-scope constructs. With attribution they lie in 4 of the six applications, and none lies in the other 2. Edge counts should be read per tier and not summed as distinct constructs, because one unresolved site can produce many over-approximated may edges. An optional measurement switch records every condition that demotes a may witness. Disabling it leaves output unchanged on all six applications (Section 4.3.3). Table 5.1 groups reasons for 1657 may-only witness locations. 1631 involve an application decision identified by the extractor: a guard or resolver, branch, callback or template condition. The number of locations that stay at may solely through extractor uncertainty is 26. Under the present rules, better resolution alone could promote at most those to must. Proving that a guard always admits or that a branch always executes could promote more. The small must tier thus follows largely from its definition. A guard that always admits still causes demotion, because the census identifies decision constructs and does not test whether they vary at run time.

Mifos, the largest application, holds 1289 of these 1657 locations. It also holds most of the corpus's constructs and edges (Sections 5.5.2 and 5.5.3), so it dominates the corpus-wide counts of constructs and edges and the shares pooled from them.

Table 5.3 compares extracted nodes with the independent route parser's declared screens. Every declared screen has a node on all six applications, relative to that denominator.

The two readers still disagree about a handful of addresses. Two qualifications

Table 5.1: Why the may-only witness locations are may, by family of reason.

			Any, by application					
Family	Any	Sole	realworld	alfresco	ghostfolio	k8s	bitwarden	mifos
async	333	10	4	0	15	5	61	248
branch	231	9	4	4	17	1	85	120
template	790	31	30	3	37	7	30	683
guard	1518	174	16	9	99	11	202	1181
reader	127	1	1	1	0	1	3	121
attribution	194	0	1	4	8	3	94	84
target	690	4	13	3	29	5	97	543
same screen	203	0	8	3	14	1	6	171

Note. *Any* counts the witness locations with at least one reason of the family, *Sole* those whose every may witness has that one reason only. The application's own decisions are the async, branch, template and guard families; reader, attribution and target are the extractor's uncertainty; same screen is neither.

Table 5.2: Must conversion per application, baseline.

Application	Constructs	In scope	Nodes	Must	May	Must conversion
realworld	40	39	10	1	23	2.6%
alfresco	34	27	37	0	7	0.0%
ghostfolio	198	186	112	0	61	0.0%
k8s dashboard	72	68	72	0	7	0.0%
bitwarden	131	112	113	1	89	0.9%
mifos	1343	1343	642	15	4429	1.4%

Note. Must conversion is the fraction of in-scope constructs whose own witness earns must; the edge columns count edges. *Nodes* counts every node of the baseline graph: the screens of Table 5.3 together with its unplaced origins and dynamic targets (Section 5.3 counts the nodes of the attributed graphs, which add anchors and more of the latter). The baseline and attributed graphs are extracted from the same final snapshot on identical corpus inputs.

apply.

- The denominator is the screens a *static reader* can enumerate. Against the addresses the running framework mounts, the same node set is 892/1146 (Section 5.5.3). Almost all of the difference is one construct in one application that the extractor declares unread.
- The two static readers were partly corrected against each other, so their agreement is weaker evidence (Section 6.2). The runtime route oracle is the external check, and it finds no node at an address where the framework mounts nothing (Section 5.5.3).

This completeness concerns only resolvable screens. Blind-spot counts identify

Table 5.3: Node extraction against the independent route parser, in both directions.

Application	Declared	Nodes	Matched	Coverage	Unmatched	Blind spots
realworld	10	10	10	100.0%	0	0
alfresco	26	27	26	100.0%	1	10
ghostfolio	84	84	84	100.0%	0	2
k8s dashboard	67	67	67	100.0%	0	0
bitwarden	98	98	98	100.0%	0	1
mifos	605	606	605	100.0%	1	1
All		892			2	

Note. Declared screens that have a matching node, and nodes for which the independent route parser holds no declared screen. The blind-spot column counts route entries the parser could not follow (an unfound lazy route array, an unresolvable path expression, a custom matcher, or a route source spread from a call it does not follow), which marks where either direction is incomplete. The node set is the same with scope attribution off and on, since attribution creates no nodes.

but do not bound omissions: one unresolved route entry can conceal many screens (Section 5.5.3).

5.2 Where static extraction fails, and why (RQ2)

Table 5.4 separates evidence from named evaluation methods and other observations. The guard class has no named method. Attribution, addressed by Chapter 3, relies most heavily on observations among the other classes, and these are not independent instruments.

Five classes were identified during development, each with at least two confirmations. A sixth, nodes unlinked to their rendering component, has only the parser–graph comparison as evidence. It remains a candidate and has no instances in the final version.

Table 5.5 classifies 1818 constructs under Table 4.3, recording their tier or loss reason. A construct counts as must only through its own must witness. Rows are loss categories, distinct from the defect classes of Table 5.4; guards are assessed separately (Section 4.2).

Under the loss rules' precedence, attribution is the largest loss category in every application. The two attribution categories contain 454 of 1818 constructs; no other loss category contains a tenth as many. Unresolved destinations are counted separately and number about a third as many.

Ownership is tested before the target. Among attribution losses, 101 targets fail the literal test; reversing this order would make target losses larger than attribution on alfresco and k8s dashboard.

Constant paths, lazy boundaries and unlinked components have no instances in Table 5.5 because the final extractor resolves their causes (Section 5.5.1). Under the stated precedence, the remaining unplaced navigation mostly lacks a single owning

Table 5.4: Defect classes and the evidence for each.

Class	Evidence	Methods	All
Attribution: navigation outside route components is invisible	mutants excluded on the baseline graph; multi-mount edge absent; manual audit; an application, k8s dashboard, with nodes and no edges on the early development version of Section 4.3.2	1	4
Lazy module boundary not followed	independent route parser; test-source cross-check; mutation	3	3
Guards unreliable in both directions	red-team panels and a source survey of the guard factory and the call-form `canActivate` (Section 5.5.1)	0	2
Named router outlets not implemented	test-source cross-check; source survey	1	2
Constant-valued route paths not resolved	independent route parser; node-count collapse	1	2
Candidate: node not linked to its component	independent route parser against graph	1	1

Note. A class is admitted on two lines of evidence, at least one from a named evaluation method; the methods column counts only those. The guard class is the one exception: both its lines of evidence are source review, and the mutation run neither confirms nor refutes it (Section 5.4). This summary is compiled by hand; it is not harness output.

screen, motivating scope attribution.

Category interactions mean that the 454 attribution constructs neither lower-bound nor upper-bound attribution's recoveries (Section 5.3).

Table 5.6 reports the witness check on the final attributed tier and the test-source cross-check. Section 5.5 treats the must tier separately. 623 edges in 3 applications have their only witness outside the application directory. Both tools read those files, so their agreement there is not independent evidence.

5.3 Scope attribution (RQ3)

The evaluation covers the scopes, attribution rules and anchored representation defined in Sections 3.2.2, 3.2 and 3.2.6 of Chapter 3. The section first reports what attribution recovers on the whole corpus and checks the anchored edge set. Sections 5.3.1 to 5.3.5 then check the computed scopes against the rendered interface and the widest possible assignment, measure what each mechanism contributes, and check the scopes against structural references and the applications' own end-to-end suites. Sections 5.3.6 to 5.3.8 compare the analysis with earlier work: with earlier work's mechanisms inside this extractor, on Rima et al.'s subjects and against their tool.

Table 5.7 compares identical inputs with and without attribution, construct by construct. A baseline attribution loss is *recovered* if a must or may edge from a placed screen witnesses it after attribution. Dynamic sinks and unplaced origins do not count

Table 5.5: Outcome of every navigation construct, baseline.

Outcome	rw	alf	gf	k8s	bw	mif	Total
Converted, must	1	0	0	0	1	19	21
Converted, may	16	3	55	4	53	1025	1156
Attribution, shared component	21	10	63	51	37	224	406
Attribution, service	1	8	9	5	15	10	48
Lazy boundary	0	0	0	0	0	0	0
Constant path	0	0	0	0	0	0	0
Component unlinked	0	0	0	0	0	0	0
Constant target	0	0	0	0	0	0	0
Node missing, other	0	2	3	1	0	34	40
Other	0	0	6	0	2	0	8
Dynamic destination	0	4	50	7	4	31	96
Source the analysis cannot place	0	0	0	0	0	0	0
Query only (not a transition)	1	0	11	4	19	0	35
Excluded: names no destination	0	0	1	0	0	0	1
Excluded: navigates a named outlet	0	7	0	0	0	0	7
All constructs	40	34	198	72	131	1343	1818

Note. Columns: realworld, alfresco, ghostfolio, k8s dashboard, bitwarden, mifos. All rows are measured on the final snapshot with attribution disabled.

Table 5.6: Results of the witness check on the final attributed tier, and of the test-source cross-check.

Method	Result	*n*
Witness check, final attributed must tier	Detected failures: 0. Edges with a destination the checker cannot read: 62. Unread edges are not passes, and target agreement does not establish Claim 3.1.	159
Test-source cross-check, alfresco	10 of 14 URLs the specifications name have a node; one negative test excluded.	14

Note. Mutation is reported separately in Section 5.4, on the attributed graph. This summary is written by hand from the harness output rather than generated.

as recovery.

Attribution recovers 303/454 assigned constructs: a per-application median of 40.2 % (range 14.3 %–100.0 %) and a pooled share of 66.7 %. Most recoveries occur on mifos. Excluding it reduces the share to 38.6 %, and recovery stays under half on 4 of the six applications. Edge counts rise from 4729 to 13830 (per application in the *anchored* column of Table 5.8): 159 must, 13121 may and 550 unknown. The

Table 5.7: Before → after scope attribution, on the final evaluated snapshot.

	Attribution losses, recovered			Must conversion (RQ1)			Must, read and passed	
Application	Lost	must	may	Before	After	pp	Before	After
realworld	22	2	20	2.6%	7.7%	+5.1	1/1	9/9
alfresco	18	0	4	0.0%	0.0%	+0.0	–	–
ghostfolio	72	0	33	0.0%	0.0%	+0.0	–	–
k8s dashboard	56	0	8	0.0%	2.9%	+2.9	–	63/125
bitwarden	52	0	18	0.9%	0.9%	+0.0	1/1	1/1
mifos	234	0	218	1.4%	1.4%	+0.0	15/15	24/24
All	454	2	301					

Note. First, the constructs assigned to attribution in the baseline and how many recover onto must or may, the measure of RQ3. Then must conversion (RQ1) in per cent and its change in percentage points, and must-tier edges whose destination the witness check read and passed; a construct's tier is the modality its own witness earns. The remaining must edges are unread destinations, all 62 of them on k8s dashboard, which are neither passes nor failures; no edge failed. Constructs converted from other baseline categories are in Table 5.9.

attributed graphs hold 1412 nodes: 892 screens, 28 anchor nodes (each stores the edges anchored at one scope, Section 3.2.6) and 492 unplaced origins or dynamic targets. Section 5.3.3 measures what the *full* step adds beyond the levels that reimplement earlier work and what its call graph adds over the class rule it replaced.

Measured by RQ1's must conversion, the gain is far smaller. The recovered constructs divide into 2 at must and 301 at may, and must conversion rises on 2 of the six applications. The must constructs rise from 21 to 25. The recoveries at must, on realworld, account for 2 of the rise. The other 2 are k8s dashboard constructs that the baseline already converted at may and attribution raises to must. The recovery count does not include them.

Counted in edges, the must tier grows from 17 to 159. The edges added are the links of two application shells, drawn once for each screen the shell renders (Section 5.5).

Construct identifiers are tracked across the two runs. The number of conversion regressions they show is 0, and the number of must-to-may demotions is 0. No baseline must conversion was lost. The tracking measures recovery and does not test whether targets stay the same. Section 5.5.3 assesses the must tier and remaining edges.

Claim 3.2 requires anchored-edge expansion to equal the materialised edge set. A separate implementation expands each scope-anchored edge over its member screens and deduplicates the result. Both sets agree on every application; otherwise Table 5.8 is not produced. The check is the only measurement of storage savings: on mifos, materialisation yields 13.7 times as many edges as stored.

The check tests member traversal and deduplication, but both paths share the scope-membership definition. It supports rather than proves the claim on 6 applications and fixtures. The library's materialisation function is not used: on 5 applications it returns more entries than either path, reflecting iteration rather than distinct-edge

Table 5.8: The anchored edge set against its expansion.

Application	Screens	Anchored	Expanded	Independent
realworld	10	59	88	88
alfresco	27	55	100	100
ghostfolio	84	395	1467	1467
k8s dashboard	67	313	698	698
bitwarden	98	576	764	764
mifos	606	12432	170011	170011

Note. *Anchored* is what the graph stores, *expanded* what the per-screen query yields, and *independent* a second materialisation written for this check. Claim 3.2 requires the last two to be equal; the check refuses to emit this table if they differ on any application.

Table 5.9: Where recovered constructs came from on the final snapshot.

Baseline class of the construct	must	may	total
Attribution, shared component	2	281	283
Attribution, service	0	20	20
Other, no class rule applied	0	2	2
Out of the attribution classes	2	301	303
Out of the node classes	0	0	0
Out of the remaining classes	0	2	2
All conversions gained	2	303	305

Note. Rows are their baseline loss categories; columns give the modality each construct's own witness earns after attribution. Constructs are followed by identity, not matched by source location.

differences.

Table 5.9 gives the recovery by the loss category each construct held in the final baseline and by the modality its own witness earns.

Table 5.10 classifies each construct that stays lost.

The 454 assigned constructs comprise 303 recovered and 151 remaining. All but 2 of the remaining constructs have a recorded reason. Each of the 2 without a reason has a scope and a destination node but no diagnosed cause. The reasons combine implementation gaps and limits of the source population.

- Some components have no recovered route or template path to a screen. An empty computed scope does not prove that the application never renders them.
- Some sites lie beyond the member-level call graph's supported receiver and provider forms.

Table 5.10: Why attribution losses remain after attribution.

Reason	rw	alf	gf	k8s	bw	mif	Total
Owner component not reached from any screen	0	8	9	5	17	1	40
In a service or guard, not placed	0	6	4	4	7	7	28
Scope found, target screen has no node	0	0	24	2	6	3	35
Scope found, target fully dynamic	0	0	2	37	2	5	46
Scope found, not explained	0	0	0	0	2	0	2

Table 5.11: Recorded scope predictions of an earlier graph against the rendered interface, on the 5 applications the harness could serve.

Application	Screens	Judged	Pairs rendered	Predicted by the scope	Not predicted	Raw anchors
realworld	10	7	25	25	0	59
alfresco	26	3	0	0	0	0
k8s dashboard	67	1	0	0	0	0
bitwarden	98	18	8	4	4	8
mifos	605	1	0	0	0	0
All	806	30	33	29	4	67

Note. These predictions were not recomputed for the final call graph. *Judged* counts screens that rendered themselves rather than redirecting to a sign-in page; the rest cannot be judged from what rendered, because a redirected screen shows another screen's links. *Not predicted* is the only column that counts against the analysis; the other direction is reported in the text, because it can be judged only where the graph names a destination exactly.

- Other sites have a scope but a dynamic destination, or a target for which the graph has no node.

5.3.1 Checking the scopes against the running interface

Recovery does not establish whether a site's computed scope matches its actual screens. The rendered-interface check (Section 4.2) tests one direction. Table 5.11 reports an earlier graph's observations: neither the browser experiment nor its predictions were recomputed for the final call graph. These findings concern that earlier graph.

Scopes too small. Of the 33 (screen, target pattern) pairs that the running applications render on the judged screens, 29 are predicted by the computed scope (87.9 %). Behind these predictions stand 67 raw anchors. On realworld, which renders most of its screens without a backend, every rendered pair on every judged screen is predicted. The pairs that are not predicted are all bitwarden's.

The unit is a (screen, destination) pair rather than a site, so a confirmed pair does

not validate the scope of every site behind it. The 4 unpredicted pairs still demonstrate under-attribution. The check covers 30/806 screens; 776 redirect to sign-in without a backend. It includes 5 of six applications, excluding ghostfolio because its per-locale bundles do not fit the harness's single served directory. These restrictions leave only one direction tested (Section 6.2).

The 4 unpredicted links share one cause. The graph holds their edges, with the right witness, but anchors them at an origin node instead of a screen. The component that renders them lives in a workspace library outside the application directory, and attribution computed the empty scope for it. These sites belong to the *unplaced* class of Table 5.20.

Scopes too large. The 80 predicted but unrendered links combine three causes. Two are not defects: realworld's header hides links from unsigned users, and a tag page visited with a placeholder segment lists no articles. The third is target over-approximation, which concerns the destination rather than the scope. The combined count cannot measure over-attribution.

Only predictions that name a destination exactly can be refuted by a page not linking there. There are 7 of them across the judged screens, against 102 over-approximated. Of these 7, the number not rendered is 0, and a denominator of 7 supports no rate. The over-attribution direction was therefore not refuted on this corpus, because the graph almost never claims an exact destination at a screen this check can observe.

5.3.2 What the attribution discriminates

Since the rendered-interface check cannot establish over-attribution, Table 5.12 compares computed scopes with assigning every represented site to every screen. This widest assignment places sites but neither resolves unknown targets nor adds missing destinations, and it does not recover every construct. The ratio measures how much the analysis narrows source scopes.

Computed scopes are 1.7 to 18.7 times narrower than the baseline. The lower bound limits RQ3: on k8s dashboard, the average site is attributed to 39.5/67 screens (59 %), excluding only about two screens in five.

The comparison has two implications.

1. The trivial baseline is not a competitor for the must tier. An unguarded site in a shared component attributed to every screen would yield a must edge from screens that never render it, so Claim 3.1 would fail by construction. That none of the readable must edges attribution added fails the witness check is meaningful only where the attribution is narrow. Of the 142 must edges it added, 62 are unread, all on k8s dashboard, where the scopes are widest.

2. Some recovery relies on broad scopes. Attributing a site to three fifths of an application provides little discrimination. The evaluation neither validates

Table 5.12: The computed attribution against scope ⊤ for every site.

Application	Screens	Sites	Attributed pairs	Mean scope	All-screens pairs
realworld	10	39	188	4.82	390
alfresco	26	19	117	6.16	494
ghostfolio	84	193	4827	25.01	16212
k8s dashboard	67	62	2450	39.52	4154
bitwarden	98	170	1790	10.53	16660
mifos	605	1319	42750	32.41	797995

Note. A site is a source construct a witness cites; an edge anchored at a scope is counted over the screens that scope stands for, since counting it at its anchor would understate the analysis's breadth. *Mean scope* is the attributed pairs per site.

Table 5.13: The corpus extracted at each attribution level.

		sites / (site, screen) pairs				
Application	Screens	owned	nesting	rooted	outlet	full
realworld	10	17 / 26	25 / 48	38 / 178	38 / 187	39 / 188
alfresco	26	11 / 33	14 / 37	14 / 37	14 / 37	19 / 117
ghostfolio	84	112 / 122	124 / 147	174 / 4347	186 / 4435	193 / 4827
k8s dashboard	67	11 / 12	59 / 275	59 / 275	59 / 2250	62 / 2450
bitwarden	98	124 / 134	152 / 394	157 / 884	159 / 1453	170 / 1790
mifos	605	1088 / 1445	1251 / 1773	1310 / 37468	1310 / 41223	1319 / 42750
All	890	1363 / 1772	1625 / 2674	1752 / 43189	1766 / 49585	1802 / 52122

Note. Sites counts the source locations a witness cites that the level places on at least one screen; one construct can have more than one, and a location can lie outside the application directory, so site counts are not comparable with the construct counts of Table 5.5. *Pairs* are the (site, screen) attributions it asserts. The levels are nested, and the table is produced only if each one's pairs are a subset of the next one's. *Screens* counts the screens at an address a link can name, so the catch-all screens of alfresco and mifos are left out of it and of the pairs, and the column falls short of the 892 screens of Section 5.3 by those screens.

nor refutes these scopes because the rendered-interface check cannot test over-attribution.

5.3.3 What each mechanism contributes

Table 5.13 extracts the corpus at 5 nested levels, each withholding every mechanism above it, including the stand-ins of Section 3.3, because otherwise the lowest level would credit direct ownership with sites that only the stand-ins place.

Containment adds the most sites. The pre-existing shell rule adds the most (site, screen) pairs and reference recall. Outlet descent mostly widens scopes; counts alone cannot validate this width, which the next subsection evaluates. Full adds the

Table 5.14: Each attribution level against the structural reference of each application.

	realworld			alfresco		
Level	Precision	Recall	Off shell	Precision	Recall	Off shell
owned	100.0 %	13.8 %	44.8 %	100.0 %	9.2 %	36.4 %
nesting	100.0 %	25.5 %	82.8 %	100.0 %	11.5 %	43.2 %
rooted	100.0 %	94.7 %	82.8 %	100.0 %	11.5 %	43.2 %
outlet	100.0 %	99.5 %	98.3 %	100.0 %	11.5 %	43.2 %
full	100.0 %	100.0 %	100.0 %	82.0 %	23.6 %	90.9 %
class	95.4 %	100.0 %	100.0 %	92.3 %	13.8 %	52.3 %

Note. Precision and *recall* are agreement with that reference. *Off shell* is recall over the pairs outside the shell, where attribution decides. *class* is the class-level join that the call graph of *full* refines.

member-level service call graph, fallback join and guard/effect stand-ins, which only full and its class variant include. Relative to the class rule it replaced, the call graph adds 7 sites (0.4 % of all sites the *full* level places) and 331 pairs. The class rule remains the fallback for members without identified callers.

Comparison with earlier work uses two levels. The shell rule is pre-existing; Rima et al.'s specification also includes ancestor-route components, as *outlet* does. These levels reimplement earlier mechanisms inside this extractor; they do not run the earlier tools or show that those tools would place fewer sites. Against the closest level, *outlet*, full adds 36 sites (2.0 % of all sites the *full* level places), all in services, guards or effects rather than components.

5.3.4 Agreement with a structural reference

Source inspection produced 39 sites and 188 (site, screen) pairs for realworld (Section 4.2). Realworld has only one service navigation and barely exercises the new step. The alfresco reference includes service navigation: 16 sites and 174 pairs. Table 5.14 scores every level against both references in both directions. Both references apply the analysis's own relations (Section 4.2), so precision and recall here measure agreement with a reading made under those relations. They do not measure agreement with what the running application renders.

On realworld, the table requires nested pair sets and no full-level pair outside the reference; levels therefore differ in recall alone. Most reference pairs come from the shell rule (*rooted*) and the enclosing-route rule (*outlet*), and full adds the remaining 1 of the 188 pairs. On alfresco, full adds service, guard and effect sites beyond earlier levels, but is less precise than both those levels and the class join.

- *The reference is not independent of the graph.* The realworld reference was completed in its first version with nine header sites the scorer reported from the graph. Its precision is therefore weakest as evidence where the shell dominates.
- *The shell dominates.* On realworld, 13/39 sites are placed at $\top$, supplying

130/188 pairs that every level from *rooted* upward places trivially. On alfresco, 5 sites, 130/174 pairs, sit in a shell taken from a library outside its sources, which no level reads. The off-shell column is the one where attribution decides.

- *Two repairs were guided by the reference:* the call graph, written after the realworld reference exposed the class-level defect, and the rule that places an empty-path child on the screen above it (Section 6.3). The 100.0 % precision and the recall of *full* are scored on the same reference and test neither repair independently (Section 6.2).

- *The class join's excess on realworld* is 9 pairs, all at the `navigate` inside `UserService.logout`. The class join attributes the service to every screen whose components inject it, but its only caller is a button on the settings screen.

- *An AI assistant built the alfresco reference without the graph,* using realworld's rules under a protocol committed before scoring (`ground-truth/ALFRESCO-PROTOCOL.txt`). 17 undetermined sites are not scored. Afterwards, four lines reported by the scorer as placed but unlisted were inspected using only line numbers, without graph placements. None was a navigation site. They were added as excluded, a protocol deviation: only adding missed sites was permitted. The addition changes no figure because unlisted lines are not scored.

- *A configuration-driven site accounts for all full-level excess on alfresco.* The number of sites with full-level excess is 1: `navigateToSearch` produces all 9 excess pairs. Its only caller is an effect triggered by a toolbar button in extension configuration that the analysis does not parse. The full level assigns the site to $\top$ at may, which produces most pairs added over *outlet*. The class join's excess pairs, 2 in number, arise at the same site. Both levels also place an undetermined site at $\top$. That site is unscored, and treating it as firing nowhere would increase both excess counts.

- *The protocol's predictions.* The first (A1), that the full level recalls more than *outlet*: ✓. The second (A2), that its precision is ≥ the class join's: ✗.

What the two references establish is therefore narrower than soundness. Neither licenses the corpus-wide widths of Table 5.12 on an application of six hundred screens.

5.3.5 Agreement with the applications' own suites

The end-to-end reference comes from execution, without applying the analysis's relations (Section 4.2). Both applications pass the second and third validity checks. Mifos also passes the first: its uninstrumented run fails the same tests. Realworld has only instrumented run records, all without failures.

Table 5.15 gives the final figures for both applications. With the extractor frozen before this check's logs were read, the graph held 12/22 mifos suite pairs. Post-hoc classification found among the misses three source forms the extractor did not support,

Table 5.15: The graph against the pairs of screens the applications' own end-to-end suites navigate between.

	realworld	mifos
Navigations completed	141	62
Distinct pairs	19	22
Pairs in the graph (hits)	19	19
through the source screen or a scope that names it	16	17
only through the shell scope	3	2
Hits without attribution	13	8
Hits at the rooted level	19	16
Pairs redirected (redirect bucket)	0	5
of which hits	0	2
Pairs the expanded graph contains	71	162567
Possible ordered pairs of screens, self-pairs included	100	367236
Misses	0	3
redirect	0	3
base	0	0
dynamic	0	0
other	0	0
Misses that meet the dynamic test in any order	0	2

Note. A redirected navigation is compared on the address it asked for (the preregistered rule). *Rooted* is the level that stands for earlier work's mechanism. The miss classes after *redirect* were defined after the misses had been read, and are tested in the order redirect, base, dynamic, first match winning. All cells are generated figures.

rather than destinations known only at run time, and each form was then repaired with a test (Table A1). The repairs apply general Angular rules without naming sites, but this check found and evaluated them. Only the frozen figure is therefore independent of these repairs, and no other runnable suite remains for an independent test (Section 6.2).

- *realworld.* The result does not bear on what the levels above *rooted* add, because the *rooted* level already contains every pair.
- *mifos, hits.* Table A1 records the scorer's departure from the preregistered rule. As predicted, no matched pair has a must edge. One of the 2 redirect-bucket hits depends solely on the notifications tray. The tray is a shell site whose target retains two leading parameters, so it connects every screen to every account screen. The hit shows that the graph contains the pair, but it does not show that the site was resolved. After the repairs, the same site widens the graph from 28185 to 162567/367236 possible pairs, which reduces discrimination. Uniform pairs at that density would yield 9.7/22 hits (realworld: 13.5/19). That chance level is not the suite's expected success rate, and the observed hits demonstrate containment more than discrimination. No other preregistered prediction is reported here.

Table 5.16: The graph without attribution, with the mechanism of earlier work (*rooted*), and with the full analysis.

	no attribution	rooted	full
Suite transitions in the graph, realworld	13/19	19/19	19/19
Suite transitions in the graph, mifos	8/22	16/22	19/22
Graph pairs, realworld	24	69	71
Graph pairs, mifos	4275	158873	162567
Sites placed on at least one screen, corpus	–	1752	1802
Navigation inside services, constructs lost at baseline and recovered	–	–	20/48
Rima et al.'s reference pairs, their seven subjects	42/73	60/73	68/73
Sites placed in services, guards and effects	–	–	36

Note. Suite transitions are the distinct pairs of screens the application's own end-to-end suite navigates between (Section 5.3.5); *graph pairs* are those the graph holds. Only the full column of the suite rows was preregistered. The no-attribution graphs and realworld's rooted graph come from existing measurements; the mifos rooted graph was mapped for this comparison after the suites had been scored. The row for Rima et al.'s subjects is out-of-sample (Section 5.3.7). A dash marks a figure not measured. All cells are generated figures.

- *mifos, misses.* Of the miss classes, the redirect class alone was preregistered, and its misses are not classified further. The other classes were defined after the misses had been read, before the repairs. Every remaining miss is in the redirect class (3/3): the graph holds the pair on the screen the redirect lands on. For 1 of them a better reader would also hold the address asked for (Section 6.3). The dynamic test is also met by 2 of them. The test fires on any unresolved site in the source component or above it, so it does not show that a missed navigation is that site. The preregistered five-class scheme, and its prediction that misses would mostly be a scope too narrow or a site not read, are not scored.

5.3.6 What scope attribution adds

Table 5.16 compares no attribution, earlier work's mechanism (*rooted*) and full analysis. Rooted reimplements the mechanism using this extractor's nodes (Section 5.3.3). Only Rima et al.'s subjects are out-of-sample (Section 5.3.7).

Three things follow, with the figures in Table 5.16.

- *Attribution puts more of the suites' transitions into the graph, in a wider graph.*
- *The gain is reached by the mechanisms of earlier work.* On realworld the rooted level holds every suite pair. On mifos it holds 16. The other 3 hits have a child tab as source and a site in the parent view. The *outlet* level, which stands for the enclosing routes of Rima et al.'s specification (Section 5.3.3), places such sites on the child screens and holds 19 pairs, as many as the full analysis. These pairs were missed before the repairs, so the difference is in-sample (Section 5.3.5).

Oshima and Kishi's restrictions on `children` attributes and route parameters (Section 2.4) admit 0 of the 6 corpus applications, which all declare both. Rima et al. state no such restriction, and their specification covers the *outlet* level.

- *The service join generalises the router-service tracing of Rima et al.'s tool.* Their tool follows a call into a service whose class name matches a router or navigation pattern, to a bounded depth. A navigation call inside an injected service belongs to no component, so on every application of the corpus the levels up to *outlet* cannot place it on any screen (Table 5.5). The member-level call graph is more precise than a class-level join on realworld, but on alfresco the precision order reverses (Table 5.14). The levels above *rooted* find the reference pairs that rooted misses. On the two applications with suites, the step above *outlet* adds no transition the suites take.

Shell links produce wide scopes, although computed scopes are 1.7 to 18.7 times narrower than universal attribution (Section 5.3.2). Width alone does not establish over-attribution. Full produces no excess reference pair on realworld. On alfresco it produces 9 excess pairs, all at the configuration-driven site of Section 5.3.4, and it places an undetermined site at $\top$. Both references cover two applications and share the limit stated in Section 5.3.4. Over-attribution can pass witness checks, and the suites test omissions alone (Section 6.2).

5.3.7 On the applications of Rima et al.

Rima et al. publish seven Angular subjects with ground truths. Each ground truth is a set of enumerated workflows with manually annotated start and terminal routes [64]. None of the subjects was used in extractor development. Two preregistered comparisons use them. The first is this thesis's only out-of-sample measurement: preregistration preceded the scorer and the results, and the extractor was frozen before extraction. The second uses the extractor repaired after the first (Table A1) and is therefore in-sample. Their registry pins no commits, so each subject uses its last commit before its ground-truth date. The AI assistant assessed both comparisons, and a stronger model reassessed a gated subset specified below.

The preregistered unit is a reference pair: when an entry's terminal differs from every start route, each start is paired with the terminal. Their tool contains every reference pair by construction, because its output generated the reference. Their published measure is test-execution success rather than graph recall. This comparison therefore measures how many of their pipeline's pairs the graph contains and examines the graph's additional pairs. It does not rank the tools.

Table 5.17 gives the result. The full level holds 68 of the 73 reference pairs, and the *outlet* level holds 65. Two of the four preregistered predictions held: the full level holds at least 90 %, and the *outlet* level is within five points of it. The other two failed. The third predicted that most misses would be a route the graph lacks or a navigation outside the router, and no miss was classed as either. The wildcard pair, whose route

Table 5.17: Rima et al.'s reference pairs held at each attribution level, and the graph's other pairs.

		Reference pairs held				Other graph pairs	
Subject	Pairs	none	rooted	outlet	full	all	real/read
posts-users	4	3	4	4	4	35	35/35
heroes	0	0	0	0	0	9	9/9
airbus	3	0	0	0	3	20	17/20
petclinic	35	25	32	32	32	152	35/40
traduora	11	9	9	9	9	104	37/40
jumpstart	11	3	11	11	11	29	29/29
event-booking	9	2	4	9	9	8	8/8
All, preregistered	73	57.5 %	82.2 %	89.0 %	93.2 %	357	170/181
Without 3 read as absent[†]	70	60.0 %	85.7 %	92.9 %	97.1 %		
Per start route[†]	292	21.2 %	64.0 %	87.3 %	88.4 %		
Repaired extractor[†]	73	58.9 %	83.6 %	90.4 %	94.5 %	341	

Note. *none* is the graph without attribution; *rooted* and *outlet* are the rules of Oshima and Kishi and of Rima et al.'s specification (Section 5.3.3). heroes has no pair under the preregistered rule: its entries are external links or list their terminal among their start routes. *Read*: pairs judged from the source, all of them where a subject has at most 40 other pairs and a seeded sample of 40 otherwise. The reference's wildcard route matches no screen, so its one pair counts as missed, while the graph's own wildcard screen is counted among its other pairs. [†]Post hoc, not preregistered: the first drops the reference pairs the source shows to be absent, and only missed pairs were read, so it can only rise; the second keeps the entries the preregistered rule drops, and its misses were not read; the third maps the subjects again after the repairs this comparison prompted, which gain the locale link and remove three invalid non-reference airbus pairs from sign-in, one traduora pair and petclinic's target over-approximations. All cells are generated figures.

matches no screen, was classed as absent from the source. The fourth predicted that most other pairs would come from the shell scope, but 63 of the 357 are reached only from it.

Of the 5 misses, the source holds 3 as no transition at all: dropdown toggles, a form with no router and a form that returns to the owner's page. One relative link whose only segment is an expression causes the other 2. The extractor leaves its target unknown, although the route it reaches can be read. One of these misses also starts on a route that only redirects. The step from *outlet* to *full* is 3 pairs, all on airbus, where the navigation runs inside an injected router service. The specification's rule does not follow such a service, but their implementation traces it to a bounded depth (Section 2.4). The preregistered rule drops every entry whose component is also shown on its target, which removes most links of a shell. The per-start-route row of Table 5.17 counts them.

Besides the counted pairs, the graph holds 357 others, and 190 of them are pairs of the entries the preregistered rule drops. Of the 181 pairs read from the source, 170 are transitions the application has and 11 are not. A navigation whose target is matched to longer routes than its command can produce accounts for 6 of the spurious pairs. A

navigation placed on a sign-in or password-reset screen, where the component holding it is not shown, accounts for the other 5. Read per start route, Rima et al.'s reference lists these 5 pairs too. All 11 spurious pairs are at the may tier: the number with a must edge is 0.

All assessments use one AI model family, so they establish consistency rather than correctness. The run logs of the head-to-head comparison record Claude Sonnet 5.5 as the reader and Claude Opus 5.5 as the stronger model. The archived records of the other readings do not name the model, so for them the model family rests on my own record. A stronger model reassessed every miss, every pair not judged real and every sixth real pair. It reassessed 45 pairs and changed 3 of the verdicts. One batch was repeated because its first response omitted all 20 additional pairs. Adversarial review later found that the second response wrongly classified a shell bar as rendered on sign-in and password-reset screens, where a guard hides it (Section 6.3). Manual correction changed 2 of the verdicts and 1 of the citations. A script verifies the cited navigation sites but not their rendering on start screens.

The comparison prompted four repairs (Table A1), after which the post-hoc re-extraction holds 69/73 reference pairs and 341 additional pairs, none of them reassessed.

5.3.8 The tool of Rima et al. on the same applications

This comparison runs their tool at `f33003f`. Its static extraction and workflow enumeration stages ran on their seven subjects and the six corpus applications; backend-dependent test stages did not run. Preregistration preceded inspection of its corpus output. Workflow counts match their reference on every subject; start and terminal routes also match on the six with the same reference format. Their output lists workflows, which the comparison converts to pairs: a workflow whose trigger resolves to a navigation gives one pair from each of its start routes to its terminal route. Other workflows record their start route as terminal and are excluded. This extractor's pairs come from its edges once scope nodes are expanded. Relative recall concerns real pairs in the tools' union, not all real transitions. Proposing more pairs enlarges that reference union. Within each application, pairs are stratified as shared, only-theirs or only-ours. An only-ours stratum exceeding 1000 pairs is split into shell-only and other pairs. Every pair in each stratum, or a seeded sample of 12 when larger, was assessed from source without disclosing the proposing tool. Precision estimates the real share of a tool's pairs; relative recall estimates its share of real pairs in the union. A pair judged unclear counts as not real, so both measures estimate pairs judged real.

Their tool records constant-valued paths as expression text and omits lazy-module parent prefixes. None of its 88 ghostfolio paths names a declared route. To avoid losing pairs through naming differences, routes were matched by declaration site: first to the recorded path, then mounts ending in that path, otherwise all mounts. Moves between multiply mounted routes remain within one mount. This rule was chosen after inspecting their output.

Table 5.18: Rima et al.'s tool and this extractor on the same thirteen applications.

	Rima et al.'s tool			This extractor		
Application	pairs	precision	recall	pairs	precision	recall
posts-users	39	100.0 %	100.0 %	39	100.0 %	100.0 %
heroes	6	100.0 %	100.0 %	6	100.0 %	100.0 %
airbus	23	87.0 %	100.0 %	20	100.0 %	100.0 %
petclinic	169	99.4 %	100.0 %	165	98.8 %	97.0 %
traduora	113	99.1 %	99.1 %	113	100.0 %	100.0 %
jumpstart	30	100.0 %	75.0 %	40	100.0 %	100.0 %
event-booking	20	73.3 %	93.6 %	17	92.2 %	100.0 %
realworld	66	95.5 %	100.0 %	63	100.0 %	100.0 %
alfresco	23	84.8 %	41.9 %	29	100.0 %	62.4 %
ghostfolio	10	30.0 %	0.4 %	1039	66.8 %	100.0 %
k8s dashboard	5	40.0 %	0.8 %	393	66.2 %	99.2 %
bitwarden	113	67.0 %	14.0 %	560	83.4 %	86.2 %
mifos	11095	80.0 %	77.4 %	161363	6.5 %	92.2 %
Their seven subjects	400	97.4 %	97.0 %	400	99.2 %	98.8 %
Corpus	11312	79.9 %	69.1 %	163447	7.4 %	92.4 %
Corpus without mifos	217	75.2 %	10.2 %	2084	72.6 %	94.1 %

Note. *Recall* is relative: the share of the real pairs either tool finds. Real pairs are estimated per stratum from a reading of at most 12 pairs, so a figure over a large stratum is an estimate; the size of each stratum and the pairs read in it are in `rima/head2head/results-v3/score.json`. Routes are identified by declaration site, and the row without mifos was added after reading (both post hoc). All cells are generated figures.

Table 5.18 reports point estimates. On their seven subjects, the tools are close: theirs recalls 97.0 % at 97.4 % precision; this extractor recalls 98.8 % at 99.2 %.

Without mifos, a post-hoc group, this extractor finds a larger share of the pairs judged real at similar estimated precision. It recalls 94.1 % of real union pairs versus 10.2 %, at 72.6 % versus 75.2 % precision. This extractor proposes 2084 pairs and theirs 217, with estimated real counts of 1512 and 163.

On mifos, this extractor trades precision for recall. It proposes 161363 pairs, theirs 11095, and both share 9308. Most of this extractor's pairs are shell-only (Section 5.3.5); recall is 92.2 % versus 77.4 % and precision 6.5 % versus 80.0 %.

Setting each stratum's rate to its Wilson interval endpoints gives bounds, not a joint 95 % interval. Without mifos, the recall bounds stay separate: 88.4 %–97.1 % versus 5.6 %–17.9 %. The pooled corpus recall ranges, 79.6 %–99.4 % versus 10.3 %–85.6 %, rest mostly on mifos (Section 5.1) and do not rank the tools. They also depend on how unclear verdicts count: 1 sampled pair, in mifos's shell-only stratum, was judged unclear. Counted as real, it would put this extractor at 14.4 % precision and 96.0 % recall on the pooled corpus and their tool at 36.9 % recall; the figures without mifos do not change. The bounds assume independent sampled pairs despite site clustering,

and they cover sampling error alone. Systematic error from the single model family would shift both endpoints.

Three predictions were preregistered. The first, that this extractor's recall on the corpus is the higher, holds at the point estimate, and its bounds separate only without mifos, a group formed after reading. The second, that their precision on their subjects is at least this extractor's, did not hold, and neither did the third, that their tool fails on some application.

Assessment follows the first comparison's rules. It covers 263 pairs, of which a stronger model reassessed 114, changing 10 of the verdicts. I checked 41 sampled assessments underlying the five-application figures, three per stratum, against source. The model verdict was visible, so this was not blind and measures agreement rather than correctness. I judged 41 cases and agreed on 41 (Wilson 95 % interval 91.4 %–100.0 %). An initial pass incorrectly paired start routes of non-navigating workflows. It was withdrawn; valid remaining assessments were reused and 108 new sample pairs assessed. Mifos estimates reuse those samples after excluding cross-mount moves. Preregistration deviations are recorded with the results.

5.4 Change sensitivity (RQ4)

Mutation testing re-extracts each application per mutant, so it covers three of six applications: realworld and alfresco, used in earlier mutation runs, and guard-heavy bitwarden. Results use the attributed graph, harness mutate 1.9.1, a fixed seed and a quota of eight mutants per operator. Table A1 records harness corrections.

Table 5.19 gives the sample, the exclusions and the detections on the attributed graph and without attribution; the detection rate per operator and application, and the outcomes summed over the applications, are in the accompanying repository.

Rename detection is nearly guaranteed: mutants require an existing node for the renamed segment, and full paths identify screens (Section 3.1). The other operators are similarly restricted. Deletion samples only `routerLink` lines cited by existing edges. Retargeting requires a cited `navigate` call beginning with a plain absolute string. Guard removal is scored only if an edge carries that guard into the guarded screen; detection is conditional on that edge existing. Exclusions are few with attribution but more numerous without it. Samples are small (Section 6.2), favouring interpretation by operator pattern rather than rate. RQ4 checks witness-to-source mapping, not change-impact accuracy: only cited lines can fail, and the harness was corrected after inspecting misses (Table A1). The headline uses scored mutants, 67/67; the paired row includes only mutants scored by both graphs.

Guard removal succeeds if an edge loses a guard from the removed list; all 6 exclusions fail the eligibility condition above. Without attribution, class matching by imported component file cannot distinguish realworld's empty-path and `:slug` editor children, which load the same file. No edge enters `/editor`, but one enters `/editor/:slug`. The empty-path guard is consequently scored and missed rather than excluded: the sole paired disagreement reflects the harness, not the graph.

Table 5.19: Mutants sampled, excluded, scored and detected, with scope attribution on and off.

	attributed graph	without attribution
Candidate mutations found	272	–
Sampled under the quota	73	73
Excluded	6	22
guard: graph enters no screen the route yields	5	–
guard on a route with no component and no children	1	–
Scored	67	51
Detected	67	50
rename a route path	22/22	–
delete a `routerLink`	14/14	7/7
retarget a `navigate`	20/20	–
remove a guard	11/11	–
Paired mutants, detected	51/51	50/51

Note. An excluded mutant carries no information about sensitivity. All exclusions are guard removals: for the first kind no edge of the unmutated graph enters any screen the guarded route yields, and the second guards a route that declares neither a component nor children. *Paired* counts the mutants both graphs can score. A dash means the figure is not reported for that graph. Operator rows are detected over scored.

The mutation run neither confirms nor contradicts the defect class *guards unreliable in both directions*; Chapter 6 gives the reading. In this sample the graph responded to every scored change (67/67) in the constructs it represents, a result the construction makes close to certain.

5.5 What the losses were (RQ5)

RQ5 traces the defect classes of Section 5.2 to causes in the final version, using the three outcomes defined in Section 4.2.

5.5.1 The cause of each defect class

A *class* is a row of Table 5.4; a *cause* is its explanation from source review.

The three-part answer attributes repaired classes to tool defects, without extending that conclusion to all constructs. The 1631/1657 may-only witness locations involving application decisions (Table 5.1) remain may under rules that do not test whether the decision can vary. They therefore do not establish inherent application uncertainty. The lists count causes; Tables 5.10 and 5.20 count constructs, including 2/151 residual attribution constructs without a recorded reason.

Read by the final version. The information was in the source, so the loss was the tool's.

- *Lazy module boundary* and *constant-valued route paths*: causes in the extractor, read by the final version, and empty in Table 5.5.
- *Guards*: the two unread forms (a guard on a route a wrapper returns, a `canActivate` element written as a call) are read, and the guard rule is the router's own semantics (Section 3.1).
- The candidate class *node not linked to its component*: empty on the final version.
- Three forms that the end-to-end check exposed (Section 5.3.5, Table A1).

Still not read (open). The final version does not read these causes.

- *Attribution*: the share of its constructs recovered has a per-application median of 40.2 % and is 66.7 % pooled. The remainder is in Table 5.10, including sites that no relation the prototype implements reaches. It mixes gaps of the tool with components that no screen may render, and no measurement separates them.
- *Named router outlets*: a construct the model represents and the extractor does not implement. Its navigations are excluded from the denominator (7 sites) and its screens left out of the runtime route oracle's mounted set (64 route entries).
- The spread of computed routes in ghostfolio is a construct the extractor could read and does not. It hides 242 of the 251 mounted leaf screens that have no node.

Beyond a static reader of a route table. No reader of a route table can resolve these.

- ghostfolio's custom `UrlMatcher` computes a route's path in a function. No reader of a route table can resolve it, and both tools report it as a blind spot.
- mifos chooses between two route tables by a value injected at deployment, which the runtime route oracle of Section 5.5.3 finds and the sources leave undecided.

Resolved causes form three groups. *Unreached route tables:* a `loadChildren` module keeps routes in a separate routing module. Stopping early places screens at the application root. Both static readers made this error; only the runtime route oracle exposed it. The final extractor follows up to three hops (Table A1). *Referenced paths:* one application declares every path through a constant table; failing to resolve it collapses the graph to one node. *Referenced destinations:* targets may use fields, route-table chains, commands-array elements, builder returns or lookup values. Literal-initialised fields remain unresolved because they are mutable; resolving them as constants would turn runtime values into must edges.

Table 5.20: Construct accounting after the loss list was worked.

Application	Sites	Excl.	In scope	Recovered	Dynamic	Unplaced
realworld	40	1	39	100.0 %	0.0 %	0.0 %
alfresco	34	7	27	25.9 %	25.9 %	44.4 %
ghostfolio	198	12	186	48.4 %	44.1 %	6.5 %
k8s dashboard	72	4	68	17.6 %	70.6 %	8.8 %
bitwarden	131	19	112	64.3 %	14.3 %	17.9 %
mifos	1343	0	1343	94.0 %	3.1 %	2.5 %
All	1818	43	1775	83.5 %	11.0 %	4.7 %

Note. Percentages are of the in-scope sites. The accounted share, the sum of the recovered, dynamic and unplaced shares, is 99.2 % over all applications.

5.5.2 Construct accounting

Table 5.20 separates four outcomes. A site is *recovered* when a must or may edge from a placed screen witnesses it with a destination. It is *dynamic* when the destination is unresolved: backend values, history entries, fields treated as mutable despite route-table initialisation, or commands-array template literals (Section 6.3). These yield unknown-modality edges to a screen's `u_` sink. A site is *unplaced* when its target is resolvable but its source screen is not; its edge uses an `o_` origin. Remaining sites have a recorded reason but no transition. The check enforces reason presence, not correctness.

Before the graph is consulted, 43 sites are excluded on their construct text. They form three groups: 35 with an empty commands array that changes only query parameters, 1 with no destination, and 7 with a secondary outlet as target. The first two groups connect no screens. The model represents a secondary outlet as a second mount (Section 3.2.3), but the extractor does not implement it. Excluding the outlet sites removes a recall limitation from the denominator and so raises alfresco's ratios. Both columns of Table 5.7 use the same 1775 sites, and the comparison rejects any changed exclusion reason.

On k8s dashboard, recovery is low and many destinations depend on run time. Its dynamic sites span 42 files, 37 of them with one site each, which indicates a distributed coding pattern rather than a single defect. Mifos contributes 75.7 % of the in-scope sites. Without it, recovery is 50.9 % and the accounted share 97.9 %. Table A1 records the changes to the route denominators.

The named-outlet restriction reaches a named outlet's *subtree* too. The extractor created nodes for alfresco's viewer routes at the primary address, although Angular reaches those routes only as `(viewer:...)`. Both tools read the subtree the same way, so only the runtime route oracle of Section 5.5.3 exposed the error.

5.5.3 The final tier and the node set

The final tier. Among the 159 must edges of the attributed tier, the number of witness-check failures is 0. The checker cannot read the destination of 62 of them, so those edges are neither passes nor failures. All unread destinations are toolbar edges in one application, and they rely on the toolbar component's computed scope and on the route-order assumption. The check compares construct and edge destinations. It does not test unconditional execution or router admission, which the demotion rules address. Neither source inspection (Section 6.2) nor this mechanical check independently proves Claim 3.1.

The whole graph. The 13830 edges carry 15638 witness citations, including merged witnesses and repeated locations. Sites producing more edges therefore receive more weight. Only 7.0 % are clean; 87.2 % are over-approximations without a target-agreement claim. One `navigateByUrl` call to an empty-prefix template literal, `view-transaction.component.ts:135` in mifos, produces 39.4 % of edges (5454). Mifos contributes 89.9 % of all edges. Among 1437 judged citations, 76.2 % are clean; no remaining citation belongs to a must edge. Must-tier consistency does not describe the whole graph.

The node set. The number of screens the compared graph declares at unmounted addresses is 0. Every claimed address appears in the framework configuration. That match is external evidence for the six applications, because static readers can agree on an incorrect prefix. Node validity is a precondition of the must-tier property rather than part of it. The address-based census excludes named outlets and `UrlMatcher` entries (65 route entries) from both columns.

The runtime route oracle. The oracle loads each application, built in its development configuration, in a headless browser. It reads the live router's route configuration after expanding every lazily loaded subtree. An address counts as mounted when the configuration places a component there on the primary outlet. The oracle parses no route file. Of the 1146 mounted addresses, the graph represents 892/1146. The 251 missing addresses are leaf screens. Besides them, the mounted addresses without a node include prefixes, which are not missing screens: 2 with a redirect into the subtree below and 1 with a layout alone, 3 in all.

The applications were probed once, on 29–30 October, against an earlier graph already corrected using oracle findings. Against the final graph, the archived configuration gives identical counts and address sets on 6 of the 6 applications. The final counts are 892 for nodes, 0 for unmounted addresses and 251 for missing leaf screens. Navigation checks agree with the oracle on 41/41 addresses, and the 251 missing addresses were not checked individually.

5.6 Dead links, found from the source

A link to an unmounted target is a dead link. Static candidates are checked against the recorded runtime route configuration before reporting. None was navigated to in a running application; the list therefore establishes neither runtime occurrence nor confirmed defects.

The clearest case is mifos, where a transactions tab links to `/journal-entry/view/:id` although the application declares that screen at `/accounting/journal-entry/view/:id`. The link omits the parent prefix, and mifos's catch-all route renders the not-found screen, which a user can mistake for missing data rather than a broken link.

The list is not one defect class, and its groups differ in the strength of their evidence.

- 18 of the candidates are an absolute link written against a path the application does not mount, usually the name the server uses rather than the one the router does. These originate in application code; whether users encounter them is untested.
- 5 come from a *relative* `routerLink`, such as `[routerLink]="'edit'"` or `[routerLink]="['../../transactions']"`. Their target depends on the route the link resolves against, that of the component holding it (Section 3.2.6). The extractor's resolution of that base must therefore be verified before such a candidate can be attributed to an application defect.
- 6 cannot be classified because the result files preserve insufficient construct text. They are counted separately.

The last two groups provide weaker evidence. Counting source locations rather than targets gives 18, 3 and 6 respectively. One shared-template relative link resolves against three owning screens, explaining the difference.

A further 69 sites in ../../libs, ../aca-shared, ../browser, across 2 repositories, fall outside the measured application directories and are excluded from the census. Most belong to sibling frontends whose routers declare addresses that would otherwise be labelled dead.

5.7 Summary

This section answers each research question briefly. The sections cited give the figures and the limits.

RQ1. Every screen that a static reader can enumerate has a node, and the runtime route oracle finds no node at an address where the framework mounts nothing. Against the addresses the running framework mounts, the missing nodes lie almost entirely behind one construct in ghostfolio that the extractor declares unread. Deterministic

transitions are rare: 25 constructs earn must, and 125 of the 159 must edges come from one toolbar in k8s dashboard. That toolbar's computed scope is untested, and most must edges hold only under the unvalidated route-order assumption A1 (Sections 3.1 and 5.5.3). Most may-only witness locations involve a guard, branch or other application decision, and the demotion rules do not test whether that decision can vary (Section 5.1).

RQ2. Under the precedence of the loss rules, attribution is the largest loss category in every application. An attribution loss is a construct that the extractor, without attribution, cannot place on a screen it fires from. The evidence supports five defect classes and one candidate class (Table 5.4). The lazy-boundary, constant-path and unlinked-component classes have no instances on the final version (Section 5.2).

RQ3. Scope attribution recovers a per-application median of 40.2 % (range 14.3 %–100.0 %) of the constructs lost to attribution, and 66.7 % pooled. Almost all recovered constructs are at the may tier, so the gain in must conversion is small. Most of the recovery comes from mechanisms of earlier work, which the analysis includes as levels. Its added step places navigation in services, guards and effects, and on the alfresco reference it raises recall at a cost in precision. No check establishes that the computed scopes are not too large (Section 5.3).

RQ4. The graph responded to 67 of the 67 scored mutants. The operators act only at constructs the graph already cites, so the result is a conditional check of the mapping from witnesses to source, and change-impact accuracy remains unmeasured (Section 5.4).

RQ5. The repaired classes were defects of the tool, the attribution losses remain open, and two causes lie beyond any reader of a route table (Section 5.5). 2 of the 151 constructs still lost to attribution are labelled as having no reason found. The harness checks that every construct has a label, not that the label is correct.

Chapter 6 interprets these answers and their limits.

6 Discussion

This chapter interprets the results of Chapter 5. Section 6.1 answers the research questions and discusses two uses of the graph, and Section 6.2 records the threats to validity. Section 6.3 declares what the analysis approximates, Section 6.4 considers other routers, and Section 6.5 sets priorities for future work.

6.1 Key findings

The results locate extraction losses at two levels, the nodes and the edges, and each level limits the other.

RQ1: the node level. Node completeness has two definitions. Against the static readers of the route configuration, extraction is complete on all six applications (Table 5.3), and the loss classification uses this weaker definition. The runtime route oracle finds no node at an unmounted address, an absence of observed errors that falls short of a proof. It does find mounted leaf screens without nodes, most of them in ghostfolio (Section 5.5.3), so only 2 applications have a node for every mounted leaf screen. The constant-path and lazy-boundary defects of Section 5.2 are resolved in the final version, and their loss categories are empty (Section 5.5).

RQ1: the must tier. The demotion rules explain most of the must tier's small size, since guards, branches, callbacks and template conditions account for most demotions (Table 5.1). Proving that a guard always admits could enlarge the tier: ignoring all guards while keeping the other demotions raises it from 159 to 203 edges in a separate corpus extraction (`route-oracle/guard-only.mjs`). On k8s dashboard, 125 of the toolbar edges that attribution adds remain at must under these rules (Section 5.5.3). The rules were developed partly in response to red-team panels run with the AI assistant. The witness check finds no failure among the must edges, but it cannot read the destinations of about half of that application's must edges, all of them toolbar edges.

RQ2: what remains is lost at the edge. After scope attribution, the attribution losses that remain (Table 5.10) show three failure modes:

- a source scope that is still not found;
- a target whose text matches no node's address, mostly in ghostfolio, such as the name of a field holding the route;
- a fully dynamic target.

The other 2 have no diagnosed cause. The node and edge levels depend on each other. Attribution needs destination nodes. Complete nodes, in turn, yield few transitions

while sites outside routed components remain unattributed. Chapter 5 therefore reports node coverage beside must conversion.

Under the chosen rule order, which is part of the measure (Section 6.2), the largest loss category in every application is the one that scope attribution addresses (Table 5.5). The ranking is a property of this implementation. A lost construct enters the first category that applies (Table 4.3), so a shared-component construct rendered only on screens without nodes is an attribution loss that attribution alone cannot recover. The categories diagnose losses and do not bound what attribution can recover.

RQ3: what scope attribution achieved. Scope attribution is one model whose lower levels reimplement the mechanisms of earlier work: the shell rule is the *rooted* level, and the enclosing-route components of Rima et al.'s specification are the *outlet* level (Section 5.3.3). Most of what attribution recovers comes from these levels. The step above *outlet* places navigation that belongs to no component, in services, guards and effects, which no lower level can place on a screen. On alfresco, the one structural reference with enough service navigation to tell, the step raises recall and lowers precision below that of the lower levels and of the class join it refines (Section 5.3.4). On the two applications with suites, it adds no transition the suites exercised beyond *outlet* (Section 5.3.6). Most recovered constructs earn may, so attribution raises must conversion little (Section 5.3). The contribution of scope attribution is therefore this model and the measurement of what each of its levels adds. Section 6.2 states what the references and suites can establish.

RQ4: a sanity check, conditional on RQ1. The mutation test checks whether witness mapping reacts to injected edits, and it does not measure change-impact accuracy. Rename detection is almost guaranteed by construction, so the informative count leaves renames out. The other three operators are restricted to represented constructs in the simplest supported form. On their 45 scored mutants, which are the 67 of Section 5.4 without the renames, the attributed graph detects 45. The harness was corrected after its misses were examined (Section 4.3.2), so the result is not independent and need not generalise to a larger or differently drawn sample. Every scored change to a represented construct was visible, and RQ1 measures how often constructs are represented. Change impact queries are likewise limited to the represented part of an application.

Guard mutants are observable only if an edge enters a screen of the guarded route (Section 5.4). The class "guards unreliable in both directions" in Chapter 5 rests instead on two forms examined in Section 5.5: a guard on a route returned by a wrapper function, and a `canActivate` element written as a call. Red-team source review found these defects, which the witness check missed, and the mutation run neither confirms nor refutes the class.

RQ5: whose losses they are. The cause trace of Section 5.5.1 assigns every repaired class to the tool, and two open causes as well: the named outlets and

ghostfolio's computed routes. The losses that this study can explain are therefore mostly limits of this extractor rather than of static extraction. The evidence for a limit of the approach is narrow. It consists of the two causes beyond any reader of a route table, each found in one application. The rest is open, and no measurement here assigns it. Assigning the remaining attribution losses would need a check that separates tool gaps from components that no screen may render. Assigning the may-only locations behind application decisions would need rules that test whether a decision can vary.

Two uses of the graph. The graph is meant for two uses, neither of which was tested with consumers. The first is change impact analysis. Each edge records its construct's file and line, which links changed lines to edges and to their source and destination screens. This screen-level counterpart of Chianti [61] follows the impact analysis tradition of Arnold and Bohner [4, 12, 43]. Its only evidence here is RQ4, which tests the necessary mapping property on represented constructs under a design that almost guarantees detection and measures neither missed affected screens nor excess predictions.

The second use, continuous integration without application execution, is unmeasured. The graph can be recomputed deterministically on each commit without installing dependencies, serving or crawling the application (Section 4.3). Properties such as "a screen still has an inbound deterministic transition" can then be asserted like a type check. Guard assertions remain unsupported, and a screen without incoming edges supplies no evidence. A dynamically built model needs a running application, test data and an exploration budget [50, 51, 36]. In both uses the may tier generates candidates, and the must constructs are the ones intended to describe transitions guaranteed under Claim 3.1's assumptions.

Where both uses stop. Both uses meet dynamic targets, route tables assembled from startup configuration, and overlays without routes, which the graph omits or approximates at may without a soundness claim. The must tier is a lower bound only under Claim 3.1's assumptions about route order, ambiguous selector joins, unresolved calls and execution, and each assumption admits counterexamples. Section 3.1 counts the edges that rely on the first two. The graph alone therefore cannot answer queries that need every transition. Authentication also limits the must tier, because guards depend on unresolved state and Claim 3.1 requires their demotion.

6.2 Limitations and threats to validity

The tool's author evaluated one tool on one corpus with five methods and end-to-end checks on two applications, which limits the conclusions. The heaviest threat is that every corpus figure is in-sample. The structural references, which apply the analysis's own relations, and the corpus selection, which favours the tool, come next. Table 6.1 records each threat, its effect, the mitigation applied and what remains open.

Table 6.1: Threats to validity: the effect of each on the results, the mitigation applied, and what remains open.

Threat	Effect on the results	Mitigation, and what remains open
Internal validity		
Tool developed on its evaluation corpus	Every figure on the six applications is in-sample, because the extractor, parser, mutation harness and structural reference were corrected against their inputs (Section 4.3.2). The reference guided two repairs and the end-to-end check three. Corpus figures therefore do not estimate performance elsewhere.	The end-to-end check came closest to an out-of-sample corpus test: before its logs were read, the frozen extractor held 12/22 mifos pairs. The first comparison on Rima et al.'s seven subjects, mapped after freezing, is out-of-sample (Section 5.3.7). *Open:* the end-to-end preregistration shares a commit with its logs, so the archive cannot show which came first, whereas for Rima et al.'s subjects the commits place it before the scorer and results.
Dependencies not installed	TypeScript cannot resolve framework or third-party types, which weakens steps such as narrowing an injected receiver.	The configuration is reported. *Open:* the results apply only to this configuration.
Mutation experiment	Some operator–application samples are single-digit, and graph-based exclusions tie the sample to the tool's recovery.	The quota is seeded and exclusions are reported. Operator choice affects mutation results [57], so patterns are read rather than rates. *Open:* the sample is not independent of the artefact under test.
Test-source cross-check	It measures node existence on one application.	It adds the URLs that alfresco's end-to-end specifications name to the syntactic proxy. *Open:* the specifications are parsed without being run, so the URLs are expected addresses that no run visited.
Scope attribution		
Scope checked in both directions on two applications	The other methods of Chapter 4 never compare a site's computed screen set with the set from which it fires.	Three partial methods, below. *Open:* over-attribution passes every witness check, because the construct exists and names the target, and on the other four applications it is untested.
Structural references (Section 5.3.4)	They cover the two smallest applications, and on alfresco only 16 sites are determined (17 undetermined). There, full trades precision for recall against the lower levels and the class join it refines. On realworld, the replaced *class* rule adds 9 excess pairs at one site (Table 5.14).	The AI assistant built the alfresco reference without the graph, under a protocol committed before scoring. *Open:* both references apply the analysis's own relations, so agreement does not establish the true scope. The realworld reference was partly completed from graph output, and the call graph, written after its finding, is scored on it, so that score shows consistency with the repair and cannot validate it independently.

Threat	Effect on the results	Mitigation, and what remains open
End-to-end check (Section 5.3.5)	It checks whether the graph contains the navigations of two applications' own suites. On both, the enclosing-route level of earlier work already contains every pair that full contains (Section 5.3.6).	Section 5.3.5 lists the hits and misses. *Open:* the check covers two applications, the mifos result after the repairs is in-sample, and the mifos graph is wide enough that its hits show containment more than discrimination (Section 6.3).
Rendered-interface check (Section 5.3.1)	It checks one direction on 5 of six applications and 30/806 screens, with a graph that predates the final call graph.	*Open:* a predicted link missing from the DOM may need sign-in, so its absence cannot show over-attribution.
Construct validity		
Denominator of must conversion (Section 4.2)	It includes constructs in unmounted code and omits navigation outside the parser's forms, which are a `routerLink` attribute or binding and a `navigate` or `navigateByUrl` call on a receiver whose name contains *router*. Helper-wrapped navigation, `location.href` assignments, server-driven redirects and third-party navigation count neither way, so the figure is neither an upper nor a lower bound.	It is read as recall against an enumerable syntactic proxy, which the test-source cross-check extends. *Open:* proxy completeness is unmeasured.
Denominator of node coverage	Screens whose paths neither parser resolves are excluded, so parsing less can raise coverage.	The runtime route oracle gives a second reading (Section 5.5.3). *Open:* the threat is the stronger of the two denominator threats and acts in the opposite direction.
Witness check	Must conversion rests on must witnesses, whose target consistency is limited to what re-parsing the cited source resolves. 62 destinations stay unresolved and are reported apart.	Every must witness, merged ones included, is compared with its edge, and its destination with the independent route set. *Open:* the check establishes neither navigation execution nor runtime mounting; the runtime route oracle checks mounting separately.
Precedence of the loss rules	Testing targets first would make that category larger than attribution on 2 applications (alfresco and k8s dashboard).	Category sizes are compared within an application under a fixed order (Chapter 5). *Open:* comparison across orders is unsupported.
External validity		
Corpus and selection	Six open-source applications at single commits may differ from proprietary ones in size, routing and navigation [34, 35]. Selection favours the tool: after screening began, `SAP/spartacus` was rejected for unsupported configuration-driven routing (Section 4.1), so the results are likely optimistic for Angular applications in general. Mifos dominates pooled percentages, so the per-application median, 40.2 %, is primary.	Six criteria were fixed before results, and rejections are reported with their reasons. *Open:* tool support was outside the criteria, and two retained applications fail the test-suite criterion (Section 4.1).

Threat	Effect on the results	Mitigation, and what remains open
Application size and test density	Route-tree size and end-to-end test density were inversely related among the candidates (Section 4.1). No application offers both a large route tree and dense execution-based ground truth: the mifos suite covers 22 transitions against ≥909 route entries.	Methods that need suites run where suites exist. *Open:* the execution-based evidence cannot confirm the findings about scale.
Uneven outside evidence	Parsed test URLs come only from alfresco, partial rendered-link observations from 5 applications against an earlier graph, executed suites from realworld and mifos, and runtime configurations from all 6.	*Open:* the findings about scale rest mainly on runtime configurations and the extractor's own parsing.
Route declaration style	Of the three mutated applications, realworld and alfresco mostly register standalone route arrays with `provideRouter`, and bitwarden mostly uses NgModule routing modules through `RouterModule.forRoot` and `forChild`. Every scored mutant was detected under both styles (Section 5.4).	*Open:* the corpus was not stratified by style and three applications were not mutated, so nothing general about the two styles follows.
Angular only	The results concern Angular, while the graph model and lattice are framework-independent.	Section 6.4 discusses other routers. *Open:* transfer is unmeasured.
Oracles and experimenter		
Oracle independence	The independent parser shares no code with the extractor but may be wrong. Both apply the same rules to forms added during development, namely frozen tables, constant-built paths and workspace aliases (Table A1), so agreement on those forms is weaker evidence.	Disagreements were resolved against the source. *Open:* the author of both resolved them, and differential testing detects disagreement without establishing correctness.
Experimenter bias	I wrote the extractor, designed the evaluation, selected the corpus and defined the defect classes, the last after initial observations. The experimenter was unblinded.	Commits and harness are pinned. Tables are generated from results (Section 4.3) except the hand-compiled defect-class and witness/test-source summaries, and all protocol corrections are reported, unfavourable ones included (Section 4.3.2). *Open:* the loss classification had no second annotator, and residual constructs had no independent audit.

Threat	Effect on the results	Mitigation, and what remains open
AI-assistant reading of the final must tier	The declared AI assistant, which also helped write the extractor and the parser, reviewed the final must tier's 25 constructs with extractor decisions visible. Both Rima comparisons used one model family, and second passes changed 3 of 45 and 10 of 114 verdicts.	In the corpus comparison the model was blind to tool identity. My 41 re-reads, with model verdicts visible and tool identity hidden, agreed in 41 of 41 cases (Wilson 95 % interval 91.4 %–100.0 %). *Open:* no human second reader checked the review, agreement shows rule consistency without establishing correctness, and the experimenter was unblinded.
Two lines of evidence per defect class	The requirement limits fitting, and it leaves one class as a candidate.	*Open:* three of five accepted classes combine one named method with my observations, and the guard class rests on source review alone (Table 5.4).
The runtime route oracle (Section 5.5.3)		
Oracle selection effect	The oracle can expose wrong addresses that both static parsers share, but it needs applications that build and boot, which biases the selection.	*Open:* the selection is not random.
Configuration, not rendered page; workspace scope; development build	Routes count as mounted even if guards always refuse them or rendering throws. A workspace may mount routes outside the measured library, as in two of alfresco's three differences. A production build removes the required debug interface, so all builds are development builds.	*Open:* the first two enlarge the mounted set beyond reachable screens, which eases containment. A production configuration may add or remove routes in no fixed direction.

These oracle threats enlarge the mounted set and do not mark mounted addresses as unmounted. Under the abstraction of Section 5.5.3 and the recorded development configuration, the graph contains 0 unmounted addresses, so no containment failure was observed there. The observation does not prove failures impossible.

6.3 Soundiness and declared approximations

The thesis claims soundness for the must tier alone, over the declared route set and under the assumptions of Claim 3.1. Following the soundiness position of Livshits et al. [47] (Section 2.6.2), the claim comes with a declared list of the constructs the analysis approximates, so that a user of the graph can tell which queries it answers. The main approximations and their effects are these.

- *Dynamic route targets.* A target built at runtime cannot in general be enumerated from the source. Without a static part, it becomes an edge of unknown modality into its source node's dynamic sink. With a static leading part, it becomes may edges to every declared route that extends the prefix, which can still omit routes

with parameters at the prefix's positions. Such navigations yield no must edge and count as dynamic losses.

- *Navigation inside reactive signal effects.* Whether an `effect` or a `subscribe` callback fires depends on a runtime dependency graph instead of textual control flow (Sections 2.7.3 and 2.8). The model labels such navigation unknown with a symbolic read-set guard, which the prototype does not implement. The navigation receives its class's scope instead and stays at may, so the graph names the possible exits without their firing conditions.
- *Injection tokens and provider bindings resolved application-wide.* The call graph resolves field calls through declared types and joins the providers of one token instead of resolving them per injector, and it does not support factory or value providers. The joins can only widen a scope, and an unresolved token adds no edge.

Table B1 in Appendix B lists the constructs the analysis reads exactly or approximates, with the consequences for queries. Failures on exactly read constructs are defects counted in Chapter 5, whereas approximated constructs carry no completeness claim. They remain in the construct denominator of must conversion (Table 4.2) as dynamic, unplaced or attribution losses. The runtime route oracle (Section 5.5.3) found the omissions in the first two rows of Table B1, which inspecting the analysis had missed. Before the repairs in Table A1, mifos's end-to-end check exposed a relative-link base error in a template shared with child tabs, and no such miss remains (Section 5.3.5).

Why the may tier is incomplete. The may tier is intended to over-approximate (Section 3.1) but is not a proved upper bound, because the first-route, one-segment, catch-all-order, dispatch and dialog rules can omit transitions. Missing application transitions are therefore defects, and excess may edges are not. Table 6.2 classifies every remaining miss in the two suites (Section 5.3.5) and the two structural references (Section 5.3.4) by where its destination is fixed. *Inside* the static boundary, frontend code, installed libraries or committed configuration fix the destination, so recovering the pair needs a better parser, and such a miss marks a limit of this tool, not of static extraction. *Outside*, targets or routes depend on backend data, fetched configuration or user input. *Held* pairs exist at the landed screen but are scored at the requested address under the preregistered redirect rule, so better parsing cannot make them hits and they belong to neither side.

All alfresco reference misses lie inside (133 of 133). Mifos's 3 suite misses divide into 1 inside and 2 held, and realworld has no miss in either measure. Recovering every inside miss would yield 100.0 % of alfresco's reference pairs (now 23.6 %) and 90.9 % of mifos's suite pairs (now 86.4 %). On the references the ceiling is weak evidence, since they place every pair inside by construction (Section 6.2). Only the suites can reveal outside misses, and neither does. The ceiling excludes unexercised causes, such as the two cases beyond a static route-table reader in Section 5.5.1, and excess

Table 6.2: Why the graph still misses transitions: every remaining miss, grouped by gap.

Gap	Example	Boundary	Fix	Pairs
Shell from a library (alfresco)	`src/lib/components/sidenav/components/sidenav-header.component.html:20`	inside	read the shell route that @alfresco/adf-core/shell provides under node_modules and the extension JSON in the repository that fills it	129
Routed component nested in another screen's template (alfresco)	`src/lib/components/files/files.component.ts:143`	inside	attribute its sites to the screens of the component that nests it as well as to its own routes	4
Redirected navigation (mifos)	`src/app/login/login.component.ts:121`	held	none in the reader: the graph holds the pair on the screen the redirect lands on, and the rule scores the address asked for	2
Default commands of a service navigation (mifos)	`src/app/clients/clients-view/client-actions/client-action-notifier.service.ts:34`	inside	read a parameter's default value and the route each caller passes as relativeTo	1

Note. Covers the two suites and the two structural references. *Pairs* counts missing suite pairs and missing (site, screen) pairs of a reference. JSON stands for JavaScript Object Notation.

may edges: mifos's graph contains 162567 of 367236 possible pairs (Section 5.3.5). The author classified each miss, and the joining script rejects missing and surplus classifications.

6.4 Transfer to other frameworks

All measurements concern Angular. The following account of React Router and Vue Router is qualitative, based on documented behaviour [60, 80]. The graph model, scope lattice, fixpoint, anchored edges and expansion query would transfer unchanged. So would outlets: all three routers let a parent route render a placeholder for its matched child. The other four input relations in Section 3.2 require framework-specific derivation. Hooks and composables simplify the call graph by giving navigation sites owning components. React nesting is harder, because recovering component relationships from JavaScript XML (JSX), the markup syntax of React, is a dataflow problem resembling points-to analysis. A React Router loader redirect plays the role of `canActivate`, and Vue's global `beforeEach` most closely resembles `canActivateChild` on a root route anchored at $\top$.

Redirect semantics differ. Vue redirects a record only when it is the last matched

record, so a redirecting parent can render its component for a matched child, which Angular's validator forbids. Applying Angular's rule in the Vue extractor dropped those components. The Vue extractor retains them and records a residual limitation at the parent: visiting its own address redirects without mounting its component. Mixing Vue 2's catch-all `'*'` with Vue 3's `'/:pathMatch(.*)*'` maps both to one address. The extractor keeps the first declaration, possibly the wrong one, and reports the collision. These expectations need evaluation on real applications (Section 6.5).

6.5 Future work

Priorities, ordered by their potential effect on the claims, are:

- *An oracle for screen sets on a large application.* The references in Section 5.3.4 cover the two smallest applications using the analysis's relations, but most attributions concern applications with hundreds of screens (Table 5.12). Observe rendered links after sign-in or annotate a stratified sample of sites.
- *Unimplemented model features.* These include dispatch relations for NgRx effects and router events, the injector hierarchy, since provider bindings are currently resolved application-wide (Section 6.3), and guards returning a `UrlTree`. The broader guard model in Chapter 3 requires substantial work, and guard-removal tests currently reach only recorded guards (Section 5.4).
- *A runtime tier.* Observation identifiers are already valid witnesses. Comparison with dynamic approaches [50, 51, 74] would measure transitions found only by a crawler or only statically. Observation confirms a may edge in one run without promoting it to must, and an unobserved candidate remains may, preserving Claim 3.1. This tier is not built or evaluated here.
- *A broader corpus.* Automated selection from a larger repository pool, retaining poorly handled applications [34, 35], would reduce tool-driven bias (Section 6.2). A proprietary industrial case study, reporting aggregates, would address external validity. A minimum density of user-interface tests should become an explicit selection criterion whenever execution-based ground truth is needed. Route-tree size and test density were inversely related among the candidates (Section 4.1), and without such a criterion a study ends up with its oracle and its scale on different applications.
- *Consumer evaluation.* Controlled experiments are needed to test benefits for mobile testing agents, coding agents (Chapter 1) and automated test generation. None was attempted here.

6.6 Summary

Node extraction is complete against the route configuration that static readers can read, although some running applications mount leaf screens that have no node, and most

remaining losses occur at the edge. Scope attribution states the mechanisms of earlier work as nested levels of one model. The added step places navigation in services, guards and effects, at a cost in precision on the one reference that can tell, and adds no transition that the suites exercised. The graph responds to changes in the constructs it represents, and the repaired losses were the tool’s. The heaviest threat is that every corpus figure is in-sample, and transfer to other routers remains unmeasured. The priorities are an oracle for screen sets on a large application and a runtime tier that extends the graph to signed-in use.

7 Conclusion

This thesis measured how much navigation static extraction recovers from six Angular applications, classified the navigation it loses and evaluated scope attribution as a way to recover part of that loss. Section 5.7 answers the five research questions with their main limits, and Chapter 6 interprets the answers.

Contributions. The thesis makes two contributions. The first is scope attribution, a formal model of the screens from which a navigation site can fire, which contains the attribution rules of earlier Angular extractions as levels. The second measures what each level adds and what static extraction loses. The measurement covers six pinned applications, uses five evaluation methods with different blind spots and four further scope checks, and classifies every lost construct. The extractor also supports nested and parameterised route trees, which Oshima and Kishi exclude (Section 5.3.6).

The apparatus makes the measurements inspectable: tables are generated from harness results, and the source-based methods re-run from pinned commits. Runtime reproduction is partial: the route oracle re-runs in part, the realworld end-to-end check uses an unpinned public backend, and superseded figures cannot be regenerated.

Scope attribution is specified as a dataflow analysis over screen sets and implemented over three of its five input relations (Section 3.2). Its added step places navigation in services, guards and effects. The step's service join is type-directed and transitive, so it generalises the router-service tracing in Rima et al.'s implementation, which is bounded to one level and matches by name. With a fallback join and stand-ins for guards and effects, the step places 36 further cited source locations beyond outlet descent, and the call graph alone 7 beyond the per-class rule it replaced: 2.0 % and 0.4 % of all locations the full level places (Section 5.3.3).

What remains open. Dispatch is specified but not implemented, guards use a stand-in, and the call graph ignores the injector hierarchy. Screen sets were checked in both directions on two applications only, against references that apply the analysis's own relations. Rendered-interface checks covered only the screens reachable without signing in, so testing the rest needs signed-in execution (Section 6.5). Consumer benefit, including support for UI agents and test generation, remains unmeasured.

The main corpus results, and those on the subjects of Rima et al. after the extractor was repaired on them, are in-sample (Section 6.2 and Appendix A).

Part of the evidence rests on AI readings. The must-tier judgments were made by the AI assistant, and two readings by one model measure consistency rather than correctness. The comparison with Rima et al.'s tool uses small AI-read samples, and I confirmed 41 of the 41 cases I re-read (Section 5.3.8).

A route table alone is insufficient for static navigation extraction. May edges are candidates that need further checks. The must transitions, which come from 25 constructs, hold only under the assumptions of Claim 3.1.

References

[1] Manuel Alabor and Markus Stolze. Debugging of RxJS-based applications. In *Proceedings of the 7th ACM SIGPLAN International Workshop on Reactive and Event-Based Languages and Systems (REBLS)*, pages 15–24. ACM, 2020.

[2] Lars Ole Andersen. *Program Analysis and Specialization for the C Programming Language*. PhD thesis, DIKU, University of Copenhagen, 1994. DIKU report 94/19.

[3] Anastasios Antoniadis, Nikos Filippakis, Paddy Krishnan, Raghavendra Ramesh, Nicholas Allen, and Yannis Smaragdakis. Static analysis of Java enterprise applications: Frameworks and caches, the elephants in the room. In *Proceedings of the 41st ACM SIGPLAN Conference on Programming Language Design and Implementation (PLDI)*, pages 794–807, 2020.

[4] Robert S. Arnold and Shawn A. Bohner. Impact analysis – towards a framework for comparison. In *Proceedings of the Conference on Software Maintenance (CSM)*, pages 292–301. IEEE Computer Society Press, 1993.

[5] David F. Bacon and Peter F. Sweeney. Fast static analysis of C++ virtual function calls. In *Proceedings of the 11th ACM SIGPLAN Conference on Object-Oriented Programming, Systems, Languages, and Applications (OOPSLA)*, pages 324–341, 1996.

[6] Engineer Bainomugisha, Andoni Lombide Carreton, Tom Van Cutsem, Stijn Mostinckx, and Wolfgang De Meuter. A survey on reactive programming. *ACM Computing Surveys*, 45(4):52:1–52:34, 2013.

[7] Herman Banken, Erik Meijer, and Georgios Gousios. Debugging data flows in reactive programs. In *Proceedings of the 40th International Conference on Software Engineering (ICSE)*, pages 752–763. ACM, 2018.

[8] Ira D. Baxter, A. Yahin, L. Moura, M. Sant'Anna, and L. Bier. Clone detection using abstract syntax trees. In *Proceedings of the International Conference on Software Maintenance (ICSM)*, pages 368–377. IEEE Computer Society, 1998.

[9] Carlo Bellettini, Alessandro Marchetto, and Andrea Trentini. WebUml: Reverse engineering of web applications. In *Proceedings of the 2004 ACM Symposium on Applied Computing (SAC)*, pages 1662–1669, 2004.

[10] Gavin Bierman, Martín Abadi, and Mads Torgersen. Understanding TypeScript. In Richard Jones, editor, *ECOOP 2014 – Object-Oriented Programming*, volume 8586 of *Lecture Notes in Computer Science*, pages 257–281, Berlin, Heidelberg, 2014. Springer.

[11] Justus Bogner and Manuel Merkel. To type or not to type? A systematic comparison of the software quality of JavaScript and TypeScript applications

on GitHub. In *Proceedings of the 19th International Conference on Mining Software Repositories (MSR)*, pages 658–669. ACM, 2022.

[12] Shawn A. Bohner and Robert S. Arnold, editors. *Software Change Impact Analysis*. IEEE Computer Society Press, Los Alamitos, CA, USA, 1996.

[13] Miao Chen, Tengfei Tu, Hua Zhang, Qiaoyan Wen, and Weihang Wang. Jasmine: A static analysis framework for Spring core technologies. In *Proceedings of the 37th IEEE/ACM International Conference on Automated Software Engineering (ASE)*, pages 1–13. ACM, 2022.

[14] Sen Chen, Lingling Fan, Chunyang Chen, Ting Su, Wenhe Li, Yang Liu, and Lihua Xu. StoryDroid: Automated generation of storyboard for Android apps. In *Proceedings of the 41st International Conference on Software Engineering (ICSE)*, pages 596–607, 2019.

[15] Gregory H. Cooper and Shriram Krishnamurthi. Embedding dynamic dataflow in a call-by-value language. In Peter Sestoft, editor, *Programming Languages and Systems, 15th European Symposium on Programming (ESOP 2006)*, volume 3924 of *Lecture Notes in Computer Science*, pages 294–308, Berlin, Heidelberg, 2006. Springer.

[16] Patrick Cousot and Radhia Cousot. Abstract interpretation: A unified lattice model for static analysis of programs by construction or approximation of fixpoints. In *Proceedings of the 4th ACM SIGACT-SIGPLAN Symposium on Principles of Programming Languages (POPL)*, pages 238–252, 1977.

[17] Jeffrey Dean, David Grove, and Craig Chambers. Optimization of object-oriented programs using static class hierarchy analysis. In Mario Tokoro and Remo Pareschi, editors, *ECOOP'95 – Object-Oriented Programming*, volume 952 of *Lecture Notes in Computer Science*, pages 77–101. Springer, 1995.

[18] Guillaume Demarty, Fabien Maronnaud, Gabriel Le Breton, and Sylvain Hallé. SiteHopper: Abstracting navigation state machines for the efficient verification of web applications. In *Web Services and Formal Methods (WS-FM 2012)*, volume 7843 of *Lecture Notes in Computer Science*, pages 103–117. Springer, 2013.

[19] Richard A. DeMillo, Richard J. Lipton, and Frederick G. Sayward. Hints on test data selection: Help for the practicing programmer. *Computer*, 11(4):34–41, 1978.

[20] Joscha Drechsler, Guido Salvaneschi, Ragnar Mogk, and Mira Mezini. Distributed REScala: An update algorithm for distributed reactive programming. In *Proceedings of the 2014 ACM International Conference on Object Oriented Programming Systems Languages & Applications (OOPSLA)*, pages 361–376. ACM, 2014.

[21] Conal Elliott and Paul Hudak. Functional reactive animation. In *Proceedings of the 2nd ACM SIGPLAN International Conference on Functional Programming (ICFP)*, pages 263–273, 1997.

[22] Asger Feldthaus and Anders Møller. Checking correctness of TypeScript interfaces for JavaScript libraries. In *Proceedings of the 2014 ACM International Conference on Object Oriented Programming Systems Languages & Applications (OOPSLA)*, pages 1–16. ACM, 2014.

[23] Asger Feldthaus, Max Schäfer, Manu Sridharan, Julian Dolby, and Frank Tip. Efficient construction of approximate call graphs for JavaScript IDE services. In *Proceedings of the 35th International Conference on Software Engineering (ICSE)*, pages 752–761, 2013.

[24] Martin Fowler. Inversion of control containers and the dependency injection pattern. `https://martinfowler.com/articles/injection.html`, January 2004.

[25] Zheng Gao, Christian Bird, and Earl T. Barr. To type or not to type: Quantifying detectable bugs in JavaScript. In *Proceedings of the 39th IEEE/ACM International Conference on Software Engineering (ICSE)*, pages 758–769. IEEE, 2017.

[26] Google. Angular documentation: Effects. `https://angular.dev/guide/signals/effect`, 2026. Accessed November 2026.

[27] Google. Angular documentation: Routing. `https://angular.dev/guide/routing`, 2026. Accessed September 2026.

[28] David Grove and Craig Chambers. A framework for call graph construction algorithms. *ACM Transactions on Programming Languages and Systems*, 23(6):685–746, 2001.

[29] Simon Holm Jensen, Anders Møller, and Peter Thiemann. Type analysis for JavaScript. In Jens Palsberg and Zhendong Su, editors, *Static Analysis, 16th International Symposium, SAS 2009*, volume 5673 of *Lecture Notes in Computer Science*, pages 238–255, Berlin, Heidelberg, 2009. Springer.

[30] Yue Jia and Mark Harman. An analysis and survey of the development of mutation testing. *IEEE Transactions on Software Engineering*, 37(5):649–678, 2011.

[31] Carlos E. Jimenez, John Yang, Alexander Wettig, Shunyu Yao, Kexin Pei, Ofir Press, and Karthik Narasimhan. SWE-bench: Can language models resolve real-world GitHub issues? In *The Twelfth International Conference on Learning Representations (ICLR)*, 2024.

[32] Ralph E. Johnson and Brian Foote. Designing reusable classes. *Journal of Object-Oriented Programming*, 1(2):22–35, June/July 1988.

[33] René Just, Darioush Jalali, Laura Inozemtseva, Michael D. Ernst, Reid Holmes, and Gordon Fraser. Are mutants a valid substitute for real faults in software testing? In *Proceedings of the 22nd ACM SIGSOFT International Symposium on Foundations of Software Engineering (FSE)*, pages 654–665. ACM, 2014.

[34] Eirini Kalliamvakou, Georgios Gousios, Kelly Blincoe, Leif Singer, Daniel M. German, and Daniela Damian. The promises and perils of mining GitHub. In *Proceedings of the 11th Working Conference on Mining Software Repositories (MSR)*, pages 92–101, 2014.

[35] Eirini Kalliamvakou, Georgios Gousios, Kelly Blincoe, Leif Singer, Daniel M. German, and Daniela Damian. An in-depth study of the promises and perils of mining GitHub. *Empirical Software Engineering*, 21(5):2035–2071, 2016.

[36] Iva Kertusha, Gebremariam Assres, Onur Duman, and Andrea Arcuri. A survey on web testing: On the rise of AI and applications in industry. *Science of Computer Programming*, 253:103473, 2026.

[37] Gary A. Kildall. A unified approach to global program optimization. In *Proceedings of the 1st Annual ACM SIGACT-SIGPLAN Symposium on Principles of Programming Languages (POPL)*, pages 194–206, 1973.

[38] Łukasz Kurant. Static components dependency graph detection with evaluation metrics in React.js projects. In *Communication Papers of the 20th Conference on Computer Science and Intelligence Systems (FedCSIS)*, volume 45 of *Annals of Computer Science and Information Systems*, pages 127–134, 2025.

[39] Duling Lai and Julia Rubin. Goal-driven exploration for Android applications. In *Proceedings of the 34th IEEE/ACM International Conference on Automated Software Engineering (ASE)*, pages 115–127, 2019.

[40] Rodrigo Laigner, Diogo Mendonça, Alessandro Garcia, and Marcos Kalinowski. Cataloging dependency injection anti-patterns in software systems. *Journal of Systems and Software*, 184:111125, 2022.

[41] Davy Landman, Alexander Serebrenik, and Jurgen J. Vinju. Challenges for static analysis of Java reflection – literature review and empirical study. In *Proceedings of the 39th IEEE/ACM International Conference on Software Engineering (ICSE)*, pages 507–518. IEEE, 2017.

[42] Kim G. Larsen and Bent Thomsen. A modal process logic. In *Proceedings of the Third Annual Symposium on Logic in Computer Science (LICS)*, pages 203–210, 1988.

[43] Steffen Lehnert. A taxonomy for software change impact analysis. In *Proceedings of the 12th International Workshop on Principles of Software Evolution and the 7th Annual ERCIM Workshop on Software Evolution (IWPSE-EVOL)*, pages 41–50. ACM, 2011.

[44] Jiawei Li, Jiahao Liu, Jian Mao, Jun Zeng, and Zhenkai Liang. UI-CTX: Understanding UI behaviors with code contexts for mobile applications. In *Network and Distributed System Security Symposium (NDSS)*, 2025.

[45] Changlin Liu, Hanlin Wang, Tianming Liu, Diandian Gu, Yun Ma, Haoyu Wang, and Xusheng Xiao. ProMal: Precise window transition graphs for Android via synergy of program analysis and machine learning. In *Proceedings of the 44th International Conference on Software Engineering (ICSE)*, pages 1755–1767, 2022.

[46] Zhe Liu, Chunyang Chen, Junjie Wang, Mengzhuo Chen, Boyu Wu, Xing Che, Dandan Wang, and Qing Wang. Make LLM a testing expert: Bringing human-like interaction to mobile GUI testing via functionality-aware decisions. In *Proceedings of the IEEE/ACM 46th International Conference on Software Engineering (ICSE)*, pages 1–13. ACM, 2024.

[47] Benjamin Livshits, Manu Sridharan, Yannis Smaragdakis, Ondřej Lhoták, J. Nelson Amaral, Bor-Yuh Evan Chang, Samuel Z. Guyer, Uday P. Khedker, Anders Møller, and Dimitrios Vardoulakis. In defense of soundiness: A manifesto. *Communications of the ACM*, 58(2), 2015.

[48] Magnus Madsen, Benjamin Livshits, and Michael Fanning. Practical static analysis of JavaScript applications in the presence of frameworks and libraries. In *Proceedings of the 2013 9th Joint Meeting on Foundations of Software Engineering (ESEC/FSE)*, pages 499–509. ACM, 2013.

[49] Magnus Madsen, Frank Tip, and Ondřej Lhoták. Static analysis of event-driven Node.js JavaScript applications. In *Proceedings of the 2015 ACM SIGPLAN International Conference on Object-Oriented Programming, Systems, Languages, and Applications (OOPSLA)*, pages 505–519. ACM, 2015.

[50] Ali Mesbah, Arie van Deursen, and Stefan Lenselink. Crawling Ajax-based web applications through dynamic analysis of user interface state changes. *ACM Transactions on the Web*, 6(1):1–30, 2012.

[51] Amin Milani Fard and Ali Mesbah. Feedback-directed exploration of web applications to derive test models. In *2013 IEEE 24th International Symposium on Software Reliability Engineering (ISSRE)*, pages 278–287, 2013.

[52] Shabnam Mirshokraie, Ali Mesbah, and Karthik Pattabiraman. Efficient JavaScript mutation testing. In *Proceedings of the 2013 IEEE Sixth International Conference on Software Testing, Verification and Validation (ICST)*, pages 74–83. IEEE, 2013.

[53] Shabnam Mirshokraie, Ali Mesbah, and Karthik Pattabiraman. Guided mutation testing for JavaScript web applications. *IEEE Transactions on Software Engineering*, 41(5):429–444, 2015.

[54] Flemming Nielson, Hanne Riis Nielson, and Chris Hankin. *Principles of Program Analysis*. Springer, Berlin, Heidelberg, 1999.

[55] Frolin S. Ocariza, Karthik Pattabiraman, and Ali Mesbah. Detecting inconsistencies in JavaScript MVC applications. In *Proceedings of the 37th IEEE/ACM International Conference on Software Engineering (ICSE)*, pages 325–335. IEEE, 2015.

[56] Naito Oshima and Tomoji Kishi. Model checking method for SPA page transition based on component-based framework. In *Proceedings of the International Conference on Software Engineering and Knowledge Engineering (SEKE 2018)*. KSI Research, 2018.

[57] Mike Papadakis, Marinos Kintis, Jie Zhang, Yue Jia, Yves Le Traon, and Mark Harman. Mutation testing advances: An analysis and survey. In *Advances in Computers*, volume 112, pages 275–378. Elsevier, 2019.

[58] Renaud Pawlak, Martin Monperrus, Nicolas Petitprez, Carlos Noguera, and Lionel Seinturier. SPOON: A library for implementing analyses and transformations of Java source code. *Software: Practice and Experience*, 46(9):1155–1179, 2016.

[59] Upsorn Praphamontripong and Jeff Offutt. Applying mutation testing to web applications. In *Proceedings of the 2010 Third International Conference on Software Testing, Verification, and Validation Workshops (ICSTW)*, pages 132–141. IEEE, 2010.

[60] Remix Run. React router documentation, version 8. `https://reactrouter.com/`, 2026. Version 8.4.0. Accessed 5 October 2026.

[61] Xiaoxia Ren, Fenil Shah, Frank Tip, Barbara G. Ryder, and Ophelia Chesley. Chianti: A tool for change impact analysis of Java programs. In *Proceedings of the 19th Annual ACM SIGPLAN Conference on Object-Oriented Programming, Systems, Languages, and Applications (OOPSLA)*, pages 432–448. ACM, 2004.

[62] Filippo Ricca and Paolo Tonella. Analysis and testing of web applications. In *Proceedings of the 23rd International Conference on Software Engineering (ICSE)*, pages 25–34, 2001.

[63] Gregor Richards, Francesco Zappa Nardelli, and Jan Vitek. Concrete types for TypeScript. In John Tang Boyland, editor, *29th European Conference on Object-Oriented Programming (ECOOP 2015)*, volume 37 of *Leibniz International Proceedings in Informatics (LIPIcs)*, pages 76–100, Dagstuhl, Germany, 2015. Schloss Dagstuhl – Leibniz-Zentrum für Informatik.

[64] Bachar Rima, Anas Shatnawi, and Abdelhak-Djamel Seriai. E2E test generation over a source-code-derived workflow space: An Angular case study. In *Software Engineering and Advanced Applications*, Lecture Notes in Computer Science,

pages 334–351, Cham, 2026. Springer Nature Switzerland. Published online 27 August 2026; print volume dated 2027. The method is described in this thesis from the publisher abstract and the replication package; the full text was not available.

[65] Guido Salvaneschi and Mira Mezini. Debugging for reactive programming. In *Proceedings of the 38th International Conference on Software Engineering (ICSE)*, pages 796–807. ACM, 2016.

[66] Guido Salvaneschi, Sebastian Proksch, Sven Amann, Sarah Nadi, and Mira Mezini. On the positive effect of reactive programming on software comprehension: An empirical study. *IEEE Transactions on Software Engineering*, 43(12):1125–1143, 2017.

[67] Manu Sridharan, Shay Artzi, Marco Pistoia, Salvatore Guarnieri, Omer Tripp, and Ryan Berg. F4F: Taint analysis of framework-based web applications. In *Proceedings of the 2011 ACM International Conference on Object Oriented Programming Systems Languages and Applications (OOPSLA)*, pages 1053–1068. ACM, 2011.

[68] Stryker Mutator contributors. Stryker mutator: Mutation testing for JavaScript, TypeScript, C# and Scala. `https://stryker-mutator.io/`, 2026. Accessed 2026-10-21.

[69] Oksana Tkachuk and Sreeranga P. Rajan. Method and system to extract a navigation model for analysis of a web application, 2013. US Patent 8,402,319 B2, Fujitsu Limited.

[70] Chenxu Wang, Tianming Liu, Yanjie Zhao, Minghui Yang, and Haoyu Wang. LLMDroid: Enhancing automated mobile app GUI testing coverage with large language model guidance. *Proceedings of the ACM on Software Engineering*, 2(FSE):1001–1022, 2025.

[71] Junjie Wang, Yuchao Huang, Chunyang Chen, Zhe Liu, Song Wang, and Qing Wang. Software testing with large language models: Survey, landscape, and vision. *IEEE Transactions on Software Engineering*, 50(4):911–936, 2024.

[72] Siyi Wang, Sinan Wang, Yujia Fan, Xiaolei Li, and Yepang Liu. Leveraging large vision-language model for better automatic web GUI testing. In *Proceedings of the 2024 IEEE International Conference on Software Maintenance and Evolution (ICSME)*, pages 125–137. IEEE, 2024.

[73] Hao Wen, Yuanchun Li, Guohong Liu, Shanhui Zhao, Tao Yu, Toby Jia-Jun Li, Shiqi Jiang, Yunhao Liu, Yaqin Zhang, and Yunxin Liu. AutoDroid: LLM-powered task automation in Android. In *Proceedings of the 30th Annual International Conference on Mobile Computing and Networking (MobiCom)*, pages 543–557. ACM, 2024.

[74] Rahulkrishna Yandrapally and Ali Mesbah. Fragment-based test generation for web apps. *IEEE Transactions on Software Engineering*, 49(3):1086–1101, 2023.

[75] Hong Yul Yang, Ewan Tempero, and Hayden Melton. An empirical study into use of dependency injection in Java. In *Proceedings of the 19th Australian Conference on Software Engineering (ASWEC)*, pages 239–247. IEEE, 2008.

[76] John Yang, Carlos E. Jimenez, Alexander Wettig, Kilian Lieret, Shunyu Yao, Karthik Narasimhan, and Ofir Press. SWE-agent: Agent-computer interfaces enable automated software engineering. In *Advances in Neural Information Processing Systems 37 (NeurIPS 2024)*, pages 50528–50652, 2024.

[77] Shengqian Yang, Haowei Wu, Hailong Zhang, Yan Wang, Chandrasekar Swaminathan, Dacong Yan, and Atanas Rountev. Static window transition graphs for Android. *Automated Software Engineering*, 25(4):833–873, 2018.

[78] Shengqian Yang, Dacong Yan, Haowei Wu, Yan Wang, and Atanas Rountev. Static control-flow analysis of user-driven callbacks in Android applications. In *Proceedings of the 37th International Conference on Software Engineering (ICSE)*, pages 89–99, 2015.

[79] Shengqian Yang, Hailong Zhang, Haowei Wu, Yan Wang, Dacong Yan, and Atanas Rountev. Static window transition graphs for Android. In *Proceedings of the 30th IEEE/ACM International Conference on Automated Software Engineering (ASE)*, pages 658–668, 2015.

[80] Evan You and Eduardo San Martin Morote. Vue router documentation, version 5. `https://router.vuejs.org/`, 2026. Version 5.3.1. Accessed 5 October 2026.

[81] Tao Zhang, Fuzhong Hao, Yunfan Wang, Bo Zhang, and Guangwei Xie. VeriFlow: A framework for the static verification of web application access control via policy-graph consistency. *Electronics*, 14(18):3742, 2025.

[82] Carlos Zimmerle, Kiev Gama, Fernando Castor, and José Murilo Mota Filho. Mining the usage of reactive programming APIs: A study on GitHub and Stack Overflow. In *Proceedings of the 19th International Conference on Mining Software Repositories (MSR)*, pages 203–214. ACM, 2022.

A Corrections made to the protocol

Table A1 lists the corrections in chapter order, and the repository records their commit hashes. *Figure* marks an implausible or unwelcome result, *panel* a red-team panel, *review* an assistant-supported source review, *harness* a mutation-harness change, and *end-to-end check* an extractor gap exposed by the suites in Section 5.3.5, then repaired with a regression test. Reviews and panels used the declared AI assistant. Every reported defect was reproduced before repair.

Table A1: The corrections made to the instruments.

Found by	What was wrong	What changed	Effect on the figures
Protocol and harness			
One application's recall fell when the rule was tightened.	Test-source cross-check: a URL parameter in a test could match a literal route segment.	A parameter no longer matches a literal segment. A negative test is excluded.	The earlier recall cannot be regenerated. End-to-end recall there is now 71.4 %.
Figure: inspecting the flagged edges.	A low witness-check pass rate came entirely from may edges at one over-approximated call site.	Over-approximated sites are reported separately. The extractor is unchanged.	The low rate was a measurement artefact. No must edge failed.
Harness: not stated beyond one application's misses.	Mutants with targets absent from the baseline graph repeat recovery gaps counted elsewhere.	These mutants are classified separately.	Most of that application's misses dropped out of the detection rate.
Figure: a construct count that tripled.	Substring matching counted `routerLinkActive` as `routerLink`.	The independent route parser counts constructs, giving the ratio and loss breakdown one definition.	alfresco: the independent route parser finds 34, about a third of the substring count.
Figure: coverage on mifos reached 100 per cent.	The independent route parser skipped wrappers, stopped before lazy modules' routing modules, and lost lazy-table prefixes.	It follows wrappers and routing modules, retaining prefixes.	mifos has 610 declared screens, 58 with a node. The extractor was the same in both runs.
Figure: on mifos, a ratio rose implausibly after attribution.	Must conversion originally divided must edges by constructs.	Conversion is counted per construct. Subtree-scoped edges are stored once (Section 3.2).	Edge storage inflated the rate without adding evidence. Section 4.2 defines conversion.

Found by	What was wrong	What changed	Effect on the figures
Harness, *review*, *figure*: a harness review and a detection-rate shift exceeding twenty points.	The mutation harness patched the first textual match, judged detection on node and edge sets alone, and retargeted only the head of a commands array.	Patches use construct offsets, detection includes witnesses, and retargeting selects another declared route. Guards without incoming edges are excluded. Renames require the renamed route to appear.	realworld's detection rose sharply. The seed stayed fixed, but the sample changed from 70 to 73 mutants, of which 46 share an earlier mutant's location: the same operator, file and line, though destinations may differ. The final sample omits 6/9 earlier misses. Of six earlier paired disagreements favouring attribution, five used retargeting that replaced only the commands-array head. Correcting it also lets the baseline detect them.
Harness: reading the graph at the guard mutants the exclusion rule kept.	The rule that excludes guard mutants located a guard's route by the nearest preceding path literal, which fails for children, empty paths, constants and template literals.	Brace matching after blanking comments and strings locates the route. Routes without screens are excluded.	Four unobservable mutants, previously counted as graph defects, are now excluded. The superseded run sampled 70 mutants.
Harness: prompted by an extractor change rather than a figure.	Guard removal was misjudged: a guarded parent's child lost the parent guard, and set comparison hid changes among three identical guard texts.	Guard multiplicities are counted. An edge losing a guard is detection, whereas absence from all baseline edges means exclusion.	Scoring changes, but the sampled mutants do not.
A table script stopped.	The construct denominator included constructs from commented-out links and code comments.	Comments are blanked before parsing, with the compiler's scanner for TypeScript.	One construct leaves the denominator in each of bitwarden and mifos. Nothing else changes.
The extractor and the deterministic tier			
Panel: red-team panels reading the graph and source after commit f17a302 found what the witness check missed.	A guard on a wrapper-returned route was omitted, making all descendants unconditional.	Screen guard sets include function-returned routes.	The results of commit f17a302 record 1024 must edges with 26 witness failures. Reading the guard cut mifos to 1 on the intermediate version.

Found by	What was wrong	What changed	Effect on the figures
Panel, as above.	Calls in `canActivate`, including guard factories, were skipped.	Unresolved elements still gate edges.	bitwarden's deterministic edges into its root are no longer unconditional.
Correcting the witness check itself.	It checked only the first construct producing an edge, so it missed that later bitwarden citations targeted unresolved `/vault`.	It checks every cited site. Edges initially took the weakest cited modality (replaced below).	Later citations are 11.6 % of all witness citations, and up to 55.1 % in one application. The intermediate tier held 20 must edges: 20/20 passing, 0 failing.
Review, then *panel*, of all intermediate-tier constructs: 20 edges, 30 distinct constructs, 36 citations. Witness checking compares targets, not unconditional execution.	The demotion rules were incomplete: a link under `*ngIf`, inside `@if`, a callback or a `catch`, or after `await` or an early return was must.	Demotion covers Section 3.1's conditions. Panels added members without unconditional handler calls, conditional component placement, ICU cases and ambiguous routes. Later review added potentially superseding navigation.	32/36 citations ran under such a condition.
Review, as above.	A guard on an empty-path child without a component was not read, and `canMatch`, `canActivateChild` and `canLoad` guards not at all.	Such a guard gates the screen above. The other three kinds are read.	The current graph reads a guard, missed before, on 10 of the intermediate tier's edges: every bitwarden edge of that tier and none from another application.
Panel: a red-team panel.	Gating on every destination guard also demoted navigations the router permits.	The extractor computes the guards the router runs for each source and destination: levels left, matched, entered, kept.	Navigations between screens under a shared guarded parent pass, and mifos again holds 24 must edges.
Review, as above.	Navigation to the current address was treated as occurring on every execution.	Such an edge is drawn per screen: must from the others, a may self-loop at the destination.	1 of the intermediate tier's edges is such a self-loop.
Reading the edges that survived.	Weakest-modality merging discarded certainty, as with realworld's unconditional Global Feed link.	Edges take their strongest construct's modality. Other witnesses keep theirs. Must checking uses only witnesses that earn must.	The final tier has 159 edges on 25 constructs, a set not contained in the intermediate tier. Current conversion counts require each construct to earn must, whereas superseded tables credited every cited construct.

Found by	What was wrong	What changed	Effect on the figures
The independent route parser and the denominators			
First node-audit disagreements: the extractor was scored against paths the independent route parser could not resolve.	The parser missed constant, enum and `$localize` paths; first-party `baseUrl` imports; and named route arrays.	It resolves all three forms.	Without the last repair, it missed almost all kubernetes/dashboard screens and labelled nearly every extracted node unmatched.
The dead-link corrections.	It included Storybook fixtures and a sibling application's aliased tables but failed to resolve a path in an `Object.freeze` table.	It excludes fixtures and sibling tables and resolves the frozen table. Excluding frozen routes as unreadable was rejected: they were one call away.	Declared sets changed in 5/6 applications, with counts unchanged in 1.
Fixing the provenance of the comparison.	An interim parser's output was mistaken for earlier output. The script also treated missing files as unchanged denominators.	It uses the earlier parser's committed output.	The resulting bitwarden decomposition is withdrawn.
Resolving the first node audit's disagreement.	The first-audit extractor lost parent prefixes and created a * node per catch-all.	The loss-list repairs: a table not reached, a path or destination named but not written (Section 5.2).	Of 143 nodes with no declared route, 138 were the tail of a declared route and 5 catch-alls. The final version has 2 nodes matching no declared screen, 2 at *.
Not stated. Classification followed graph inspection.	Denominator exclusions were tested after consulting the graph.	Exclusions are decided from source text first. Comparison rejects changes between runs.	Attribution had enlarged the denominator. Both columns now use the same 1775 sites.
Findings of the runtime route oracle			
The oracle found two alfresco addresses served by the catch-all. Both static readers made the same error, hidden by 100 per cent coverage.	A named-outlet route could inherit a sibling's primary address. Descendants also received primary-address nodes.	Traversal preserves outlet context and creates no screen beneath a named outlet.	Extracted screens at unmounted addresses: 0. Remaining gaps limit recall.
Comparing the oracle with both static route sets. Each static reader had acquired the defect independently.	Lazy modules reached through barrels were followed only one hop. Their tables were scanned at the root.	Both follow bounded chains, rejecting at each hop the route's own file, files loaded by other routes, and files with root-registered tables.	10 addresses in alfresco, bitwarden, including the two alfresco addresses, were wrong. Static node coverage had been 100 per cent against the same wrong addresses.

Found by	What was wrong	What changed	Effect on the figures
Instrumenting the predicate (commit `d40c8dc`).	The root-table exclusion predicate recognised `RouterModule.forRoot` and never fired in the corpus.	Both tools mark root tables per array, from the argument of either registration API.	The inactive predicate had been credited with an untested property.
The dead-link census			
Refutation attempts on the dead-link finding.	An earlier correction followed only the redirecting kind of catch-all route.	Both kinds are followed. Edges remain may and record the catch-all destination.	The original correction worked (✓) on its development application but failed (✗) on two using the other kind.
Refutation attempts.	Target resolution was conflated with dead-link detection. An unresolved enum sent all bitwarden authentication paths through the wildcard.	The enum and wrapper are resolved. Lower-bound route sets are noted. Caveated candidates are checked against runtime configuration.	Wildcard matching had raised bitwarden's recovered figure. Checked against the framework's route configuration, dead-link candidates were withdrawn: 23 where the framework serves a screen, 19 where the matching route places no screen and 0 where a redirect answers. 0 keep the caveat.
Refutation attempts.	Unresolved routes were treated as undeclared: a spread-built array, ghostfolio's `matcher` and alfresco's tsconfig `paths` alias.	Both tools record unresolved regions and exclude undeclared claims within them.	ghostfolio's `/admin/users` and two live alfresco links had been called dead. All three constructs are now resolved.
Refutation attempts.	The route-set test treated a commands array's first quoted segment as its complete target.	Whole-array patterns, with computed segments as holes, are matched against route patterns.	alfresco's two candidates were cleared and more mifos sites found. The census contains 29 candidates.
Mutation sweep and end-to-end check			
Mutation sweep of the demotion rules, at commit c8eec9b.	The final revision's first sweep found untested rules added by router repairs.	Tests cover most survivors, and observationally equivalent mutants were retired.	Of 20/164 survivors, 18 gained tests and 2 were retired. At a9d29df, all 174 deletions fail a test. Testing also exposed relative links climbing above the root.
Panel: a review panel checking the scorer against the preregistration.	Redirects were scored at landed addresses, contrary to preregistration.	The scorer applies the preregistered rule.	mifos then scored 12/22 pairs, 5 of them redirected. The landed-target figure had been 15/22.

Found by	What was wrong	What changed	Effect on the figures
End-to-end check: mifos's suite missed transitions to action screens.	A commands-array template literal (`navigate([ \`actions/ ${name}\`])`) was treated as an unresolved variable.	Static text is retained. Interpolated segments become parameters.	The client, group and savings views reach their action screens.
Figure: after that repair, the re-measured mifos graph joined every pair of screens.	Truncation at the first parameter emptied the notifications tray's two-parameter prefix, connecting its shell site to every route.	Only trailing parameters are dropped from a target prefix. A parameter inside the prefix matches any one segment.	The tray reaches its named account screens. Graph pairs: 28185 before repair, 162567/367236 after.
End-to-end check: the same.	Unresolved `relativeTo` in an options variable made loan-view actions root-relative.	A `const` initialised with an object literal is read as that literal.	The loan view reaches its action screens.
End-to-end check: mifos's suite missed tab transitions.	A parent-template link rendered with child tabs was resolved against each child, producing unmounted targets.	Resolution uses the route rendering the component (Section 3.2.6), as does witness checking in harness 1.10.0.	With the preceding two repairs, mifos suite hits rose from 12 to 19/22. All extractor-dependent figures were re-measured.
Comparison on Rima et al.'s subjects (Section 5.3.7), after recording its frozen figures			
Comparison: petclinic's `navigate([ '/owners', id])` reached the owner's edit and pet screens.	A commands array ending in a bound value was read as a prefix, although Angular splits only the first command on slashes.	A command after the first is one segment, so the array is a pattern. A matrix-parameter object adds no segment.	The target pairs read as spurious are gone.
Comparison: traduora's locale link produced no transition.	A single bound command, `[routerLink]= "[l.code]"`, produced an unknown target.	It becomes one segment below the declaring route, as already assumed for first-command interpolation. Some corpus bindings violate this assumption: mifos's `[cancelRoute]` can be `../../`. Pair counts were unchanged.	The locale list reaches the translation screen.
Comparison: airbus's navigation bar was placed on its sign-in screen.	An unrendered outlet holder and its navigation bar were mistaken for the shell.	If the bootstrapped root renders an outlet holder, unrendered holders are excluded from the shell.	The bar is placed on the pages that include it.

Found by	What was wrong	What changed	Effect on the figures
Comparison: airbus's router-service call from its dialogs was placed on the sign-in screen.	A class-opened component (`dialog.open(X)`) had no screens. A service member called only there therefore inherited every injecting component's screens.	Class-named opening (`open`, `createComponent`, `ComponentPortal`) creates a nesting relation to the opener.	With the preceding three repairs, Rima reference hits rose from 68 to 69/73, and other pairs changed from 357 to 341. All extractor-dependent figures were re-measured.

Note. Found by names the method as the main text does.

Except for the mifos graph that joined every pair of screens, results marked *figure* exposed measurement errors rather than extractor defects.

B Declared approximations

Table B1: Declared approximations and exact-read constructs, and what the graph loses for each.

Construct	How it is treated	Consequence (tier, missing edges, note)
Found by the runtime route oracle		
Route declared outside the language	alfresco registers its about page through a JSON path and identifier, bound to a component by the runtime extension registry. TypeScript parsing cannot recover it, and supporting the plugin format as a second source language lies beyond this thesis.	The route is absent without a note: no TypeScript construct is unresolved.
Table chosen outside the sources	mifos selects between two users-module route tables, one per identity provider, using a constant derived from deployment-injected `window.env`. Source-fixed conditions, such as environment-file flags, select an arm, whereas unresolved conditions, as here, mark the subtree unread. Extraction uses checked-out environment values. Production values may select differently. The oracle likewise uses a development build to measure the same configuration.	Declared unread subtree, 5 screens wide.
Read exactly: the analysis's responsibility		

Construct	How it is treated	Consequence (tier, missing edges, note)
Literal routes, links, calls, nesting, outlet, guards	The analysis reads these exactly. String-literal route paths are read together with their components. A `routerLink` binding is read when its value is a literal or an array of literals. `Router.navigate` and `Router.navigateByUrl` calls are read when their target is a literal. Nesting comes from element selectors, resolved through the imports of the rendering file when a selector is ambiguous. The primary `<router-outlet>` is read together with the shell rule that follows from it. Guards named on a route become labels on the edges entering that route's subtree.	Failures are defects, not approximations, counted in Chapter 5: constant-path, lazy-boundary and unlinked-node bugs (Section 5.2), resolved in the final version (Section 5.5).
Approximated: no completeness claimed		
Components created programmatically (dialogs, overlays)	Unrouted overlays have no screen nodes. Components or directives opening a class in their own file (`open(X)`, `createComponent(X)`, `new ComponentPortal(X)`) supply their screens through nesting (Table A1). Service-opened or unresolved constructions have empty scope. Attribution gives these navigating components file-labelled unplaced origins, with *may* edges from their sites (Section 3.1).	The graph omits overlay states, entry/exit transitions and, for empty scopes, the screens from which exits fire, but exit destinations remain represented. Navigation in a service injected only by such a component produces only a note.
Named outlets and `{outlets: ...}` targets	An auxiliary route adds a second active route and mount point, encoded as `(name:path)` [27]. Chapter 3 models a second mount, but the adapter omits it. Non-primary-outlet routes, their descendants and routes with unresolved outlet strings create no nodes. `navigateByUrl` drops auxiliary groups and marks primary-path edges *may*, because an unmatched auxiliary route would prevent navigation.	Unresolved outlet strings contribute to the incomplete-route-set note. Commands containing `{outlets: ...}` produce neither edges nor notes, even when the group includes the primary outlet. Later string commands are single segments, so auxiliary URLs there match only as single segments. The test-source cross-check misses such URLs.
Navigation relative to the active route (`relativeTo`)	Template `routerLink`, and `Router.navigate` with a `relativeTo` key in an inline or `const` options object, resolve against the route rendering their component on the source screen's ancestor chain (Section 3.2.6), applying leading `.` and `..` segments. Calls without `relativeTo`, and `navigateByUrl`, resolve against the root.	Edges remain *may* when the chain has no rendering route (template-nested components), `relativeTo` is shorthand, uses other variables or names another route, a link binds `[relativeTo]`, or the source is the shell. They may name wrong screens, which must forbids.

Construct	How it is treated	Consequence (tier, missing edges, note)
Templates passed as values (`TemplateRef`)	Template values are not traced to instantiation. Links in `<ng-template>` are attributed to the declaring component's screens.	Links remain *may*: neither instantiation nor its location is established. Passing templates to other screens does not move their edges. Recovering instantiation requires dataflow analysis, as for JSX (Section 6.4).
Dispatch through NgRx effects and router event subscriptions	Chapter 3's dispatch relation is unimplemented, leaving dispatching screens unresolved. At *full* and *class*, navigation calls in application files containing `createEffect` receive $\top$: *may* edges from the shell, or edges into its `u_` sink for fully dynamic targets. Lower levels omit this stand-in.	Sound and coarse, since an effect can fire on any screen. An effect outside the application directory, or declared with the older `@Effect` decorator, is treated like any service call.
Navigation inside signal effects and observable subscriptions	Firing depends on a runtime dependency graph rather than textual control flow (Sections 2.7.3 and 2.8). The model assigns *unknown* with a symbolic read-set guard, but neither is implemented. Navigation in an `effect`, `subscribe` callback or other called function receives its class's scope (Section 5.5).	Conditional invocation keeps edges at *may*. Fully dynamic targets enter the `u_` sink. The graph identifies possible exits without their firing conditions.
Computed targets, no static part	Runtime-built targets cannot generally be enumerated statically. Targets without static parts, including interpolated `routerLink` values, produce edges into their source node's `u_` sink.	Modality *unknown*, confidence 0.
Computed targets, static leading part	Prefixes come from template-literal heads, concatenation's left operands, or commands arrays up to trailing variable elements. Earlier variable elements and interpolations in an opening template literal each represent one segment. Edges connect to every declared route extending the prefix by at least one segment. An empty prefix reaches every route except the root.	*May*, with incomplete over-approximation: static prefix segments require identical route segments, excluding routes with parameters at those positions.
Computed targets, commands array with trailing constants	An array with constants after a variable element becomes a pattern with one edge to its first matching route.	*May*. Variable elements match only parameter segments, silently omitting routes with constants in those positions.

Construct	How it is treated	Consequence (tier, missing edges, note)
Injection tokens and provider bindings	The call graph resolves field calls through declared types (Section 3.3). Bindings are application-wide: different providers for one token are joined rather than resolved by injector. Factory/value providers and externally returned provider arrays are unsupported. Calls fall back to all components injecting the class, or empty scope if none does.	Joins only widen scope. Unresolved tokens add no edges. The corpus has 5574 calls resolved directly through application types, 4937 through provider bindings and 18 through subclasses, with 1111 unresolved across 2 of six applications.
Configuration-driven route registration	Array tables passed to `RouterModule.forRoot`, `forChild` or `provideRouter` are parsed wherever registered, including application-called provider factories. Routes created by startup configuration are omitted. Declared route objects with configuration-supplied `path` produce unresolved-path records without nodes.	Unresolved paths mark the route set as a lower bound. Function-computed tables, `ROUTES` providers, `Router.resetConfig` calls and JSON extension manifests (Section 5.5.3) silently omit routes without a lower-bound note. The independent route parser behaves identically, so both agree on omissions. This approximation also caused one corpus rejection (Section 4.1).
Guards that would let the navigation through	State-dependent guards are not evaluated. Expression text identifies them, so equivalent expressions remain distinct. Projected `<ng-content>` follows its host template's placement. Only `mat-menu` and `mat-tab` are recognised as lazy containers. Conditional outlets demote their route components, but nested components use their own placement. The root outlet is not inspected.	Any executed guard keeps an edge at *may*, even if it always passes. Conditional rendering inside a projection recipient does not demote projected links.
Navigations a later one may cancel	Claim 3.1's execution model excludes this case and modifier-key activation. Code that may start another navigation before event handling completes demotes the earlier one (Section 3.1). Calls are followed to application bodies: class members, file-level functions or methods resolved through typed receivers.	Library calls, untyped receivers and runtime-loaded code are assumed not to navigate, potentially leaving cancelled navigation at must. Scheduled navigation and other listeners for the event are not followed.
Empty-path children of a route that a function returns	The first empty-path child under a screen and its first empty-path child render at the screen's address, so their links originate there. For function-returned routes such as `withShell([...])`, traversal does not place the returned child components under the enclosing screen.	Links remain at unplaced origins with *may*. Screen edges are missing, without false assertions.

Construct	How it is treated	Consequence (tier, missing edges, note)
Activation with a modifier key	Anchor `routerLink` leaves modifier-key and non-primary clicks to the browser, opening another browsing context, as does a `target` naming another context. The model treats all activation as application navigation.	Must applies to plain activation and excludes these cases.
Order in which Angular matches routes	Angular matches routes and redirects in declaration order: an earlier parameter route can shadow a literal route. The extractor instead prefers exact paths. Separately registered tables follow provider order, which is unresolved. The extractor assumes their catch-alls follow all other tables' routes and records each dismissed catch-all in a note.	Overlapping routes or redirects keep edges at *may*, potentially naming Angular's unchosen route. The catch-all exception is the route-order assumption. Section 3.1 counts the must edges relying on it, with Claim 3.1.
Ambiguous selectors in the nesting relation	Shared selectors are narrowed to components the rendering file imports. If it imports none, all candidates are joined.	A note records the ambiguity. Sites may be attributed to screens that never render their components.

Note. "May" is the tier without a soundness claim.